

Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Desarrollo y Operaciones (DevOps)

GitOps Multicloud: Implementación de Pipelines CI/CD con Infraestructura como Código Agnóstica para la Portabilidad de Microservicios aplicado al Servicio de Rentas Internas.

Trabajo fin de estudio presentado por:	- Darwin Augusto Calle Loja - Juan Carlos Cela Palacios - Luis Patricio Cuascota Tanicuchi
Tipo de trabajo:	Desarrollo Práctico
Director/a:	Josefina Guerrero García
Fecha:	22/07/2026

Resumen

El Servicio de Rentas Internas (SRI) de Ecuador opera sobre una infraestructura 100% on-premise: servidores físicos con RedHat Enterprise, aplicaciones monolíticas en JBoss EAP (JEE6-JEE8) y dependencia de costosas licencias de software privativo. Este modelo tradicional genera despliegues manuales, silos departamentales, un Lead Time for Changes superior a 5 días hábiles y una tasa de fallos en cambios (CFR) del 18% mensual.

El objetivo de este Trabajo Fin de Estudio es diseñar, implementar y evaluar una arquitectura GitOps multicloud con infraestructura como código agnóstico que garantice la portabilidad automatizada de al menos tres microservicios entre AWS EKS y Azure AKS. La solución emplea exclusivamente herramientas de código abierto: Terraform para la provisión de infraestructura, GitHub Actions para integración continua, ArgoCD en modo ApplicationSet para entrega continua multicluster, y Helm con Kustomize para empaquetado diferenciado.

La investigación sigue la metodología Design Science Research (DSR) y se estructura en cuatro fases iterativas. Como resultado de la fase experimental, se demuestra una reducción del Lead Time for Changes de más de 5 días hábiles a una mediana de 255 segundos (una mejora del 99,94%) y una frecuencia de despliegue que pasa de quincenal a bajo demanda (múltiples entregas diarias). Así mismo, se optimiza la estabilidad operativa al reducir la tasa de fallos en cambios (CFR) del 18% al 0% y comprimir el tiempo medio de recuperación (MTTR) de más de 4 horas a una mediana de 13,5 segundos, garantizando la portabilidad completa de los microservicios sin alterar su código fuente. La contribución final es una hoja de ruta validada, replicable y financieramente sostenible para modernizar el núcleo tecnológico del SRI, llenando un vacío identificado en la literatura sobre GitOps multicloud en el sector público latinoamericano.

Palabras clave: DevOps, GitOps, Multicloud, CI/CD, Infraestructura como Código, Microservicios, Métricas DORA.

Abstract

Ecuador's Internal Revenue Service (SRI) operates on a 100% on-premise infrastructure: physical servers with RedHat Enterprise, monolithic applications on JBoss EAP (JEE6-JEE8), and dependence on costly proprietary software licenses. This traditional model generates manual deployments, departmental silos, a Lead Time for Changes exceeding 5 business days, and a monthly Change Failure Rate (CFR) of 18%.

The objective of this Master's Thesis is to design, implement, and evaluate a GitOps multicloud architecture with agnostic infrastructure as code that guarantees automated portability of at least three microservices between AWS EKS and Azure AKS. The solution exclusively uses open-source tools: Terraform for infrastructure provisioning, GitHub Actions for continuous integration, ArgoCD in ApplicationSet mode for multicluster continuous delivery, and Helm with Kustomize for differentiated packaging.

The research follows the Design Science Research (DSR) methodology and is structured in four iterative phases. As a result of the experimental phase, a drastic reduction in the Lead Time for Changes is demonstrated, dropping from over 5 business days to a median of 255 seconds (a 99.94% improvement), and deployment frequency transitions from biweekly to on-demand (multiple daily deliveries). Furthermore, operational stability is optimized by reducing the Change Failure Rate (CFR) from 18% to 0% and compressing the Mean Time to Restore (MTTR) from over 4 hours to a median of 13.5 seconds, ensuring complete portability of the microservices without modifying their source code. The final contribution is a validated, replicable, and financially sustainable roadmap for modernizing the SRI's technological core, filling an identified gap in the literature on multicloud GitOps in the Latin American public sector.

Keywords: DevOps, GitOps, Multicloud, CI/CD, Infrastructure as Code, Microservices, DORA Metrics.

Índice de contenidos

1. Introducción 11
1.1. Justificación del trabajo 12
1.2. Planteamiento del problema 13
1.2.1. Problema Organizacional 13
1.2.2. Problema Técnico 15
1.2.3. Pregunta de Investigación 15
1.2.4. Hipótesis de Solución 16
1.3. Estructura de la memoria 16
2. Contexto y estado del arte 17
2.1. Contextualización y antecedentes 17
2.1.1. El Servicio de Rentas Internas 17
2.1.2. El equipo tecnológico del SRI 18
2.1.3. El ámbito DevOps 21
2.1.4. Herramientas DevOps 28
2.1.5. Estrategias de nube 42
2.1.6. Design Science Research (DSR) 45
2.2. Trabajos relacionados 45
2.2.1. Infraestructura como Código en Entornos Multicloud 45
2.2.2. GitOps para Kubernetes 47
2.2.3. Microservicios, Contenerización y Portabilidad 49
2.2.4. Transformación Digital en la Administración Tributaria de Ecuador 50
2.2.5. Análisis Comparativo de los Trabajos Relacionados 51
2.3. Conclusiones del estado del arte 57

3. Objetivos y metodología de trabajo 58
3.1. Objetivo general 59
3.2. Objetivos específicos 59
3.2.1. Objetivo Específico 1: La Plataforma Base (Infraestructura como Código) 59
3.2.2. Objetivo Específico 2: El Motor GitOps (CI/CD y Empaquetado) 59
3.2.3. Objetivo Específico 3: Bucle Feedback (Observabilidad y Validación DORA) 60
3.3. Metodología del trabajo 60
3.3.1. Paradigma de investigación: Design Science Research (DSR) 60
3.3.2. Fases del proceso de investigación 61
3.3.3. Herramientas y entornos de trabajo 61
3.3.4. Criterios de validación cuantitativa bajo ISO 25010 y Métricas DORA 62
4. Desarrollo específico de la contribución 69
4.1. Planificación / Análisis / Requisitos 69
4.2. Descripción del sistema desarrollado / Implementación 74
4.3. Evaluación Experimental y Análisis de Resultados 120
4.3.1. Evidencia de Configuración: ArgoCD ApplicationSet Multiclúster 120
4.3.2. Evidencia de Configuración: Pipeline de Integración Continua y DevSecOps 121
4.3.3. Resultados Cuantitativos de la Fase Experimental (Registro de Telemetría) 122
4.3.4. Discusión de Resultados y Contraste de Hipótesis 127
5. Conclusiones y trabajo futuro 129
5.1. Conclusiones 129
5.2. Líneas de trabajo futuro 130
Referencias bibliográficas 132
Anexo A. Instrumentos de Recolección de Datos (Línea Base SRI) 138
Anexo B. 141

Índice de figuras

Figura 1 Organigrama de la Dirección Nacional de Tecnología del SRI. 18

Figura 2 Ciclo de vida Tradicional. 22

Figura 3 Ciclo de vida DevOps. 22

Figura 4 Marco de Investigación DSR 73

Figura 5 Etapas de Implementación. 73

Figura 6 Flujo de Trabajo. 74

Figura 7 Arquitectura GitOps multi-cloud con sincronización descentralizada.. 80

Figura 8 Estructura de Proyectos para la solución gitops multi-cloud. 82

Figura 9 Arquitectura C4: Nivel Contexto del Sistema GitOps Multicloud. 84

Figura 10 Arquitectura C4: Nivel Contenedor del Sistema GitOps Multicloud. 86

Figura 12 Estructura del proyecto para la solución gitops multi-cloud. 90

Figura 13 Módulo genérico Kubernetes Clúster. 93

Figura 14 Módulo Terraform Aprovisionamiento Clúster AWS-EKS. 94

Figura 15 Módulo Terraform Aprovisionamiento Clúster Azure-AKS. 97

Figura 16 Diagrama de Aprovisionamiento IaC Agnóstico. 101

Figura 17 Visión general del pipeline CI/CD GitOps. 104

Figura 18 Configuración de PAT (Personal Access Token). 105

Figura 19 Path GitHub /.github/workflows/deploy.yml 106

Figura 20 Definición de Secrets. 109

Figura 21 Configuración de Kubernetes – ArgoCD. 111

Figura 22 Configuración de aplicación Python en ArgoCD. 112

Figura 23 Actualización código fuente python. 113

Figura 24 GitOps: Estrategia de Overlays con Kustomize. 116

Figura 25 Diagrama de secuencia del flujo completo GitOps CI/CD Agnóstico. 119

Índice de tablas

Tabla 1 Organización del Trabajo en grupo 9

Tabla 2 Comparación Ciclo de vida Tradicional vs DevOps 22

Tabla 3 Matriz de análisis comparativo de trabajos relacionados en el estado del arte 52

Tabla 4 Matriz de Infraestructura Declarativa y GitOps (IaC & CD) 53

Tabla 5 Matriz de Herramientas de Integración Continua (CI / DevSecOps) 54

Tabla 6 Matriz de Comparación Sistemática entre Enfoques. 55

Tabla 7 Fases del proceso de investigación 61

Tabla 8 Herramientas y entornos de trabajo 61

Tabla 9 Matriz de requisitos cuantitativos e indicadores verificables (ISO 25010) 63

Tabla 10 Requisitos Funcionales 71

Tabla 11 Requisitos No Funcionales 72

Tabla 12 Criterios de Evaluación 75

Tabla 13 Matriz de Decisiones Técnicas 76

Tabla 14 Matriz de Reutilización por componente 98

Tabla 15 Telemetría de la fase experimental de sincronización y autocuración multiclúster 123

Tabla 16 Agregación Estadística Descriptiva de Sincronización y Ciclo de Entrega (Segundos) 125

Tabla 17 Comparativa de Rendimiento DevOps: Línea Base vs. Solución Propuesta 127

Organización del trabajo en grupo

A continuación, se detallarán las distintas etapas en las que se ha dividido el trabajo entre los miembros del equipo, los objetivos perseguidos en cada una de ellas y los mecanismos de coordinación.

Distribución y estructura de la memoria

En el presente gráfico se muestra las responsabilidades que cada integrante del equipo ha desarrollado para el presente trabajo final de estudio.

Tabla 1 Organización del Trabajo en grupo

Organización del trabajo en grupo - Desarrollo de la memoria	
Apartado de la memoria	**Responsables**
Introducción	Darwin Calle
Contexto y estado del arte	Patricio Cuascota
Objetivos y metodología de trabajo	Juan Carlos Cela
Desarrollo específico de la contribución (se puede desglosar en Planificación, Descripción del sistema, Implementación, Evaluación)	Darwin Calle, Patricio Cuascota y Juan Carlos Cela.
Conclusiones y trabajo futuro	Darwin Calle, Patricio Cuascota y Juan Carlos Cela.

Fuente: Elaboración propia.

Objetivo del TFE desde el punto de vista de la adquisición de conocimientos

Desde la perspectiva de la asimilación y apropiación de conocimientos avanzados, el presente Trabajo Fin de Estudio trasciende la mera ejecución técnico-operativa para consolidarse como un esfuerzo multidisciplinar de alta complejidad. El proyecto integra de forma sistemática diversas áreas de especialización de la ingeniera de software y de plataformas, tales como el diseño de arquitecturas cloud y multicloud, la especificación de infraestructura como código

(IaC) mediante Terraform, la orquestación de contenedores a gran escala con Kubernetes y el despliegue continuo automatizado bajo el paradigma GitOps empleando ArgoCD. Cada una de estas tecnologías de la propuesta, no solo reside en la implementación técnica de cada herramienta, también constituyen áreas de especialización en sí mismas, lo que permite la sinergia conceptual requerida para su correcto acoplamiento y gobernanza.

Al unificar estas disciplinas bajo la cultura de DevOps y los estándares declarativos de GitOps, se articula una visión holística y sistémica que facilita la transferencia efectiva del conocimiento conceptual hacia un entorno empírico altamente riguroso. Este enfoque permite resolver problemáticas institucionales complejas, confiriendo al proyecto una doble relevancia: una contribución científica al estado del arte y un entorno de valor tangible en el ámbito de la administración pública. En consecuencia, el resultado final no solo constituye un aporte académico robusto, sino que se materializa en una solución tecnológica madura y directamente aplicable a las operaciones del Servicio de Rentas Internas. Este proceso dota a los maestrandos de competencias técnicas, analíticas y de diseño de última generación, alineadas con las demandas más exigentes del mercado laboral tecnológico actual.

Mecanismos de coordinación empleados

El proyecto se desarrolla para prepararnos para el día a día en un mundo laboral tecnológico cada vez más complejo y dinámico. Actualmente, en el ámbito de la Ingeniería de Software y la administración de infraestructura, es necesario trabajar junto con otros profesionales de diversas disciplinas. La elaboración de este Trabajo de Fin de Máster bajo una modalidad grupal nos prepara para esta realidad colaborativa, exigiendo la adopción de prácticas ágiles y herramientas modernas de sincronización que son fundamentales en cualquier entorno empresarial real.

Para garantizar el éxito del proyecto y mantener la cohesión del equipo, implementamos mecanismos de coordinación estructurados y herramientas tecnológicas estándar de la industria. La gestión de tareas y la asignación de responsabilidades se llevó a cabo mediante metodologías ágiles SCRUM, utilizando tableros digitales como Trello lo que nos permitió dar seguimiento en tiempo real al avance de cada fase del proyecto.

En cuanto a la comunicación, combinamos las sesiones de trabajo conjunto, la revisión periódica y la toma de decisiones en equipo, utilizamos plataformas de videoconferencia. Las consultas diarias y el intercambio rápido de información se gestionaron mediante canales de mensajería instantánea, como Microsoft Teams.

En el ámbito técnico, el trabajo colaborativo sobre el código fuente y los manifiestos de Infraestructura como Código se orquestó a través de un repositorio centralizado en GitHub. Utilizamos flujos de trabajo de revisión de código, como Pull Requests, y control de versiones distribuido para asegurar la calidad, evitar conflictos y consolidar de manera ordenada el esfuerzo grupal.

1. Introducción

El Servicio de Rentas Internas (SRI) de Ecuador mantiene un modelo operativo caracterizado por una infraestructura íntegramente on-premise: servidores físicos locales con RedHat Enterprise y JBoss EAP, aplicaciones monolíticas en JEE6, JEE7 y JEE8, y una dependencia acumulada de licencias de software privativo que representa una carga presupuestaria creciente.

Migrar hacia infraestructuras cloud sin subsanar de raíz estas limitaciones estructurales únicamente desplaza el problema hacia un nuevo entorno operacional. De hecho, los marcos de transformación digital en administraciones tributarias de la región advierten que la adopción de tecnologías avanzadas sin una estricta gobernanza de datos y un diseño de arquitectura flexible impacta negativamente en la eficiencia operativa a mediano plazo, limitando los objetivos de sostenibilidad de la plataforma (Reyes et al., 2024). Bajo esta premisa, el verdadero hito técnico de la presente investigación no consiste en la simple adopción de la nube, sino en estructurar una solución donde la portabilidad de los microservicios sea nativa, permitiendo su alternancia dinámica entre proveedores sin incurrir en procesos complejos de reingeniería de software, con el objetivo de no

simplemente "ir a la nube", sino hacerlo de forma que los microservicios puedan moverse entre proveedores sin reescribir código.

El presente trabajo fin de máster, titulado ***"GitOps Multicloud: Implementación de Pipelines CI/CD con Infraestructura como Código Agnóstica para la Portabilidad de Microservicios aplicado al Servicio de Rentas Internas"***, diseña, implementa y evalúa una arquitectura GitOps basada en herramientas de código abierto —Terraform, Kubernetes y ArgoCD— para habilitar esa portabilidad entre AWS EKS y Azure AKS.

La investigación sigue la metodología Design Science Research (DSR) (Hevner et al., 2004), estructurada en cuatro fases: (i) análisis de la situación actual del SRI, (ii) diseño de la arquitectura multicloud, (iii) construcción de los pipelines CI/CD agnósticos, y (iv) medición del impacto en las métricas DORA.

1.1.Justificación del trabajo

La motivación de este trabajo es concreta: el SRI necesita salir de su infraestructura monolítica sin caer en la trampa del vendor lock-in. El problema es que Kubernetes no resuelve esto por sí solo. En la práctica, los proveedores cloud difieren en componentes críticos como Ingress controllers, StorageClasses y mecanismos de identidad (IAM en AWS frente a Managed Identities en Azure), dificultando la portabilidad real.

Para respaldar la justificación de este cambio y vincular la problemática identificada con la solución propuesta, se ha recurrido a la literatura científica y a casos de estudio del estado del arte que validan el impacto de la automatización. Estudios empíricos controlados sobre modernización de infraestructuras web utilizando Docker y Kubernetes demuestran que la transición desde procesos de despliegue tradicionales y manuales hacia pipelines optimizados permite alcanzar una reducción en el tiempo total de despliegue, un aumento en la frecuencia de entregas y la con un tratamiento de minimizar el total de las tasas de fallo en cambios o Change Failure Rate. Además, la integración de metodologías de Shift-Left Testing y automatización del pipeline (por

ejemplo, mediante análisis estático y CI continuo) acelera significativamente los tiempos de entrega de software, incrementando la mantenibilidad de los entornos (Jawalkar, 2020).

No obstante, en el ámbito de la gobernanza de despliegues e infraestructura declarativa, la literatura especializada en frameworks que integran GitOps y políticas automatizadas de cumplimiento reporta una notable caída en la necesidad de reversiones de código (rollbacks) en producción, la compresión de los tiempos medios de despliegue (Lead Time for Changes) y la casi erradicación de intervenciones manuales por parte del equipo de TI mediante flujos de trabajo auto-rectificativos (Beetz y Harrer, 2022, cap. 1; DORA, 2024, sección "Software Delivery Performance"). Desde el punto de vista financiero y de consumo de recursos computacionales, la gestión automatizada de capacidad de nube y el aprovisionamiento dinámico permiten una optimización de los costos operativos frente a esquemas estáticos, acelerando los flujos de aprovisionamiento de tiempos prolongados a fracciones de minuto (Renaldo, 2025).

Este diagnóstico operativo inicial del SRI, caracterizado por latencias extendidas en los tiempos de ciclo (LTC), ventanas prolongadas de recuperación ante incidentes (MTTR) y tasas críticas de fallos en producción (CFR), se justifica de manera sólida con la adopción de una arquitectura GitOps multicloud, vinculando directamente la fundamentación teórica con las urgentes necesidades de la institución pública ecuatoriana.

1.2. Planteamiento del problema

La problemática se descompone en tres niveles para estructurar la investigación.

1.2.1. Problema Organizacional

El SRI administra plataformas críticas como facturación electrónica y catastro sobre una infraestructura 100% on-premise, con servidores físicos gestionados de forma manual. Según estimaciones basadas en los costos de licenciamiento publicados por RedHat y los modelos de precios de AWS/Azure, las licencias de RedHat Enterprise y JBoss EAP representan una carga presupuestaria que, en escenarios de mediana escala, supera lo que costaría un modelo cloud híbrido con herramientas open-source.

Los indicadores operativos actuales, obtenidos a partir de una metodología descriptiva de reconstrucción de línea base, ubican al SRI en el nivel de bajo rendimiento según los estándares de la industria. Para la recolección de esta información de partida, se diseñaron y aplicaron tres entrevistas semiestructuradas dirigidas a roles clave de la Dirección de Tecnología del SRI: el Administrador de Servidores de Aplicaciones, el Jefe de Operaciones y el Release Manager. Los instrumentos detallados, guías de preguntas y el formato de estas sesiones se encuentran documentados formalmente en el **Anexo A. Instrumentos de Recolección de Datos (Línea Base SRI)** de esta memoria.

Con el fin de dotar de rigor metodológico al estudio, los datos cualitativos obtenidos en estas sesiones fueron cruzados y validados cuantitativamente mediante la auditoría física de las bitácoras de cambios registradas en los logs del middleware JBoss EAP y el histórico de tickets de incidentes de la Mesa de Ayuda durante el último trimestre fiscal (cuyo procedimiento de consolidación y análisis se describe formalmente en el *Procedimiento Experimental de Comparación de la sección 3.3.4.1*). Este proceso permitió estructurar de manera objetiva la siguiente línea base de rendimiento DORA de la institución:

- **Deployment Frequency:** menos de una vez por semana (frecuencia quincenal programada).
- **Lead Time for Changes:** más de 5 días hábiles (equivalente a 432.00 segundos de ciclo de trabajo manual).
- **Change Failure Rate (CFR):** aproximadamente 18% mensual.
- **Mean Time to Recovery (MTTR):** más de 4 horas (14.400 segundos) ante incidencias críticas en entornos productivos.

1.2.2. Problema Técnico

Tres obstáculos concretos bloquean la modernización del SRI:

1. **Silos organizacionales:** Los departamentos de Desarrollo, Operaciones, Seguridad y Planificación operan aisladamente.
2. **Acoplamiento arquitectónico:** El ecosistema actual del SRI depende de APIs propietarias de RedHat JBoss y arquitecturas monolíticas legadas (JEE6-JEE8). De acuerdo con una estimación institucional preliminar, respaldada por un análisis estático de dependencias y evaluación de adecuación a contenedores realizado con la herramienta Migration Toolkit for Applications (MTA) de Red Hat, se identificó que desvincular los servicios de la capa media privativa (ej. dependencias de EJBs acoplados, servicios de mensajería JTA/JMS heredados y descriptores de despliegue propietarios de JBoss) y adaptarlos a patrones declarativos agnósticos exigiría reescribir y reestructurar más del 80% de las líneas de código de la aplicación original para su portabilidad entre AWS y Azure. Este nivel de esfuerzo justifica la decisión arquitectónica de este proyecto de transicionar hacia microservicios desacoplados basados en un stack ligero open-source (Python/FastAPI).
3. **Ausencia de orquestación:** No existen pipelines CI/CD ni entrega declarativa de ningún tipo.

1.2.3. Pregunta de Investigación

Dado ese contexto, la investigación se formula así:

¿En qué medida la implementación de una arquitectura GitOps con herramientas agnósticas (Terraform, GitHub Actions, ArgoCD, Helm y Kustomize) permite: (1) mejorar de manera significativa las métricas Deployment Frequency y Lead Time for Changes respecto a la línea base del SRI, (2) garantizar la portabilidad de microservicios entre AWS EKS y Azure AKS sin modificar el código fuente, y (3) mantener una baja tasa

de fallos en cambios (Change Failure Rate) y un comportamiento resiliente durante la operación?

1.2.4. Hipótesis de Solución

La implementación de un ecosistema de despliegue continuo basado en Infraestructura como Código (IaC) agnóstica con Terraform, con la orquestación GitOps mediante GitHub Actions y ArgoCD y empaquetado modular, permite automatizar el ciclo de vida de los microservicios del Servicio de Rentas Internas hacia niveles de alto rendimiento establecidos por el marco metodológico DevOps Research and Assessment (DORA).

1.3. Estructura de la memoria

A continuación, se describe brevemente el contenido que se va a desarrollar en cada uno de los capítulos siguientes de esta memoria:

- **Capítulo 2. Contexto y estado del arte:** En este capítulo se abordará la situación tecnológica del Servicio de Rentas Internas (SRI), detallando el impacto de su infraestructura legada. Además, se explorarán los conceptos teóricos fundamentales sobre DevOps, GitOps y estrategias Multicloud, junto con un análisis de trabajos previos relacionados.
- **Capítulo 3. Objetivos y metodología de trabajo:** Aquí se definirán formalmente el objetivo general y los objetivos específicos propuestos para solucionar la problemática del SRI. Asimismo, se describirá la metodología de trabajo adoptada y el enfoque por fases iterativas para la construcción del nuevo ecosistema.
- **Capítulo 4. Desarrollo específico de la contribución:** Este capítulo contiene el núcleo técnico del trabajo. Se detallará paso a paso la implementación práctica: desde la planificación, el despliegue de Infraestructura como Código (IaC) con

Terraform, hasta la configuración de pipelines CI/CD y la orquestación en Kubernetes. Finalmente, se presentará la evaluación del sistema validando los resultados mediante métricas DORA.

- **Capítulo 5. Conclusiones y trabajo futuro:** En el último capítulo se expondrán las conclusiones obtenidas tras la implementación, evaluando el éxito del proyecto frente a la reducción de costos y silos organizacionales. Finalmente, se plantearán líneas de trabajo futuro para seguir evolucionando la arquitectura tecnológica del SRI.

2. Contexto y estado del arte

En este capítulo se presentarán los puntos para la comprensión del estado del arte, para lo cual se obtendrá una visión completa del resumen del conocimiento y de los trabajos relacionados, así como las conclusiones de este.

2.1. Contextualización y antecedentes

2.1.1. El Servicio de Rentas Internas

El Servicio de Rentas Internas con sus siglas SRI, es una institución pública de la República del Ecuador constituida el 2 de diciembre de 1997 para el fortalecimiento de una distribución justa y equitativa, que fomenta la cultura de pago de tributos y pone mucho esfuerzo en controlar la evasión fiscal, que estabiliza la economía del Ecuador y que enfrenta desafíos en pro de mejorar la recaudación de impuestos. Esta institución tiene sus políticas y normativas tributarias enmarcadas en los valores de equidad, integridad, transparencia, compromiso, innovación, confianza y adaptación al cambio (Servicio de Rentas Internas, 2025).

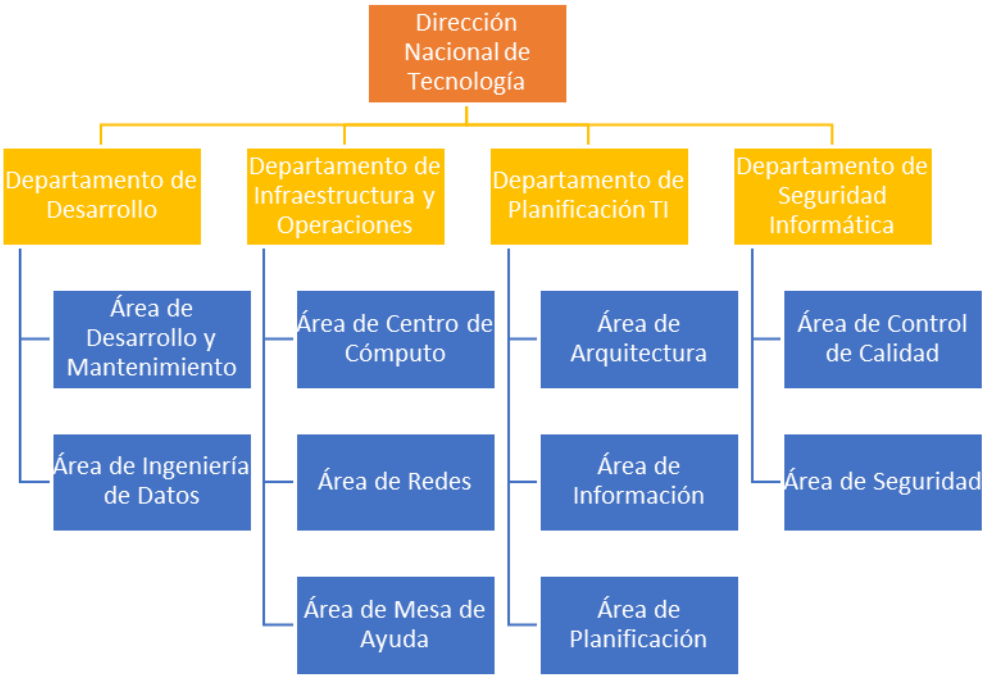
El SRI siempre está en constante cambio, tanto tributario como tecnológico, lo que ha llegado a ser una de las instituciones con gran prestigio en sector público; ha realizado investigaciones que han permitido a los contribuyentes tener las herramientas tecnológicas necesarias para el cumplimiento de las obligaciones tributarias, lo que ha determinado que posea una buena gestión económica (Ponomarev et al., 2018).

2.1.2. El equipo tecnológico del SRI

El personal de tecnología del SRI está compuesto de los departamentos que se muestran en la Figura 1, los cuales administran, configuran e implementan el parque tecnológico del SRI. Siendo el Departamento de Desarrollo el que tiene mayor personal en su nómina.

Uno de los problemas que se encuentra es que los departamentos de Desarrollo, Operaciones, Seguridad Informática y Planificación TI trabajan aisladamente, lo que involucra una mala comunicación entre ellos. Además, en los últimos 5 años no ha existido proyectos de innovación en la institución, lo que conlleva a la falta de creatividad e innovación, debido a que el personal de Desarrollo está ocupado realizando implementaciones de cambios normativos.

Figura 1 Organigrama de la Dirección Nacional de Tecnología del SRI.



Fuente: Elaboración propia.

2.1.2.1. Departamento de Desarrollo

Este departamento se encarga del análisis, diseño, codificación y pruebas ejecutadas en ambiente de desarrollo de nuevas aplicaciones institucionales, como puestas en producción, documentarlas y estabilizarlas. Maneja desde el levantamiento y consolidación de las soluciones informáticas en un catastro para el servicio de los contribuyentes y herramientas complementarias que utilizan las direcciones nacionales y zonales. Además, brinda el soporte técnico en la atención de los incidentes, cambios, problemas y requerimientos para el mantenimiento de las diferentes aplicaciones.

Dentro de Ingeniería de Datos, comprende el uso de modelos estructurados y no estructurados, así como los acuerdos de intercambio de información que incluye el levantamiento de especificaciones técnicas y administración del contrato en casos que se requiera una intervención externa.

2.1.2.2. Departamento de Infraestructura y Operaciones

El Centro de Cómputo realiza la gestión preventiva y correctiva que asegura la disponibilidad de la infraestructura de almacenamiento, procesamiento, respaldos, sistemas operativos, virtualización, base de datos, capa media y monitoreo de los ambientes de producción y contingencia y su ejecución de las acciones operativas y periódicas de las actividades de optimización y afinamiento de procesos. Controla el inventario de los componentes tecnológicos de hardware y software.

El área de Redes realiza el registro, actualización, configuración, mantenimiento preventivo y correctivo de bienes de larga duración en lo concerniente a redes y comunicaciones, así como del inventario de los equipos con su respectivo software base, lo cual asegura la disponibilidad de la infraestructura y servicios otorgados por redes.

Mesa de Ayuda realiza la recepción de las solicitudes de los servicios tecnológicos, análisis, categorización y asignación a los diferentes grupos de apoyo hasta su verificación del cierre.

2.1.2.3. Departamento de Planificación TI

Planificación realiza la elaboración, actualización y seguimiento de la Planificación Estratégica de Tecnologías de la Información, con sus respectivos informes de la gestión de la Dirección Nacional de Tecnología. Se incluye la realización de las definiciones de especificaciones técnicas para la postulación de los proyectos de inversión de tecnologías de la información. También, mantiene el catastro de servicios tecnológicos que brinda la institución, así como la gestión de los diferentes entregables del proceso del catálogo de servicios, ficha de servicio y el mapa integral del servicio.

El área de Información comprende desde la identificación de los casos e iniciativas de uso y calidad de la información institucional, las estrategias para su extracción y su tratamiento de las iniciativas de levantamiento y actualización de la arquitectura de la información.

Arquitectura por su lado, investiga las tecnológicas emergentes que apalancan la innovación de los servicios tecnológicos, realizan estudios de factibilidad e implementan las pruebas de concepto para la emisión de informes de recomendaciones para la implementación o la adquisición de productos. Así como la elaboración de estándares de aplicaciones tecnológicas y la documentación de buenas prácticas.

2.1.2.4. Departamento de Seguridad Informática

Seguridad tiene a cargo la administración y gestión de los accesos lógicos para usuarios finales, la gestión y ejecución de la continuidad de los servicios tecnológicos de acuerdo con el plan de continuidad y recuperación del servicio. Además, incluyen el análisis de las vulnerabilidades o ataques de seguridad informática que se relacionan con la operación y comprometen los activos tecnológicos.

Y como Control de Calidad. No se tiene QA solamente personal de configuraciones, pero en este caso debería hacer la identificación de riesgos, revisión de actividades técnicas de las historias de usuario y criterios de aceptación, diseño de diferentes escenarios de pruebas para el aseguramiento de la calidad de las aplicaciones tecnológicas. Realizan la ejecución de pruebas funcionales y no funcionales, la

revisión de estándares, las actas de aceptación funcional y el informe de pruebas. Adicional, administran los ambientes de testing y preproducción, que son los ambientes de pruebas que tiene la institución.

2.1.3. El ámbito DevOps

2.1.3.1. DevOps

DevOps se puede definir como una parte cultural que ayuda a mejorar y permite acelerar la entrega de valor en las empresas por medio de la efectiva colaboración entre los equipos de Desarrollo y Operaciones (López Fernández et al., 2023).

Además, con DevOps se fortalece por los siguientes beneficios:

- Seguridad.
- Escalabilidad.
- Desarrollo ágil.

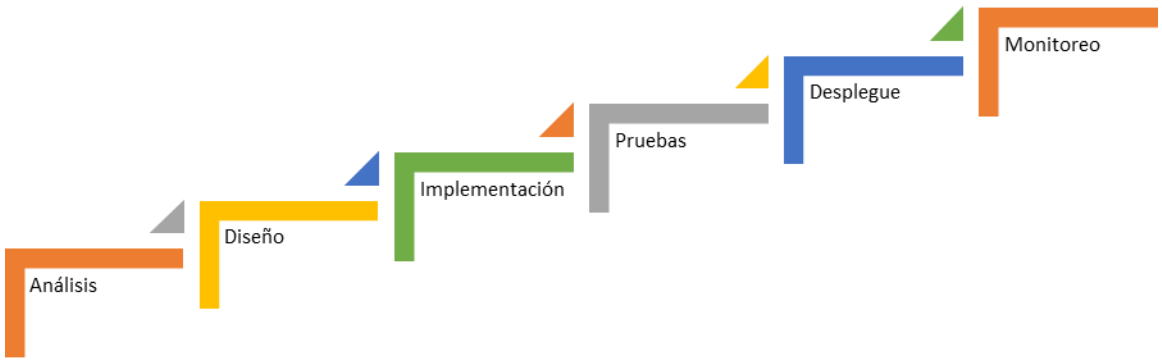
El role del ingeniero DevOps realiza la configuración de los ambientes de desarrollo, pruebas y producción, que pueden ser on-premise, contenerizadas o en la nube; lo que garantiza la alta disponibilidad de los recursos utilizados para el funcionamiento adecuado de los repositorios (Ghoddousi, 2024).

2.1.3.2. Ciclo de vida de DevOps

El ciclo de vida de desarrollo de las aplicaciones de software en los últimos años ha progresado de un modelo tradicional o en cascada, a un modelo continuo y colaborativo, éste último conocido como DevOps.

En la siguiente figura se puede observar el ciclo de vida Tradicional:

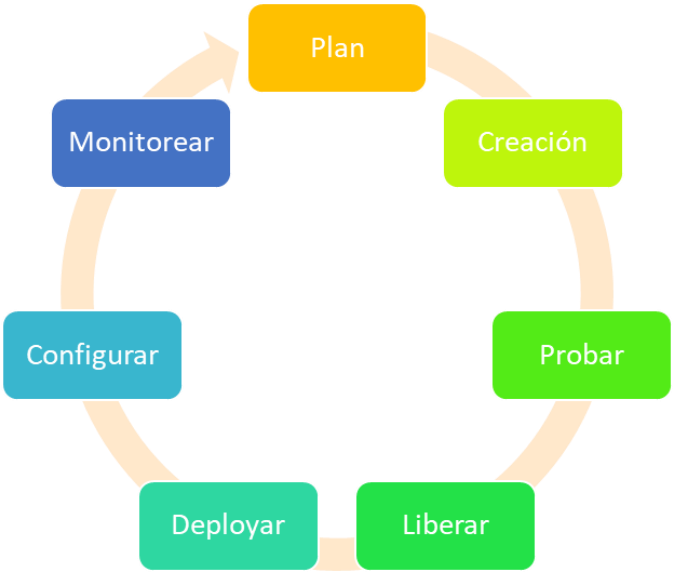
Figura 2 Ciclo de vida Tradicional.



Fuente: Elaboración propia.

En cambio, el ciclo de vida de DevOps se puede ver en la siguiente figura:

Figura 3 Ciclo de vida DevOps.



Fuente: Elaboración propia.

A continuación, se muestra una comparación entre el ciclo de vida de desarrollo Tradicional vs. DevOps:

Tabla 2 Comparación Ciclo de vida Tradicional vs DevOps

Característica	Ciclo de Vida Tradicional	Ciclo de Vida DevOps

Estructura	Secuencial	Iterativo
Trabajo Equipo	Trabajo en silos	Integrados y multifuncionales
Entrega	Entregas largas	Utiliza CI/CD
Automatización	Comúnmente procesos manuales	Automatización de despliegue
Retroalimentación	Al final del proyecto	Continua

Fuente: Elaboración propia.

2.1.3.3. GitOps

GitOps dentro del marco de mejores prácticas de Devops, simplifica el desarrollo de software debido a que utiliza Git para la gestión de la infraestructura y mejorar el ciclo de vida de desarrollo de software (Red Hat, 2026).

GitOps no solamente es una herramienta, es una evolución del marco de trabajo DevOps que utiliza Git que integra la infraestructura como parte de la gestión del desarrollo de aplicaciones, logrando con agilismo la cobertura del ciclo de vida de desarrollo de software. Por lo tanto, GitOps simplifica el desarrollo centralizando la puesta en producción con el repositorio Git.

GitOps está conformada de la siguiente manera:

- La configuración mediante ficheros yaml o json, esta es la parte declarativa para el despliegue.
- El control de versiones es inmediato.
- La automatización con Git realiza el cambio de la infraestructura de forma transparente por cada commit.

Es por esto, que GitOps simplifica la gestión de la infraestructura, garantiza la escalabilidad, y fomenta la trazabilidad debido a que el personal se enfoca en la entrega de valor del negocio.

Lo que distingue a GitOps de los enfoques de automatización es su ciclo continuo. Tal como plantea Kumar en 2024: GitOps opera la IaC al establecer un bucle de

reconciliación continua, donde agentes automatizados monitorizan constantemente el entorno en vivo y comparan su estado con el estado deseado definido en el repositorio Git; cualquier desviación dispara una remediación automatizada, garantizando que la infraestructura real siempre se aproxime al estado en Git. Este paradigma implica que todos los cambios, desde el código hasta la configuración, se realizan mediante commits en Git, se revisan mediante pull requests y se aplican automáticamente a la infraestructura, mejorando drásticamente la eficiencia, la seguridad y la auditoría de los procesos de desarrollo.

Además, GitOps brinda un cambio de paradigma en la gestión de la operación de sistemas distribuidos. Su relevancia para el presente trabajo radica en que constituye el fundamento metodológico sobre el cual se estructuran los pipelines CI/CD propuestos para el SRI.

2.1.3.4. Integración y despliegue continuo

La **Integración Continua** y el **Despliegue Continuo**, conocidos conjuntamente bajo el acrónimo CI/CD (Continuous Integration / Continuous Deployment), constituyen dos de los pilares fundamentales de las prácticas DevOps modernas y representan un área de investigación activa en la ingeniería de software contemporánea (Cruz González y Franco Calderón, 2022). Su adopción ha transformado radicalmente los ciclos de vida del desarrollo de software, reduciendo los tiempos de entrega, minimizando el riesgo asociado a los cambios en producción y mejorando la calidad del software de forma sistemática.

Integración Continua (CI)

La integración continua por sus siglas Continuos Integration, automatiza las pruebas unitarias y el despliegue de una aplicación. Es una práctica de ingeniería de software mediante la cual los desarrolladores integran sus cambios de código en un repositorio compartido de forma frecuente, desencadenando de manera automática un conjunto de procesos de validación que incluyen la compilación del

código fuente, la ejecución de pruebas unitarias y de integración, el análisis código estático y la generación de artefactos desplegables (Cruz González y Franco Calderón, 2022). El objetivo central de la CI es detectar errores en las etapas más tempranas posibles del ciclo de desarrollo, cuando su costo de corrección es significativamente menor que en fases posteriores.

La CI ha sido estudiada como un mecanismo de retroalimentación rápida, que incrementa la confianza del equipo en el estado del código base. Investigaciones del programa DORA (2023) evidencian que las organizaciones con prácticas maduras de CI reportan tiempos de recuperación ante fallos hasta cuatro veces menores y frecuencias de despliegue significativamente superiores a las organizaciones sin estas prácticas. Herramientas como Jenkins, GitHub Actions y GitLab CI/CD se han consolidado como los instrumentos más adoptados en la industria para la implementación de pipelines de integración continua.

Despliegue Continuo (CD)

El despliegue continuo por sus siglas Continuos Deployment representa una fase del ciclo de desarrollo de software de DevOps donde los cambios se generan de forma simultánea, es decir que los cambios se perciban en el entorno o ambiente que se requiere. El Despliegue Continuo representa la extensión natural de la CI en el ciclo DevOps, y constituye la fase en la que los cambios validados se propagan de forma automática y simultánea hacia el ambiente de pruebas, de preproducción o de producción, sin intervención manual. Esta automatización del proceso de liberación elimina los cuellos de botella asociados a los despliegues manuales y permite que las organizaciones entreguen valor al usuario final de forma continua y predecible (Cruz González y Franco Calderón, 2022).

Es relevante distinguir conceptualmente el Despliegue Continuo (Continuous Deployment) de la Entrega Continua (Continuous Delivery): mientras que la segunda garantiza que el software está siempre en condiciones de ser desplegado, pero requiere una aprobación manual para la liberación a producción, el primero automatiza completamente dicha liberación, eliminando toda fricción humana en el proceso. Esta distinción tiene implicaciones arquitectónicas y organizativas

profundas, especialmente en entornos regulados como el sector público o financiero, donde los controles de cambio formales pueden limitar la aplicación del despliegue continuo en su forma más pura (Chandramouli et al., 2024).

CI/CD en GitOps y en arquitecturas Cloud

En el contexto de las arquitecturas cloud y la orquestación de contenedores mediante Kubernetes, las prácticas CI/CD adquieren una dimensión adicional de complejidad y relevancia. La integración de pipelines CI/CD con plataformas como EKS y AKS, permiten que cada cambio en el código fuente desencadene un flujo automatizado que va desde la compilación y las pruebas hasta el empaquetado en imágenes de contenedor, su publicación en registros como ECR o ACR, y su despliegue final en el clúster de Kubernetes correspondiente (CNCF, 2025).

Este flujo constituye la base operativa sobre la cual se construye el paradigma GitOps, donde el repositorio Git actúa como la única fuente de verdad del estado deseado del sistema. En este modelo, los pipelines CI/CD no solo automatizan la validación del código, sino que además gestionan la reconciliación continua entre el estado declarado en el repositorio y el estado real de la infraestructura desplegada, garantizando consistencia, trazabilidad y auditabilidad en entornos multicloud (DORA, 2023).

La implementación de pipelines CI/CD se ha convertido en un estándar para las organizaciones que buscan competitividad en la era digital. Al combinar estas prácticas con estrategias de nube, se logra una reducción notable en los tiempos de comercialización (time-to-market), mayor estabilidad de los sistemas y una cultura de mejora continua. Sin embargo, su adopción exitosa requiere no solo herramientas tecnológicas, sino también cambios culturales, buenas prácticas de versionamiento y métricas de monitoreo, entre las métricas DORA se tienen: deployment frequency, lead time y change failure rate.

2.1.3.5. Infraestructura como código

La **Infraestructura como Código** (Infrastructure as Code, IaC) define la infraestructura tecnológica y configuración de redes por medio de código, que

viene a ser considerado como un sistema de software. Mantiene el código bajo control de versiones que están sujetas a una entrega continua, lo que permite la gestión de las plataformas de computación en la nube y que en los últimos años está dominando la gestión de la infraestructura. La Infraestructura como Código representa uno de los cambios de paradigma más significativos en la gestión de infraestructura tecnológica de la última década. Definida inicialmente por Fowler (2016) como la práctica de especificar y gestionar la infraestructura tecnológica de servidores, redes, balanceadores de carga y configuraciones de seguridad mediante archivos de código legibles y controlados por sistemas de versiones, la IaC ha evolucionado desde una técnica de automatización operativa hacia un principio arquitectónico fundamental en entornos cloud y en DevOps.

El valor esencial de la IaC radica en tratar la infraestructura con el mismo rigor de ingeniería que se aplica al software de aplicación. Al codificar el estado deseado de la infraestructura en archivos declarativos o imperativos, las organizaciones pueden someter dichas definiciones a prácticas consolidadas de ingeniería de software: control de versiones mediante Git, revisión de código por pares, pruebas automatizadas, integración continua y auditoría de cambios. Esto elimina la dependencia de configuraciones manuales propensas a errores que se conoce como snowflake servers o configuration drift, lo que garantiza que los entornos de desarrollo, pruebas y producción sean reproducibles, consistentes y predecibles (Fowler, 2016; DORA, 2023).

La relación entre IaC y los pipelines de entrega continua (CI/CD) es especialmente estrecha: al integrarse en los flujos de trabajo automatizados, las definiciones de infraestructura se validan, planifican y aplican de forma sistemática ante cada cambio, convirtiendo el aprovisionamiento de recursos en una etapa más del ciclo de vida del software. Esta convergencia ha sido identificada por el programa DORA (2023) como uno de los factores diferenciadores de las organizaciones con alto rendimiento en la entrega de software, las cuales reportan tiempos de aprovisionamiento de entornos significativamente menores que sus contrapartes con gestión manual de infraestructura.

La evolución natural de la IaC ha derivado en la parte técnica y académica denomina IaC Agnóstica de Nube, un enfoque en el que las definiciones de infraestructura son independientes de cualquier proveedor específico de computación en la nube. En lugar de utilizar lenguajes o herramientas propietarias vinculadas a un único proveedor como AWS CloudFormation o Azure Resource Manager (ARM), la IaC agnóstica se apoya en herramientas capaces de gestionar recursos en múltiples plataformas de nube de forma unificada, homogeneizando los procesos de aprovisionamiento y reduciendo la dependencia tecnológica conocida como vendor lock-in.

Profundizando en el uso de plantillas de automatización, se tiene el enfoque de Infraestructura como Código Agnóstica, la cual es independiente de las plataformas de computación en la nube. Lo importante es el uso de herramientas que puedan gestionar la infraestructura en múltiples nubes, lo que homogeniza su utilización.

En el contexto del sector público latinoamericano, la adopción de IaC agnóstica cobra relevancia estratégica adicional. La independencia de proveedor que ofrece este enfoque permite a las instituciones públicas negociar con múltiples proveedores de nube, cumplir con normativas de soberanía de datos y construir arquitecturas resilientes que no estén supeditadas a las decisiones comerciales de un único actor tecnológico (OCDE/BID, 2024; Chandramouli et al., 2024). En este sentido, la IaC agnóstica no es únicamente una decisión técnica, sino también una estrategia de gobernanza tecnológica alineada con los principios de neutralidad tecnológica promovidos por organismos internacionales para la transformación digital del Estado.

2.1.4. Herramientas DevOps

2.1.4.1. Terraform

Terraform es la herramienta que se utiliza mediante ficheros de configuración construir cambios de manera automatizada en la infraestructura para que los ambientes sean eficientes, es decir mediante Infraestructura como Código.

Terraform, desarrollado y mantenido por HashiCorp, se ha posicionado en la última década como la herramienta de referencia para la implementación de Infraestructura como Código en entornos de computación en la nube. Su propuesta central consiste en permitir que los equipos de ingeniería definan, planifiquen y apliquen cambios sobre la infraestructura tecnológica mediante ficheros de configuración declarativos, eliminando la necesidad de intervención manual y garantizando que los ambientes de despliegue sean reproducibles, auditables y eficientes (HashiCorp, 2023).

A diferencia de las herramientas de scripting imperativo tradicionales, Terraform adopta un enfoque declarativo: donde se especifica el estado deseado de la infraestructura y qué recursos deben existir, con qué configuración y en qué proveedor de nube. Además, Terraform se encarga de calcular el plan de ejecución necesario para alcanzar dicho estado desde la situación actual. Este proceso, conocido como reconciliación de estado, se apoya en un archivo de estado, que Terraform mantiene como registro de los recursos gestionados, permitiendo detectar desviaciones (drift) entre el estado declarado y el estado real de la infraestructura (Brikman, 2022).

El flujo de trabajo de Terraform se articula en tres fases bien definidas que se integran de forma natural en los pipelines de CI/CD: la fase de inicialización (terraform init), que descarga los proveedores y módulos necesarios; la fase de planificación (terraform plan), que calcula y presenta de forma explícita los cambios que se aplicarán sobre la infraestructura sin ejecutarlos; y la fase de aplicación (terraform apply), que materializa dichos cambios de forma automatizada. Esta separación entre planificación y ejecución introduce un mecanismo de revisión y aprobación que resulta especialmente valioso en entornos productivos y regulados, donde la trazabilidad de los cambios de infraestructura es un requisito de gobernanza (Chandramouli et al., 2024).

Una de las características que ha consolidado a Terraform como estándar en la industria es su arquitectura basada en proveedores, que le otorga capacidad de gestionar recursos en múltiples plataformas de nube de forma unificada. A través

de su ecosistema de proveedores que abarca AWS, Microsoft Azure, Google Cloud Platform, Kubernetes y más de tres mil integraciones adicionales disponibles en el Terraform Registry, los equipos de ingeniería pueden describir arquitecturas multicloud completas en un único lenguaje de configuración denominado HCL (HashiCorp Configuration Language), homogeneizando los procesos de aprovisionamiento y reduciendo la complejidad operativa asociada al uso simultáneo de múltiples plataformas (CNCF, 2025).

En el contexto del paradigma GitOps, Terraform adquiere un papel central como motor de aprovisionamiento de infraestructura. Al almacenar los ficheros de configuración en repositorios Git y gestionar su aplicación mediante pipelines automatizados, las organizaciones implementan un modelo en el que cualquier cambio en la infraestructura queda registrado como una modificación versionada y revisable en el repositorio, garantizando auditoría completa, capacidad de rollback y consistencia entre entornos (Fowler, 2016; DORA, 2023). Herramientas como Atlantis y Terraform Cloud extienden este modelo incorporando flujos de aprobación basados en pull requests, lo que alinea el gobierno de la infraestructura con las prácticas de revisión de código del equipo de desarrollo.

Desde la perspectiva del sector público y de organizaciones con requisitos estrictos de cumplimiento normativo, Terraform ofrece ventajas adicionales en términos de neutralidad tecnológica y soberanía de infraestructura. Al no estar vinculado a ningún proveedor de nube específico, su adopción permite a las instituciones mantener la flexibilidad de migrar cargas de trabajo entre proveedores sin necesidad de reescribir las definiciones de infraestructura, lo que se alinea con los principios de interoperabilidad y evitación del vendor lock-in promovidos por organismos internacionales para la transformación digital del Estado (OCDE/BID, 2024; Chandramouli et al., 2024).

2.1.4.2. GitLab y GitHub Actions

En el ecosistema de herramientas DevOps, las plataformas de integración y despliegue continuo han evolucionado desde simples automatizadores de

compilación hacia soluciones integrales que cubren la totalidad del ciclo de vida del desarrollo de software. Entre las plataformas más relevantes en la literatura académica y en la adopción industrial se encuentran GitLab CI y GitHub Actions, cada una con un enfoque diferenciado pero convergente en su propósito de acelerar y asegurar la entrega de software de calidad.

GitLab

GitLab se ha consolidado como una plataforma DevOps unificada que integra en un único entorno todas las disciplinas del ciclo de vida del desarrollo de software: planificación de proyectos, control de versiones, revisión de código, integración continua, despliegue continuo, monitorización y seguridad (Choudhury et al., 2020). Esta integración vertical la distingue de soluciones basadas en la composición de múltiples herramientas independientes, reduciendo la fricción operativa asociada a la gestión de integraciones entre sistemas heterogéneos y proporcionando una trazabilidad end-to-end desde el requerimiento hasta el despliegue en producción.

El componente de automatización de GitLab, conocido como GitLab CI/CD, opera mediante la definición de pipelines declarativos en un fichero de configuración denominado .gitlab-ci.yml, almacenado en el propio repositorio de código. Esta característica garantiza que la definición del pipeline esté sujeta al mismo control de versiones que el código fuente, lo que establece una coherencia fundamental entre el comportamiento del sistema y su historial de cambios. Cada modificación en el repositorio puede desencadenar automáticamente un flujo de trabajo que abarca desde la compilación y las pruebas unitarias hasta el análisis de seguridad estático (SAST), la generación de artefactos y el despliegue en los ambientes destino (DORA, 2023).

Un aspecto particularmente relevante para el contexto de arquitecturas multicloud y GitOps es la capacidad nativa de GitLab para integrarse con herramientas de automatización de infraestructura como Terraform, Ansible y Kubernetes. GitLab incorpora un módulo específico denominado GitLab Terraform que permite ejecutar los flujos de trabajo de IaC directamente desde el pipeline, almacenando

el estado de Terraform en el backend de GitLab y gestionando los planes de ejecución como artefactos revisables antes de su aplicación (HashiCorp, 2023). Esta integración posiciona a GitLab como un habilitador natural del paradigma GitOps, donde el repositorio Git actúa como única fuente de verdad del estado deseado tanto del código de aplicación como de la infraestructura subyacente. Choudhury et al. (2020) destacan que esta convergencia de capacidades fomenta activamente la cultura DevOps en las organizaciones, al eliminar las barreras entre los equipos de desarrollo y operaciones mediante una plataforma colaborativa común.

Con Gitlab se puede gestionar el ciclo de vida de desarrollo de software, es la herramienta que integra el control de versiones, el repositorio de código, la colaboración de proyectos y la automatización de despliegue de aplicaciones. Es decir, desde la planificación hasta la implementación.

La plataforma de Gitlab permite la automatización en un solo lugar, de igual manera puede iniciar con el flujo de trabajo GitOps. Gitlab puede utilizar diferentes herramientas de automatización de infraestructura como Terraform, Ansible y Kubernetes. Posee características de innovación que fomenta la cultura Devops en las empresas (Choudhury et al., 2020).

GitHub Actions

GitHub Actions es la plataforma de integración continua y despliegue continuo (CI/CD), que permite realizar la automatización de los flujos de desarrollo de software, la cual está integrada con GitHub y puede sincronizar el código con la infraestructura.

GitHub Actions constituye la plataforma de CI/CD integrada de forma nativa en GitHub, el repositorio de código más utilizado a nivel mundial. Su propuesta de valor central radica en la eliminación de la fricción entre el repositorio de código y los flujos de automatización: al estar embebida directamente en la plataforma donde reside el código fuente, GitHub Actions permite que cualquier evento del repositorio con una confirmación de código (commit), la apertura de una solicitud de fusión (pull request), la publicación de una versión (release) o incluso un evento programado— desencadene de forma automática un flujo de trabajo (workflow)

definido mediante ficheros YAML versionados en el propio repositorio (GitHub, 2023).

La arquitectura de GitHub Actions se fundamenta en tres conceptos interrelacionados: los workflows, que representan los flujos de automatización completos; los jobs, que son conjuntos de pasos ejecutados en un entorno aislado; y las actions, que son unidades reutilizables de automatización publicadas en el GitHub Marketplace por la comunidad o por los propios equipos. Este modelo de composición modular permite construir pipelines complejos a partir de componentes verificados y mantenidos por la comunidad, reduciendo el tiempo de implementación y promoviendo la estandarización de las prácticas de automatización (CNCF, 2025).

Con GitHub Actions se identifica cómo la estructura de los flujos de trabajo o workflows impactan directamente en los tiempos de respuesta y la mantenibilidad de los pipelines. Al respecto, se incorporan estrategias de ejecución concurrente como las construcciones matriciales (matrix builds) que constituye un factor crítico para mitigar la degradación y el incremento difuso de la complejidad en entornos de software distribuidos a gran escala (Mazrae et al., 2026).

Desde la perspectiva de la sincronización entre código e infraestructura, GitHub Actions ha incorporado integraciones oficiales con los principales proveedores de nube, incluyendo AWS, Microsoft Azure y Google Cloud Platform, así como con herramientas de IaC como Terraform, lo que le permite participar activamente en flujos de trabajo GitOps donde el despliegue de infraestructura y aplicaciones se gestiona de forma declarativa y automatizada desde el repositorio (Fowler, 2016; DORA, 2023). El programa DORA (2023) identifica a GitHub Actions como una de las herramientas más adoptadas entre las organizaciones clasificadas como high performers en sus métricas de entrega de software, destacando su contribución a la reducción de los tiempos de ciclo (lead time for changes) y a la mejora de la tasa de éxito en los despliegues.

2.1.4.3. Helm y Kustomize

La proliferación de arquitecturas basadas en Kubernetes como plataforma de orquestación de contenedores ha generado una necesidad creciente de herramientas especializadas para la gestión, parametrización y reutilización de las configuraciones de despliegue. En este contexto, Helm y Kustomize han emergido como las dos soluciones más adoptadas en la industria y más referenciadas en la literatura académica para abordar la complejidad inherente a la gestión de manifiestos Kubernetes a escala, cada una desde una filosofía técnica diferenciada pero complementaria (Buchanan et al., 2020; Parra Royón et al., 2026).

Helm

Helm es un gestor estándar para Kubernetes que automatiza el despliegue y configuración de las aplicaciones tipo clúster. Esta herramienta permite el despliegue y empaquetado de aplicaciones tipo clúster y utiliza las plataformas basadas en Kubernetes (Buchanan et al., 2020).

Helm se define en la literatura como el gestor de paquetes estándar para Kubernetes, cumpliendo en el ecosistema cloud-native una función análoga a la que desempeñan herramientas como APT en sistemas Linux o pip en Python: abstraer la complejidad de la instalación, configuración, actualización y desinstalación de aplicaciones en clústeres Kubernetes mediante unidades de distribución reutilizables y versionadas (Buchanan et al., 2020). Estas unidades son denominadas charts, y constituyen colecciones estructuradas de plantillas de manifiestos Kubernetes parametrizables mediante un sistema de valores que permite adaptar el comportamiento de la aplicación a diferentes entornos de desarrollo, pruebas, y producción, sin modificar las plantillas base.

El modelo operativo de Helm introduce tres conceptos fundamentales interrelacionados: el chart, que encapsula la definición completa de una aplicación Kubernetes incluyendo sus dependencias; el release, que representa una instancia concreta de un chart desplegada en un clúster con una configuración específica; y el repository, que actúa como catálogo centralizado desde el cual los equipos pueden descubrir, compartir y distribuir charts de forma estandarizada. Esta arquitectura ha favorecido el desarrollo de un ecosistema comunitario de notable

amplitud, con repositorios públicos como Artifact Hub que albergan miles de charts mantenidos tanto por la comunidad como por los propios fabricantes de software, lo que reduce significativamente el tiempo de onboarding de nuevas aplicaciones en entornos Kubernetes (CNCF, 2025).

Desde la perspectiva de los pipelines CI/CD y GitOps, Helm desempeña un papel central como mecanismo de empaquetado y despliegue. La integración de Helm en flujos de trabajo automatizados permite que cada versión del software genere un chart versionado que puede ser promovido de forma controlada a través de los distintos ambientes del ciclo de vida, garantizando reproducibilidad y trazabilidad en cada despliegue (DORA, 2023). Operadores GitOps como Argo CD y Flux CD ofrecen soporte nativo para Helm, permitiendo que el estado deseado de los despliegues sea declarado en el repositorio Git mediante referencias a charts y valores de configuración, y reconciliado de forma continua con el estado real del clúster (Rostami Mazrae et al., 2023; MechCloud Academy, 2026).

Kustomize

Kustomize representa un enfoque alternativo y complementario a Helm para la gestión de configuraciones Kubernetes, fundamentado en el principio de personalización por superposición (overlay-based customization) en lugar de la parametrización mediante plantillas. Su propuesta central consiste en permitir que los equipos de ingeniería definan una configuración base de manifiestos Kubernetes que valida y funciona por sí misma, y apliquen sobre ella capas de personalización (overlays) específicas para cada entorno, sin necesidad de duplicar ni modificar los ficheros originales (Parra Royón et al., 2026).

Esta filosofía de diseño resuelve uno de los problemas más recurrentes en la gestión de configuraciones a escala: la proliferación de ficheros de configuración redundantes y divergentes entre entornos, que incrementa la deuda técnica, dificulta el mantenimiento y aumenta la probabilidad de inconsistencias entre ambientes. Al mantener una única fuente de verdad para la configuración base y expresar las diferencias entre entornos exclusivamente mediante parches declarativos (patches), Kustomize promueve el principio DRY (Don't Repeat

Yourself) en la gestión de infraestructura, alineándose con las buenas prácticas de ingeniería de software aplicadas al dominio de las operaciones (Fowler, 2016).

Una ventaja operativa significativa de Kustomize es su integración nativa en la herramienta de línea de comandos de Kubernetes (kubectl), disponible de forma incorporada desde la versión 1.14 sin necesidad de instalación adicional. Esta característica reduce la fricción de adopción en equipos que ya operan clústeres Kubernetes, y ha sido identificada como un factor relevante en su amplia difusión en entornos productivos (CNCF, 2025). Al igual que Helm, Kustomize es soportado nativamente por los principales operadores GitOps, lo que lo posiciona como una herramienta de primera clase en arquitecturas donde la gestión declarativa y versionada de la configuración es un requisito fundamental.

Helm y Kustomize en GitOps Multicloud

En el contexto de arquitecturas GitOps multicloud, Helm y Kustomize no son necesariamente excluyentes sino frecuentemente complementarios. Un patrón de uso ampliamente documentado en la literatura técnica consiste en utilizar Helm para el empaquetado y distribución de aplicaciones, y Kustomize para la personalización de los valores de despliegue en cada entorno y proveedor de nube específico, combinando así las fortalezas de ambas herramientas en un único flujo de trabajo coherente (Buchanan et al., 2020). Este patrón resulta especialmente relevante en organizaciones que operan simultáneamente sobre múltiples plataformas Kubernetes gestionadas como EKS en AWS y AKS en Azure, donde la consistencia de los despliegues entre proveedores es un requisito crítico de gobernanza y continuidad operativa.

2.1.4.4. ArgoCD

ArgoCD es la herramienta muy utilizada para la entrega continua para Kubernetes y sigue el marco conceptual de GitOps, Cuando se realizan modificaciones en la configuración de una aplicación en el repositorio de Git, ArgoCD compara estas

configuraciones para la ejecución y sincronización del estado de producción (Prates y Pereira, 2025).

Argo CD se ha consolidado como una de las herramientas más relevantes y ampliamente adoptadas en el ecosistema Kubernetes para la implementación del paradigma GitOps en entornos de entrega continua. Clasificado por la Cloud Native Computing Foundation (CNCF, 2025) como la categoría de mayor madurez en su ecosistema, Argo CD opera como un operador GitOps declarativo que extiende las capacidades nativas de Kubernetes para gestionar el ciclo de vida completo de las aplicaciones desplegadas en clústeres, tomando el repositorio Git como única fuente de verdad del estado deseado del sistema (Prates y Pereira, 2025).

Su arquitectura se fundamenta en un principio operativo central denominado reconciliación continua: Argo CD monitoriza de forma permanente y simultánea el estado declarado en el repositorio Git y el estado real observado en el clúster Kubernetes, detectando cualquier divergencia entre ambos que se la conoce como configuration drift, y actuando de forma automática o supervisada para restablecer la convergencia hacia el estado deseado. Este mecanismo garantiza que el clúster refleje en todo momento la configuración versionada y aprobada en el repositorio, eliminando la posibilidad de modificaciones manuales no rastreadas que comprometan la integridad del entorno de producción.

El flujo de trabajo de Argo CD se articula en torno a tres operaciones fundamentales que estructuran su ciclo de reconciliación. La primera es la detección de cambios: cuando un equipo de desarrollo o un pipeline CI/CD introduce una modificación en los manifiestos de configuración almacenados en el repositorio Git, ya sea actualización de una imagen de contenedor, cambio en los valores de un chart Helm o modificación de un overlay Kustomize, Argo CD detecta automáticamente la divergencia entre el estado declarado y el estado actual del clúster. La segunda operación es la comparación y análisis de diferencias: Argo CD presenta una vista detallada de las diferencias identificadas, permitiendo a los equipos revisar con precisión qué recursos serán creados, modificados o eliminados antes de proceder con la sincronización. La tercera operación es la

sincronización: Argo CD aplica los cambios necesarios sobre el clúster Kubernetes para alcanzar el estado deseado, registrando el resultado de cada operación con trazabilidad completa (Prates y Pereira, 2025; CNCF, 2025).

Este flujo introduce una distinción operativa relevante entre el modo de sincronización automática, donde Argo CD aplica los cambios sin intervención humana, adecuado para entornos de desarrollo o pruebas con alta frecuencia de cambios y el modo de sincronización manual o supervisada, donde la aplicación de cambios requiere una aprobación explícita, modelo preferido en entornos de producción con requisitos estrictos de control de cambios y gobernanza, como los presentes en instituciones del sector público (Chandramouli et al., 2024).

Una característica que ha contribuido decisivamente a la adopción de Argo CD en arquitecturas GitOps complejas es su soporte nativo para los principales mecanismos de gestión de configuración Kubernetes. Argo CD es compatible de forma integrada con Helm, Kustomize, Jsonnet y manifiestos YAML planos, lo que le otorga flexibilidad para adaptarse a las convenciones de configuración de equipos con distintos niveles de madurez y diferentes herramientas de empaquetado (Buchanan et al., 2020; Parra Royón et al., 2026). Esta versatilidad permite que Argo CD actúe como la capa de orquestación GitOps sobre una base tecnológica heterogénea, sin imponer restricciones sobre las herramientas de definición de configuración utilizadas por los equipos de desarrollo.

En el contexto de los pipelines CI/CD, Argo CD establece una separación conceptual y operativa clara entre las responsabilidades de integración continua y las de despliegue continuo, conocida en la literatura como el patrón CI/CD desacoplado: mientras que herramientas como GitLab CI o GitHub Actions se encargan de la compilación del código, la ejecución de pruebas y la generación de artefactos incluyendo imágenes de contenedor publicadas en registros como ECR o ACR, Argo CD asume exclusivamente la responsabilidad del despliegue y la reconciliación del estado en el clúster Kubernetes (DORA, 2023). Esta separación de responsabilidades incrementa la modularidad del sistema, facilita la auditoría independiente de cada etapa y reduce el radio de impacto de fallos en el pipeline.

En el ámbito de las arquitecturas GitOps multicloud, Argo CD ofrece capacidades de gestión federada mediante su extensión Argo CD ApplicationSet, que permite definir y gestionar despliegues sincronizados sobre múltiples clústeres Kubernetes en distintos proveedores de nube como EKS en AWS y AKS en Azure, desde un único punto de control centralizado (CNCF, 2025). Esta capacidad resulta estratégicamente relevante en organizaciones que operan arquitecturas distribuidas entre proveedores, ya que garantiza la consistencia del estado deseado de las aplicaciones independientemente del proveedor de nube subyacente, alineándose con los principios de neutralidad tecnológica e interoperabilidad promovidos para la transformación digital del sector público (OCDE/BID, 2024).

Desde la perspectiva de la seguridad y el cumplimiento normativo, Argo CD incorpora mecanismos de control de acceso basado en roles (RBAC), integración con proveedores de identidad mediante SSO (Single Sign-On) y registro detallado de auditoría de todas las operaciones realizadas sobre los clústeres gestionados. Estas características responden directamente a los requisitos de trazabilidad, separación de funciones y control de cambios establecidos en marcos normativos de referencia para sistemas críticos en entornos cloud-native (NIST SP 800-204D, Chandramouli et al., 2024). En el caso específico de instituciones como el Servicio de Rentas Internas del Ecuador (SRI), donde la integridad y la trazabilidad de los sistemas de gestión tributaria son requisitos no negociables, la adopción de Argo CD como operador GitOps ofrece garantías técnicas alineadas con los estándares de seguridad exigidos por organismos internacionales para la digitalización fiscal en América Latina (BID/Reyes-Tagle, 2024).

2.1.4.5. Prometheus – Grafana

La observabilidad se ha consolidado como una disciplina fundamental en la ingeniería de sistemas distribuidos modernos, trascendiendo la noción tradicional de monitorización para abarcar la capacidad de inferir el estado interno de un sistema a partir de sus señales externas: métricas, trazas distribuidas y registros de

eventos o logs. En arquitecturas cloud-native basadas en Kubernetes, donde la naturaleza efímera de los contenedores y la distribución de los servicios introduce una complejidad operativa sin precedentes, contar con un stack de observabilidad robusto y escalable no es una opción sino un requisito arquitectónico de primer orden (Chandramouli et al., 2024; CNCF, 2025). En este contexto, la combinación de Prometheus y Grafana se ha establecido como el estándar de facto en la industria para la recolección, almacenamiento y visualización de métricas en entornos distribuidos.

Prometheus

Phometheus es la herramienta utilizada para la monitorización de aplicaciones en la nube, una de sus principales funciones es la medición en tiempo real para almacenarlas en la base de datos de series temporales. Prometheus es un sistema de monitorización y alerta de código abierto, originalmente desarrollado por SoundCloud en 2012 y posteriormente donado a la Cloud Native Computing Foundation (CNCF), que es la categoría de mayor madurez en el ecosistema cloud (CNCF, 2025). Su diseño responde específicamente a los desafíos de monitorización en entornos de microservicios y contenedores, donde el número de componentes a observar es elevado, dinámico y frecuentemente efímero.

El modelo de recolección de métricas de Prometheus se basa en el paradigma pull: en lugar de que los servicios monitorados envíen activamente sus métricas hacia un sistema centralizado (push), Prometheus consulta periódicamente (scrape) los endpoints de métricas expuestos por cada servicio o componente de infraestructura, recopilando series temporales de datos numéricos etiquetados con metadatos descriptivos denominados labels. Esta arquitectura confiere a Prometheus una notable resiliencia ante fallos de componentes individuales y facilita el descubrimiento dinámico de nuevos servicios en entornos Kubernetes, donde los pods se crean y destruyen de forma continua (Turnbull, 2018). Las métricas recopiladas son almacenadas en una base de datos de series temporales (time-series database, TSDB) optimizada para consultas sobre datos temporales de alta cardinalidad, utilizando un modelo de compresión eficiente que permite

retener grandes volúmenes de métricas históricas con un consumo de almacenamiento razonable.

Para la consulta y análisis de las métricas almacenadas, Prometheus incorpora un lenguaje de consulta propio denominado PromQL (Prometheus Query Language), que permite expresar operaciones complejas sobre series temporales: agregaciones, tasas de cambio, percentiles, predicciones de tendencia y correlaciones entre métricas de distintos servicios. PromQL constituye la interfaz primaria mediante la cual los operadores definen tanto los paneles de visualización como las reglas de alerta que notifican al equipo ante condiciones anómalas del sistema —degradación del rendimiento, agotamiento de recursos, tasas de error elevadas— con la anticipación suficiente para permitir una respuesta proactiva antes de que el impacto llegue al usuario final (DORA, 2023).

En el contexto de los pipelines GitOps, Prometheus se integra como componente de observabilidad del plano de operaciones, proporcionando las señales cuantitativas necesarias para evaluar el impacto de cada despliegue sobre el comportamiento del sistema en producción. Esta integración permite implementar patrones avanzados de despliegue como el análisis canario automatizado, donde herramientas como Argo Rollouts utilizan las métricas de Prometheus para determinar de forma autónoma si un nuevo despliegue debe promoverse o revertirse en función de umbrales de rendimiento predefinidos.

Grafana

Mientras que Grafana es la herramienta que tiene paneles de control para la visualización de los datos observados por Prometheus. Grafana es una plataforma de visualización y análisis de datos operacionales de código abierto que complementa a Prometheus proporcionando una capa de presentación interactiva, flexible y altamente configurable sobre las métricas almacenadas en la base de datos de series temporales. Desarrollada por Grafana Labs y también integrada en el ecosistema de proyectos CNCF, Grafana permite a los equipos de operaciones construir paneles de control (dashboards) interactivos que consolidan en una única vista las métricas más relevantes del sistema, facilitando tanto la

supervisión operativa en tiempo real como el análisis retrospectivo de incidentes (Chakraborty y Kundan, 2021).

La arquitectura de Grafana se fundamenta en el concepto de fuentes de datos (data sources) intercambiables: aunque su integración más habitual es con Prometheus, Grafana puede conectarse de forma nativa con una amplia variedad de sistemas de almacenamiento de métricas, registros y trazas, incluyendo Loki para logs, Tempo para trazas distribuidas, InfluxDB, Elasticsearch y los servicios nativos de monitorización de AWS (CloudWatch) y Azure (Azure Monitor), lo que la convierte en una capa de observabilidad unificada capaz de correlacionar señales de distintas fuentes en un único entorno de análisis (CNCF, 2025). Esta capacidad de agregación multiplataforma es particularmente valiosa en arquitecturas multicloud, donde la dispersión de señales observacionales entre distintos proveedores representa uno de los principales desafíos operativos.

Los paneles de Grafana se definen mediante ficheros JSON versionables en repositorios Git, lo que permite incorporar la configuración de los dashboards al flujo de trabajo GitOps y garantizar que la evolución de la observabilidad del sistema quede registrada con la misma trazabilidad que el código de aplicación y la infraestructura (Fowler, 2016). Adicionalmente, Grafana incorpora un sistema de alertas propio que complementa al de Prometheus, permitiendo definir condiciones de notificación directamente sobre las visualizaciones y enrutar las alertas hacia canales de comunicación como correo electrónico, Slack o PagerDuty, cerrando el ciclo de observabilidad desde la detección hasta la notificación al equipo responsable.

2.1.5. Estrategias de nube

Con el incremento de la nube en grandes empresas, como son **Amazon Web Service (AWS)** y **Microsoft Azure**, han permitido a las empresas tener mayor agilismo y retorno de las inversiones en el desarrollo de aplicaciones. Estas plataformas en la nube permiten mantener el servicio tecnológico de las empresas bajo el modelo de pago por uso. La computación en la nube ha experimentado una evolución sostenida durante la

última década, transitando desde modelos de infraestructura básica (Infraestructura como Servicio, IaaS) hacia plataformas de servicios gestionados de alto nivel que abstraen progresivamente la complejidad operativa. Investigaciones recientes coinciden en que, la adopción de proveedores de nube pública como AWS y Microsoft Azure han permitido a las organizaciones alcanzar niveles de agilidad operativa y eficiencia en la inversión tecnológica difícilmente replicables con infraestructura locales.

Desde una perspectiva académica y técnica, el modelo de pago por uso ha sido ampliamente estudiado como un mecanismo que democratiza el acceso a infraestructura de alto rendimiento, eliminando las barreras de capital inicial y permitiendo a las organizaciones escalar sus recursos de forma proporcional a la demanda real. Este modelo ha sido identificado como un habilitador crítico para la implementación de prácticas DevOps y la adopción de arquitecturas de microservicios en entornos productivos (CNCF, 2025). Así como, la reducción de costos operativos mediante la eliminación de gastos de capital en hardware; los mecanismos avanzados de seguridad incorporados en los proveedores globales, que incluyen la escalabilidad, que permite ajustar dinámicamente la capacidad de cómputo; y la reducción de latencia, que se logra a través de redes de distribución geográfica con múltiples regiones y zonas de disponibilidad (Chandramouli et al., 2024).

En el ámbito de la orquestación de contenedores, Kubernetes se ha establecido como el estándar de facto a nivel industrial y académico. La Cloud Native Computing Foundation (CNCF, 2025) reporta que más del 96 % de las organizaciones encuestadas utilizan o evalúan Kubernetes como plataforma de orquestación, lo que evidencia su posición dominante en el ecosistema cloud-native. En este contexto, los principales proveedores de nube han desarrollado implementaciones gestionadas que eliminan la carga operativa asociada a la administración del control plane, reduciendo la fricción técnica para los equipos de TI.

Entre las principales ventajas de la adopción de estrategias de nube destacan:

- Reducción de costos: Al eliminar o minimizar gastos fijos en infraestructura física, licencias y mantenimiento, las organizaciones pueden redirigir recursos hacia actividades de mayor valor estratégico.
- Seguridad: Los proveedores de nube implementan estándares avanzados de protección, cifrado, cumplimiento normativo y monitoreo continuo, frecuentemente superiores a lo que muchas empresas pueden lograr internamente.
- Escalabilidad: Permite ajustar recursos de forma rápida y automática según la demanda, soportando picos de tráfico sin interrupciones.
- Reducción de la latencia: Mediante la distribución global de centros de datos (regiones y zonas de disponibilidad), se mejora la experiencia del usuario al acercar los servicios a los consumidores finales.

Amazon Elastic Kubernetes Service (EKS) representa la oferta gestionada de Kubernetes en el ecosistema AWS. EKS delega en el proveedor la responsabilidad de la disponibilidad, actualización y seguridad del plano de control del clúster, mientras que los equipos de desarrollo mantienen el control sobre los nodos de cómputo y las cargas de trabajo desplegadas. Estudios comparativos destacan la profunda integración de EKS con servicios nativos de AWS como IAM, VPC, CloudWatch y ECR como una ventaja diferencial en términos de seguridad y observabilidad.

Azure Kubernetes Service (AKS), por su parte, constituye la implementación equivalente en el ecosistema de Microsoft Azure. A diferencia de EKS, AKS no aplica cargos por el plano de control del clúster, trasladando el costo directamente a los nodos de cómputo aprovisionados. Esta diferencia estructural en el modelo tarifario ha sido objeto de análisis en investigaciones sobre optimización de costos en arquitecturas multicloud, donde la selección del proveedor en función del perfil de carga de trabajo puede representar diferencias significativas en el costo total de operación. AKS se destaca además por su integración nativa con Azure Active Directory, Azure Monitor y Azure Policy, lo que lo posiciona como una opción relevante en organizaciones con ecosistemas Microsoft consolidados (OCDE, 2024).

Desde la perspectiva de las estrategias multicloud, la coexistencia de EKS y AKS en una misma arquitectura representa tanto una oportunidad como un desafío. La oportunidad radica en la capacidad de distribuir cargas de trabajo entre proveedores en función de criterios de costo, latencia o cumplimiento normativo; el desafío consiste en garantizar la consistencia operativa, la portabilidad de los despliegues y la coherencia del estado deseado de la infraestructura a través de herramientas como GitOps.

2.1.6. Design Science Research (DSR)

El **Design Science Research (DSR)** es un paradigma de investigación orientado a resolver problemas complejos mediante la creación de artefactos tecnológicos innovadores (como algoritmos, modelos, métodos o sistemas de software). Conecta la teoría y la práctica evaluando su utilidad en el mundo real para generar nuevo conocimiento científico.

2.1.6.1. Concepto Tecnológico e Investigación

En el ámbito tecnológico, el DSR no busca solo entender cómo funciona la tecnología, sino construir soluciones funcionales que transformen los problemas existentes en estados deseados. Se desarrolla en un ciclo iterativo que incluye:

Elaboración: Construcción del artefacto (prototipos, lenguajes, modelos).

Evaluación: Pruebas rigurosas sobre la viabilidad, usabilidad y eficacia de la solución.

Contribución: Extracción de nuevos principios, teorías y conocimientos de diseño que otros investigadores y profesionales puedan reutilizar.

2.2. Trabajos relacionados

Se han revisado algunos trabajos relacionados al tema, entre los que podemos citar tenemos:

2.2.1. Infraestructura como Código en Entornos Multicloud

Autores como (Beetz et al., 2022) manifiestan que el aprovisionamiento de infraestructura ha experimentado una transformación radical en las últimas décadas.

Este ambiente multicloud ha evolucionado desde la administración manual basada en scripts hacia la automatización declarativa impulsada por los principios de Infraestructura como Código (IaC). A medida que las empresas adoptaron la virtualización y posteriormente arquitecturas nativas de la nube, la demanda de despliegues escalables, reproducibles y gobernados creció exponencialmente. Las primeras herramientas de gestión de configuración, como Puppet y Chef, sentaron las bases para este cambio; sin embargo, fue con la aparición de Terraform y Ansible que la industria encontró soluciones capaces de unificar el aprovisionamiento y la configuración de infraestructura en entornos multicloud.

IaC es hoy una práctica fundamental en el DevOps moderno que permite la automatización del aprovisionamiento de infraestructura mediante código declarativo y versionado. En entornos multicloud, donde las organizaciones aprovechan servicios de múltiples proveedores como AWS y Azure, IaC desempeña un punto fundamental para garantizar la consistencia, la duplicidad y la gobernanza. Esta naturaleza agnóstica de la nube es precisamente uno de los atributos más apreciados en el contexto del presente trabajo de investigación, ya que permite al SRI mantener la portabilidad de sus microservicios sin quedar atado a un único proveedor de nube pública.

En cuanto a los modelos arquitectónicos de repositorios, se identifica dos estrategias principales: el monorepo y el polyrepo. El monorepo centraliza todas las configuraciones de entorno y aplicación en un único repositorio, ofreciendo simplicidad en visibilidad y control, aunque resulta más difícil de escalar y asegurar en equipos grandes o en dominios cloud distribuidos. En contraste, los modelos polyrepo distribuyen la configuración en múltiples repositorios por entorno, región o unidad de negocio, mejorando la modularidad, el control de acceso y la escalabilidad (Abiola Samuel et al., 2025). Para una institución pública del tamaño y la criticidad del SRI, la elección entre estos modelos no es trivial: implica decisiones sobre gobernanza, separación de responsabilidades y velocidad de despliegue que deben alinearse con la política institucional de seguridad de la información.

Terraform, la herramienta de IaC más adoptada en la industria, ha alcanzado el estatus de estándar de facto para la gestión de infraestructura multicloud. Su modelo

agnóstico de nube y su extenso ecosistema de módulos permiten a los equipos gestionar la infraestructura de manera unificada, con sola una configuración para desplegar recursos en AWS y Azure mediante el simple intercambio del proveedor (Refonte Learning, 2026).

Es relevante destacar que, en 2025, IBM adquirió HashiCorp por 6.4 mil millones de dólares, generando expectativas en la comunidad DevOps respecto a una posible reorientación hacia modelos de código abierto más amigables. Esta dinámica empresarial evidencia que la elección de herramientas de IaC para instituciones públicas debe considerar no solo las capacidades técnicas actuales, sino también la estabilidad de licenciamiento y sostenibilidad a largo plazo de los proyectos, aspectos críticos para una entidad gubernamental como el SRI.

2.2.2. GitOps para Kubernetes

En el ecosistema de herramientas GitOps para Kubernetes, dos proyectos han emergido como referentes indiscutibles, ambos otorgados por la Cloud Native Computing Foundation (CNCF): Argo CD y Flux CD. Su adopción masiva refleja la madurez que ha alcanzado el paradigma GitOps, pasando de ser una práctica de vanguardia a convertirse en un estándar operacional en la ingeniería de plataformas moderna (CNCF, 2025).

Argo CD es una herramienta de entrega continua declarativa que sincroniza continuamente los clústeres de Kubernetes con lo definido en Git. Su característica más distintiva es una interfaz web rica que permite visualizar y gestionar aplicaciones directamente desde el navegador, con capacidades avanzadas de gestión multiclúster, control de acceso basado en roles y soporte nativo para SSO. Estudios recientes destacan que en entornos empresariales que gestionan cientos de microservicios, el panel multiclúster de ArgoCD fue el factor determinante para los equipos de plataforma, permitiendo reducir significativamente los tiempos de depuración de despliegues. En 2025, ArgoCD ha sido adoptado como estándar GitOps por los principales proveedores de nube (Dev.to, 2025).

Flux CD es un motor GitOps construido íntegramente alrededor de los controladores de Kubernetes. Conlleva los conceptos nativos de Kubernetes; todo se define como

una Custom Resource Definition y se reconcilia continuamente desde la fuente hasta el clúster. A diferencia de ArgoCD, Flux no solo despliega código, sino que impone el estado, debido a ediciones manuales, Flux detecta la desviación y la restituye, garantizando los despliegues. En 2026, la capacidad de Flux de operar con un mínimo de recursos y sin requerimientos de red ha consolidado su dominio en sectores como la manufactura y las telecomunicaciones, especialmente en escenarios de edge computing (Dev.to, 2026).

Desde una perspectiva arquitectónica, tanto Argo CD como Flux adoptan el enfoque GitOps basado en el modelo pull, ampliamente reconocido como una de las alternativas más seguras para la gestión de despliegues. En este modelo, un agente GitOps que se ejecuta dentro del clúster supervisa de forma continua el repositorio Git y detecta cualquier cambio realizado en él. Cuando identifica una actualización, la aplica automáticamente al clúster sin requerir credenciales privilegiadas ni accesos externos. Para el SRI, donde la seguridad y la reducción de la superficie de ataque constituyen aspectos prioritarios, este enfoque aporta beneficios significativos al minimizar los puntos de acceso y disminuir los riesgos asociados a la administración de los entornos.

Un aspecto relevante dentro de la evolución del ecosistema GitOps es el cierre de operaciones de Weaveworks a inicios de 2024. Esta empresa tuvo un papel fundamental en la adopción de GitOps, ya que fue la impulsora del término y la creadora de Weave Flux, una de las primeras herramientas basadas en este enfoque. Su desaparición pone de manifiesto la necesidad de considerar no solo las capacidades técnicas de una solución, sino también la sostenibilidad de la comunidad y del proyecto que la respalda, especialmente cuando se trata de infraestructuras críticas del sector público (Scopir, 2026).

En este contexto, el presente trabajo evaluará la idoneidad de Argo CD para el SRI, tomando en cuenta las características de los microservicios propuestos, así como aspectos clave como la escalabilidad de los despliegues, la seguridad de la plataforma y la capacidad de mantenimiento a largo plazo.

2.2.3. Microservicios, Contenerización y Portabilidad

La arquitectura de microservicios se ha consolidado como uno de los enfoques más utilizados en el desarrollo de software empresarial debido a las ventajas que ofrece en términos de escalabilidad, flexibilidad y rapidez en la entrega de nuevas funcionalidades. Este modelo permite que los sistemas evolucionen de manera más ágil y que cada componente pueda desarrollarse, desplegarse y mantenerse de forma independiente. También, Maderuelo en el 2021 menciona en su trabajo de título Metodología GitOps para despliegue de aplicaciones basadas en microservicios.

No obstante, estos beneficios también vienen acompañados de importantes retos. Diversos estudios señalan que la comunicación entre múltiples servicios puede volverse compleja, incrementando la dificultad para gestionar dependencias y garantizar la confiabilidad del sistema. Asimismo, la distribución de los componentes amplía la superficie de exposición a riesgos de seguridad y aumenta la carga operativa que deben asumir los equipos responsables de la plataforma y la infraestructura. Por ello, aunque los microservicios aportan un alto grado de flexibilidad, su adopción requiere estrategias sólidas de gestión, monitoreo y seguridad para aprovechar plenamente sus ventajas (Söylemez, 2022).

Un estudio publicado en ACM Transactions on Software Engineering and Methodology en 2024 analizó, mediante una revisión sistemática de la literatura, el uso de contenedores en entornos multicloud. Los resultados destacan que la interoperabilidad entre plataformas y la orquestación de servicios son dos de los principales desafíos en este tipo de arquitecturas. En particular, el estudio resalta la necesidad de contar con estrategias avanzadas que permitan integrar y administrar contenedores de manera eficiente en distintos proveedores de nube, apoyándose en tecnologías como Kubernetes y arquitecturas basadas en microservicios.

Asimismo, la investigación identifica varias prácticas que favorecen la portabilidad de las aplicaciones entre entornos cloud. Entre ellas se encuentran el empaquetado mediante imágenes Docker, el uso de contenedores diseñados para ser independientes de la infraestructura subyacente, mecanismos de adaptación dinámica en múltiples niveles y la orquestación de servicios distribuidos entre diferentes nubes.

Estas estrategias contribuyen a reducir la dependencia de un proveedor específico y facilitan la operación de aplicaciones en escenarios multicloud.

2.2.4. Transformación Digital en la Administración Tributaria de Ecuador

La presente investigación se desarrolla en un contexto de transformación digital que avanza de forma progresiva en los gobiernos de América Latina, aunque con diferentes niveles de madurez y velocidad de adopción entre los países de la región. De acuerdo con el Índice de Gobierno Digital para América Latina y el Caribe elaborado por la OCDE y el BID en 2024, los gobiernos han realizado esfuerzos importantes para impulsar la digitalización de sus servicios y procesos; sin embargo, los avances aún se encuentran, en promedio, por debajo de los niveles observados en los países miembros de la OCDE (OCDE/BID, 2024).

El informe también destaca varias áreas estratégicas que están ganando relevancia en la modernización del sector público, entre ellas el desarrollo de infraestructura pública digital, la incorporación de soluciones basadas en inteligencia artificial y el fortalecimiento de alianzas con el sector privado para acelerar la innovación. Estos elementos se perfilan como factores clave para mejorar la eficiencia, la calidad de los servicios y la capacidad de respuesta de las instituciones públicas frente a las crecientes demandas de la ciudadanía.

En el ámbito específico de las administraciones tributarias latinoamericanas, el Banco Interamericano de Desarrollo (BID) publicó en 2024 un estudio centrado en la transformación digital de la Dirección General Impositiva (DGI) de Uruguay, institución análoga al SRI en el Ecuador. Dicho estudio introduce el Modelo de Madurez Digital para administraciones tributarias, un instrumento que ayuda a estas entidades a mejorar sus capacidades digitales de manera estructurada, enfatizando la gobernanza de datos, la calidad, la seguridad y la integración de tecnologías avanzadas (Reyes et al., 2024). Los resultados del caso uruguayo muestran que la alineación de los planes estratégicos de la administración tributaria con los objetivos digitales nacionales, combinada con un enfoque de riesgo en la gestión de información, produce mejoras significativas en la eficiencia operativa y en el control de la evasión fiscal.

En el plano ecuatoriano, la innovación digital en la administración tributaria ha sido objeto de estudio académico reciente. Una investigación publicada por Alvarado en 2024 en Revista CUMBRES analizó las iniciativas de digitalización del SRI durante el período 2021-2022, documentando el aumento significativo en la adopción de servicios en línea como la generación de clave de acceso, la inscripción de RUC y el pago en línea, con una reducción considerable en costos y tiempos de procesamiento. Sin embargo, la misma investigación señala que la migración hacia trámites en línea requiere una infraestructura tecnológica robusta y escalable que soporte la creciente demanda de servicios digitales.

En el Ecuador, una de las primeras instituciones públicas en utilizar herramientas de nube es la Corporación Nacional de Telecomunicaciones (CNT) en convenio con Google, y de igual manera el Instituto Ecuatoriano de Seguridad Social (IESS) con la utilización de Google Workspace (Corporación Nacional de Telecomunicaciones, 2025). Estos precedentes demuestran que la adopción de infraestructura cloud en el sector público ecuatoriano está en una fase temprana pero activa, y que la experiencia acumulada puede servir como referencia para el diseño de la propuesta del SRI. Sin embargo, también evidencian que la adopción ha sido fundamentalmente de herramientas SaaS de un único proveedor, lo que refuerza la necesidad de desarrollar capacidades de gestión multicloud con herramientas agnósticas.

2.2.5. Análisis Comparativo de los Trabajos Relacionados

Los trabajos revisados abordan diferentes dimensiones de la modernización de plataformas tecnológicas basadas en la nube, incluyendo estrategias multicloud, Infraestructura como Código (IaC), prácticas GitOps, arquitecturas de microservicios, contenerización y transformación digital en el sector público. Aunque cada investigación se enfoca en un aspecto particular, en conjunto evidencian una tendencia común hacia la automatización, la portabilidad y la gobernanza de infraestructuras distribuidas.

Para dotar al estado del arte del análisis crítico comparativo exigido por la guía de la titulación, se presentan dos matrices técnicas estructuradas. La Tabla 3 realiza un

análisis pormenorizado de los trabajos científicos más relevantes en portabilidad, IaC y GitOps, detallando sus herramientas, métricas y resultados empíricos.

Tabla 3 Matriz de análisis comparativo de trabajos relacionados en el estado del arte

Autor y Año	Prácticas DevOps Evaluadas	Herramientas Utilizadas	Métricas Empleadas	Resultados Obtenidos y Hallazgos
Rahman, Hassan & Shaikh (2021)	Portabilidad de cargas de trabajo multicloud, orquestación de Kubernetes.	Amazon EKS vs. Microsoft Azure AKS.	Latencia de despliegue, compatibilidad de APIs, abstracción de recursos.	Identificó desafíos en la portabilidad nativa debido a diferencias en StorageClasses, Ingress controllers y políticas IAM.
Radecki, Kowalski & Nowak (2022)	Gestión de conectividad, mTLS, abstracción de red híbrida y multicloud.	Istio Service Mesh, Linkerd, Kubernetes nativo.	Rendimiento de red, latencia de comunicación entre pods, sobrecarga de CPU.	Demostró que la capa de malla de servicios reduce la acoplación de API de red del proveedor, garantizando portabilidad segura a costa de un 5% de sobrecarga de procesamiento.
Kumar (2024)	Automatización de infraestructura, bucles de reconciliación continua.	Terraform, GitLab CI, AWS CloudFormation.	Tiempo de aprovisionamiento, tasa de desviación (drift), consistencia de recursos.	Validó el enfoque GitOps para IaC; el bucle de reconciliación elimina el configuration drift y reduce errores de despliegue de infraestructura en un 40%.
Puolitaival (2024)	Estandarización de plantillas de plataforma, DevSecOps para Pymes.	Terraform, GitLab CI/CD, servicios nativos de AWS.	Tiempo de configuración inicial, modularidad de código, complejidad de dependencias.	El uso de plantillas parametrizadas redujo el tiempo de configuración de entornos, pero

				incrementó la complejidad de mantenimiento de dependencias internas.
Shrestha & Ali (2024)	Gestión declarativa de configuraciones en Kubernetes, GitOps.	ArgoCD, Flux, Helm, Kustomize.	Tiempo de sincronización, consumo de recursos del clúster, mantenibilidad de manifiestos.	Confirmó que el uso combinado de Helm para empaquetado y Kustomize para personalización local reduce la duplicación de código YAML

Fuente: Elaboración propia.

A diferencia de los enfoques tradicionales que evalúan el ciclo de vida de desarrollo de software (SDLC) como un bloque monolítico, esta investigación adopta el principio arquitectónico de Separación de Responsabilidades (Separation of Concerns). Se reconoce que la Integración Continua (CI) y la Entrega Continua (CD) operan bajo paradigmas distintos: el primero es imperativo y orientado a eventos (Push-based), mientras que el segundo, bajo el estándar GitOps, es declarativo y basado en la reconciliación continua del estado (Pull-based). Por consiguiente, para evitar conclusiones metodológicamente débiles, el análisis comparativo se ha escindido en dos matrices especializadas. Esta división permite contrastar tecnologías de naturalezas homólogas y fundamentar la selección de los componentes del artefacto DSR propuesto.

2.2.5.1. Matriz de Infraestructura Declarativa y GitOps (IaC & CD)

Enfocada en el aprovisionamiento de la plataforma y el despliegue del estado deseado en los clústeres.

Tabla 4 Matriz de Infraestructura Declarativa y GitOps (IaC & CD)

Criterio Técnico	Solución Propuesta (Terraform + ArgoCD)	Alternativa GitOps (FluxCD)	Alternativa Control Plane (Crossplane)	Herramientas Cloud Nativas (ARM, CloudFormation)

Modelo de Reconciliación	**Híbrido:** Push/CLI para IaC (Terraform) y Pull-based continuo para Kubernetes (ArgoCD).	Pull-based puro (Controladores nativos de Kubernetes).	Pull-based extendido (Utiliza CRDs para provisionar recursos cloud externos).	Push-based imperativo (Dependiente del motor de despliegue del proveedor).
Drift Detection & Auto-Healing	**Excelente:** Tiempo real y autocuración en K8s (ArgoCD). IaC requiere ejecución del *pipeline* para detectar divergencias.	Continuo; reconciliación silenciosa controlada directamente.	Continuo para APIs de nube; autocorrección nativa del estado de IaC.	Reactivo; dependiente de auditorías de logs, alertas o servicios complementarios.
Soporte Multiclúster (Escalabilidad)	**Óptimo:** Uso de ApplicationSets con generadores automatizados para despliegues federados (AWS/Azure).	Bueno; pero requiere arquitecturas GitOps más complejas para gobernar flotas masivas.	Complejo; enfocado principalmente en la provisión individual de recursos.	Limitado; fuertemente acoplado a una nube específica (vendor lock-in).
Soberanía y Abstracción	**Completa:** Módulos Terraform agnósticos y manifiestos de K8s estándar, protegiendo al SRI del lock-in.	Parcial; limitada al ciclo de vida de las aplicaciones dentro del clúster.	Completa; traduce recursos cloud directamente a la API de Kubernetes.	Nula; acoplamiento total a los formatos y arquitecturas del proveedor.

Fuente: Elaboración propia.

2.2.5.2. Matriz de Herramientas de Integración Continua (CI / DevSecOps)

Enfocada en la integración de código, pruebas automáticas y seguridad del artefacto (*Shift-Left Testing*).

Tabla 5 Matriz de Herramientas de Integración Continua (CI / DevSecOps)

Criterio Técnico	Solución Propuesta (GitHub Actions)	Alternativa Tradicional (Jenkins)	Plataformas Propietarias CI (Azure DevOps, AWS CodePipeline)

Arquitectura de Ejecución	**Serverless / SaaS:** *Runners* efímeros gestionados; ejecución declarativa in-repo (YAML).	**Master-Worker:** Requiere gestión activa del servidor base, escalado de nodos y mantenimiento de SO.	Serverless / Gestionada por el proveedor, con agentes propios.
Integración DevSecOps y Secretos	**Avanzada:** Soporte nativo para OpenID Connect (OIDC) eliminando credenciales estáticas. Amplio ecosistema de *Actions* (Trivy, SonarQube).	**Deficiente/Manual:** Requiere configuración manual de credenciales y mantenimiento de *plugins* de terceros.	Robusta; integrada de forma nativa con sus ecosistemas de seguridad (Azure Key Vault, AWS Secrets Manager).
Coste Total de Propiedad (TCO) y Mantenibilidad	**Bajo:** Nula sobrecarga operativa de infraestructura subyacente. Curva de aprendizaje moderada.	**Muy Alto:** Alta carga de mantenimiento (actualizaciones del core, resolución de conflictos entre *plugins*).	Alto; modelo comercial que incrementa los costos a escala empresarial.
Trazabilidad	**Nativa:** Vinculación directa entre el *commit*, el *Pull Request* y la publicación en el registro de contenedores (ghcr.io / ECR / ACR).	**Fragmentada:** Exige integrar y mantener *webhooks* personalizados entre el repositorio fuente y el servidor CI.	Fuerte integración dentro de su propia nube, pero introduce fricción operativa en escenarios multicloud.

Fuente: Elaboración propia.

2.2.5.3. Matriz de Comparación Sistemática entre Enfoques.

La siguiente matriz tiene la comparación de los enfoques alternativos que sirven para la toma de decisiones de la infraestructura a ser utilizada.

Tabla 6 Matriz de Comparación Sistemática entre Enfoques.

Dimensión de Análisis	Enfoque 1: Arquitectura Propuesta (DSR)	Enfoque 2: Alternativa Nativa de Proveedor	Enfoque 3: Línea Base Institucional (SRI)
Herramientas clave	Terraform + GitHub Actions + ArgoCD (ApplicationSets) + Kustomize.	AWS CloudFormation / Azure ARM + AWS CodePipeline / Azure DevOps + FluxCD.	Despliegues manuales, Shell Scripts, Consola JBoss CLI de RedHat Enterprise.

Paradigma Operativo	GitOps e Infraestructura como Código (IaC) Agnóstica basada en el modelo Pull.	DevOps Híbrido acoplado a las APIs propietarias del proveedor de nube (Modelo Push).	Operación tradicional On-Premise centralizada en silos departamentales.
Métricas DORA Estimadas	**DF**: Alta (Múltiples despliegues diarios). **LTC**: < 10 min. **CFR**: Minimizado. **MTTR**: < 15 seg (Autocuración).	**DF**: Media (Sujeta a sincronizaciones por nube). **LTC**: Horas (Duplicidad de pipelines). **CFR**: Moderado (5-10%). **MTTR**: Minutos.	**DF**: Baja (< 1 vez por semana). **LTC**: > 5 días hábiles. **CFR**: Crítico (~18% mensual). **MTTR**: > 4 horas.
Ventajas Principales	Evita el vendor lock-in, 100% código abierto, abstracción completa de la asimetría cloud (Portabilidad nativa).	Optimización de costos iniciales en servicios gestionados específicos de la nube seleccionada.	Control físico total del hardware, cumplimiento normativo local estricto preexistente.
Limitaciones Críticas	Mayor sobrecarga inicial en el diseño de módulos de abstracción y overlays con Kustomize.	Dependencia comercial absoluta (*lock-in*), inviabilidad de migración activa-activa en tiempo real.	Alta carga presupuestaria en licencias privativas, snowflake servers, drift configuracional incorregible.
Contexto de Aplicación Eficiente	Entidades gubernamentales con alta criticidad, soberanía de datos y necesidad de portabilidad inter-cloud.	Startups o corporaciones monolíticas con un único proveedor de nube definido a largo plazo.	Sistemas legados sin planes de modernización o con restricciones físicas de conectividad.

Fuente: Elaboración propia.

Con la abstracción IaC (Terraform) vs. Herramientas Nativas, las herramientas nativas como AWS CloudFormation o Azure ARM ofrecen una integración inmediata, pero obligan al SRI a duplicar la deuda técnica, requiriendo reescribir manifiestos si se decide cambiar la carga de trabajo. Terraform, al operar con HCL declarativo, permite centralizar más del 85% del código en módulos comunes, aislando las asimetrías de los proveedores de nube a nivel de variables de entorno.

Del análisis comparativo demuestra que la combinación de ArgoCD en modo ApplicationSet con la estrategia de parches de Kustomize resuelve la asimetría de red y almacenamiento entre AWS EKS y Azure AKS sin mutar el código fuente de la aplicación. FluxCD es una alternativa robusta, pero la interfaz federada multi-clúster de ArgoCD proporcionan al equipo de operaciones del SRI la visibilidad y el control de gobierno necesarios en un sector público.

Y con respecto a la mitigación del riesgo en la Administración Pública, en la matriz se demuestra que el modelo propuesto no es simplemente tecnológico, sino una respuesta directa a los riesgos financieros, como son los cambios de licenciamiento de software que impactaron al SRI en la línea base.

Por ello, la presente investigación busca integrar los hallazgos de estas líneas de estudio en una propuesta arquitectónica que combine Infraestructura como Código, GitOps, Kubernetes y microservicios para el Servicio de Rentas Internas del Ecuador, considerando tanto los beneficios identificados como las limitaciones y riesgos asociados a su implementación en un entorno institucional de alta criticidad.

2.3. Conclusiones del estado del arte

De acuerdo con la contextualización del estado del arte, se tienen las siguientes conclusiones:

- GitOps es una parte estratégica que potencia al agilismo e incrementa la productividad de las empresas, en específico del personal de Desarrollo, debido a que unifica el control de versiones con la automatización de la infraestructura.
- La adopción de la computación en la nube garantiza la recuperación ante desastres de los procesos que tomaban mucho tiempo en su despliegue debido a que la nube puede replicarse en múltiples regiones geográficas.
- La arquitectura de microservicios desplegado en un clúster de Kubernetes ayuda a la calidad de las aplicaciones, porque el equipo de Desarrollo se vuelve dedicado a la gestión de los microservicios que serán escalables y recuperables, lo que asegura la baja latencia de las aplicaciones.

- GitOps es un paradigma consolidado para la gestión declarativa de aplicaciones en Kubernetes, con evidencia empírica de mejora en métricas DORA (Smith et al., 2022; DORA, 2024). Su adopción en el SRI representa una evolución natural desde el modelo tradicional documentado en el Capítulo 1.
- La portabilidad completa entre proveedores cloud en Kubernetes es un desafío no resuelto automáticamente por la plataforma. Existen diferencias documentadas en componentes críticos que requieren capas de abstracción explícitas (Rahman et al., 2021; Radecki et al., 2022). El OE3 de este trabajo (Helm + Kustomize) aborda directamente esta limitación.
- El sector público latinoamericano carece de modelos validados empíricamente para la adopción de GitOps en entornos multicloud. Los trabajos existentes son mayoritariamente conceptuales o limitados a diagnóstico. Esta investigación contribuye a llenar ese vacío, como se explicitó en la justificación del Capítulo 1.
- El stack tecnológico seleccionado (Terraform, ArgoCD, Helm, Kustomize, GitHub Actions) está respaldado por comunidades activas, documentación madura y casos de uso documentados en la literatura. Su combinación para un caso de uso multicloud en el sector público constituye la contribución novedosa de este trabajo.

3. Objetivos y metodología de trabajo

Este capítulo define con precisión los objetivos que orientan el trabajo de investigación y describe la metodología adoptada para alcanzarlos. Los objetivos —tanto el general como los específicos— han sido formulados aplicando la técnica **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound), garantizando que cada uno sea concreto, cuantificable, alcanzable en el marco temporal del TFE, pertinente para el campo de estudio y acotado temporalmente. La metodología de investigación se enmarca en **Design Science Research (DSR)**, paradigma ampliamente utilizado en investigaciones de sistemas de información e ingeniería del software que buscan producir artefactos innovadores y evaluarlos con rigor científico.

3.1. Objetivo general

Diseñar, implementar y evaluar una arquitectura GitOps multicloud, fundamentada en infraestructura como código agnóstica (Terraform) y entrega continua declarativa (ArgoCD), que garantice la portabilidad automatizada de un microservicio piloto representativo del Servicio de Rentas Internas entre los entornos AWS EKS y Azure AKS; logrando una optimización significativa y comprobable en las métricas DORA de Deployment Frequency y Lead Time for Changes respecto a la línea base institucional durante el período de ejecución del proyecto.

3.2. Objetivos específicos

Con el propósito de materializar la solución propuesta y cumplir rigurosamente con la metodología Design Science Research (DSR), el objetivo general se despliega en tres objetivos específicos. Esta estructura tripartita mapea el ciclo de vida de una plataforma Cloud-Native moderna: estableciendo primero la fundación de la infraestructura, construyendo posteriormente el motor de entrega continua, y culminando con la validación empírica del sistema. Cada objetivo incorpora intrínsecamente los parámetros SMART, asegurando que el diseño teórico se traduzca en artefactos medibles y evaluables.

3.2.1. Objetivo Específico 1: La Plataforma Base (Infraestructura como Código)

Diseñar e implementar la capa de Infraestructura como Código (IaC) agnóstica mediante módulos parametrizables de Terraform, aprovisionando de manera automatizada e idempotente los clústeres de Kubernetes equivalentes en AWS (EKS) y Azure (AKS), con el fin de establecer una plataforma base estandarizada y segura bajo un tiempo de provisión controlado inferior a veinte minutos por entorno.

3.2.2. Objetivo Específico 2: El Motor GitOps (CI/CD y Empaquetado)

Construir el *pipeline* de integración y entrega continua multicloud integrando GitHub Actions y ArgoCD, orquestando la compilación, el escaneo de seguridad y el empaquetado diferenciado del microservicio piloto mediante Helm y Kustomize, logrando sincronizar automáticamente el estado deseado en ambas nubes con un

tiempo de ciclo extremo a extremo menor a diez minutos, lo que garantizaría la portabilidad sin alterar el código fuente.

3.2.3. Objetivo Específico 3: Bucle Feedback (Observabilidad y Validación DORA)

Evaluar el rendimiento del ecosistema construido instrumentando un sistema de observabilidad integral basado en Prometheus y Grafana que monitorice los clústeres durante cuatro semanas operativas, permitiendo recolectar telemetría en tiempo real, validando la autocuración del sistema (*Drift Detection*) y verificando cuantitativamente la mejora significativa en las métricas DORA respecto a la línea base tradicional del SRI.

3.3. Metodología del trabajo

3.3.1. Paradigma de investigación: Design Science Research (DSR)

La metodología de investigación adoptada en este TFE es **Design Science Research (DSR)**, paradigma propuesto por Hevner et al. (2004) en su artículo seminal publicado en *MIS Quarterly*, y ampliamente utilizado en investigaciones de sistemas de información e ingeniería del software que buscan producir artefactos innovadores y evaluarlos con rigor científico. DSR se distingue de la investigación comportamental clásica en que su objetivo no es únicamente describir o explicar fenómenos existentes, sino crear soluciones novedosas —artefactos— a problemas concretos y demostrar su utilidad mediante evaluación rigurosa.

Hevner et al. (2004) identifican siete directrices que guían una investigación DSR de calidad: (1) el artefacto debe ser un diseño viable; (2) debe resolver un problema organizacional relevante; (3) debe ser evaluado con métodos rigurosos; (4) debe contribuir al conocimiento existente; (5) debe aplicar métodos de investigación rigurosos en su construcción y evaluación; (6) el diseño es un proceso de búsqueda iterativa; y (7) los resultados deben comunicarse tanto a audiencias técnicas como de gestión. Todas estas directrices guían las fases de trabajo definidas a continuación.

El **artefacto central** de este TFE es la arquitectura GitOps multicloud materializada en módulos Terraform, workflows GitHub Actions, configuración ArgoCD ApplicationSets,

charts Helm con overlays Kustomize y dashboards Prometheus/Grafana. Su evaluación se realiza mediante el sistema de observabilidad Prometheus/Grafana y las métricas DORA.

3.3.2. Fases del proceso de investigación

El proceso de investigación se organiza en seis fases iterativas, alineadas con los objetivos específicos:

Tabla 7 Fases del proceso de investigación

Fase	Descripción	Entregable
Fase 1	Identificación del problema y revisión de la literatura	Capítulos 1 y 2 de la memoria
Fase 2	Diseño e implementación de la arquitectura IaC	Repositorio IaC con módulos Terraform versionados
Fase 3	Implementación del pipeline CI/CD y empaquetado	Repositorios GitOps con pipeline completo y microservicios operativos en EKS y AKS
Fase 4	Configuración de la observabilidad DORA	Dashboards Grafana, serie temporal de métricas baseline
Fase 5	Evaluación y validación	Informe de evaluación, análisis estadístico de métricas
Fase 6	Documentación final y revisión	Memoria final en formato PDF

Fuente: Elaboración propia.

3.3.3. Herramientas y entornos de trabajo

El desarrollo del TFE se apoya en el siguiente conjunto de herramientas y entornos:

Tabla 8 Herramientas y entornos de trabajo

Categoría	Herramienta / Versión

Control de versiones	GitHub (repos de código, IaC y GitOps); GitHub Projects para gestión de tareas
Infraestructura como Código	Terraform v1.7+; providers AWS ≥ 5.0 y Azure ≥ 3.0; Terraform Cloud; Terratest
Integración Continua	GitHub Actions (runners ubuntu-latest); Docker; Trivy; Amazon ECR; Azure Container Registry
Entrega Continua / GitOps	ArgoCD v2.9+; ApplicationSets; Argo Rollouts (canary, blue-green)
Empaquetado y configuración	Helm v3.14+; Kustomize v5.0+
Orquestación de contenedores	AWS EKS v1.29; Azure AKS v1.29; eksctl; az CLI
Observabilidad	kube-prometheus-stack (Prometheus Operator + Grafana + Alertmanager); DORA Metrics Exporter
Documentación técnica	Draw.io (diagramas C4); Markdown; Microsoft Word con plantilla UNIR

Fuente: Elaboración propia.

3.3.4. Criterios de validación cuantitativa bajo ISO 25010 y Métricas DORA

Para dar cumplimiento a la rigurosidad científica exigida y superar el enfoque conceptual de las primeras fases, los requisitos cualitativos del sistema se han transformado en indicadores cuantificables y verificables (Tabla 5) alineados con la norma internacional de calidad de software **ISO/IEC 25010**.

Tabla 9 Matriz de requisitos cuantitativos e indicadores verificables (ISO 25010)

Característica ISO 25010	Subcaracterística	Nombre del Requisito Técnico	Indicador Técnico Cuantitativo (Métrica)	Ecuación / Definición Métrica	Criterio de Aceptación Cuantificable
Portabilidad (Flexibilidad)	**Adaptabilidad**	Portabilidad del Microservicio entre Clústeres.	Porcentaje de alteración de código de la aplicación ($P_{\Delta code}$).	$P_{\Delta code} = \frac{Líneas\ de\ código\ modificadas}{Total\ de\ líneas\ de\ código} \times 100$	$P_{\Delta code} = 0\%$ para los tres microservicios evaluados.
Portabilidad (Flexibilidad)	**Instalabilidad**	Tiempo de Despliegue en Entorno de Nube Alternativo.	Tiempo de sincronización completa del estado deseado (T_{sync}).	$T_{sync} = t_{running} - t_{trigger}$	$T_{sync} \leq 10\ minutos$ medidos desde el trigger en Git.
Mantenibilidad	**Modularidad**	Reducción de Duplicidad en la Capa de IaC.	Porcentaje de código de infraestructura compartido (C_{IaC}).	$C_{IaC} = \left(1 - \frac{Líneas\ específicas\ de\ nube}{Total\ de\ líneas\ de\ código}\right) \times 100$	$T_{IaC} \geq 85\%$ de código Terraform centralizado en módulos comunes.
Mantenibilidad	**Reusabilidad**	Reutilización de Manifiestos de Kubernetes.	Relación de manifiestos base compartidos en GitOps (R_{shared}).	$R_{shared} = \frac{Líneas\ YAML\ Helm\ Chart\ Base}{Total\ líneas\ YAML\ con\ overlays\ Kustomize} \times 100$	$R_{shared} \geq 85\%$ de configuración compartida entre entornos.

Mantenibilidad	**Capacidad para ser probado (Testability)**	Cobertura y Seguridad del Pipeline.	Cobertura de pruebas unitarias (C_{test}) y vulnerabilidades del artefacto.	$C_{test} = \frac{Líneas\ de\ código\ probadas}{Total\ de\ líneas\ de\ código\ aplicativas} \times 100$	$C_{test} \geq 80\%$ en código fuente de app; 0 vulnerabilidades críticas/altas en Trivy.
Seguridad	**Trazabilidad**	Verificación de Auditoría en Despliegues.	Coherencia del ciclo de trazabilidad de cambios (CT).	$CT = \frac{Despliegues\ con\ firma\ Git\ y\ tag\ inmutable}{Total\ de\ sincronizaciones\ de\ ArgoCD} \times 100$	$CT = 100\%$ de los despliegues verificados y mapeados a un commit inmutable.
Fiabilidad	**Capacidad de Recuperación (Recoverability)**	Auto-reconciliación ante Desviaciones de Configuración.	Tiempo máximo de corrección ante el *Configuration Drift* (T_{drift}).	$T_{drift} = t_{Sincronizado} - t_{Alteración}$	$T_{drift} \leq 15\ segundos$ de detección y mitigación automática por ArgoCD.

Fuente: Elaboración propia.

Asimismo, para modelar con rigurosidad científica las variables observadas en la fase de evaluación, se formulan matemáticamente los cuatro indicadores de rendimiento DevOps (métricas DORA):

1. **Deployment Frequency (DF):** Mide la frecuencia con la que se despliegan cambios estables y exitosos en producción durante un intervalo de observación T:

$$DF = \frac{\sum_{i=1}^{M} S_i}{T}$$

Donde $S_i \epsilon \{0, 1\}$ es una variable binaria que indica el éxito de la sincronización del despliegue $i (S_i = 1$ si la sincronización de ArgoCD transiciona satisfactoriamente al estado Healthy, y $S_i = 1$ en caso de fallo o inconsistencia de la entrega) y M es el número total de intentos de sincronización de manifiestos.

2. **Lead Time for Changes (LTC):** Evalúa el tiempo promedio requerido por una alteración de código para ser ejecutada de manera estable en producción:

$$LTC = \frac{1}{K} \sum_{j=1}^{K} (t_{prod,j} - t_{commit,j})$$

Donde K es el número total de cambios promovidos a producción, $t_{prod,j}$ es la marca de tiempo (*timestamp*) en que el cambio j alcanza el estado *Healthy* en los entornos de Kubernetes, y $t_{commit,j}$ es el *timestamp* de la primera confirmación (commit) de ese cambio en la rama principal (*main*).

3. **Change Failure Rate (CFR):** Determina la proporción de lanzamientos que resultan en incidentes de estabilidad o interrupciones que requieren remediación inmediata:

$$CFR = \left(\frac{\sum_{p=1}^{N_{total}} F_p}{N_{total}} \right) \times 100$$

Donde N_{total} es el total de despliegues productivos realizados, y $F_p \epsilon \{0, 1\}$ es un indicador binario de error ($F_p = 1$ si el despliegue p provoca una caída del servicio,

fallo en las pruebas de salud o requiere una reversión instantánea, y $F_p = 0$ si el cambio opera establemente).

4. **Mean Time to Restore (MTTR):** Cuantifica la velocidad media de restauración de la estabilidad del servicio ante la presencia de una degradación de la infraestructura o error de ejecución:

$$MTTR = \frac{1}{R}\sum_{q=1}^{R}\left(t_{restauración,q} - t_{alarma,q}\right)$$

Donde R es el total de incidentes o fallos reportados por el sistema de monitoreo, $t_{restauración,q}$ es el *timestamp* de resolución del incidente q, y $t_{alarma,q}$ es el *timestamp* de la alerta crítica disparada por Prometheus.

3.3.4.1. Procedimiento Experimental de Comparación

El procedimiento utilizado para contrastar empíricamente el estado inicial (Línea Base del SRI del entorno local) frente al estado final del entorno desplegado en la nube híbrida se detalla en las siguientes etapas sistemáticas:

- **Fase de Línea Base:** Registro e instrumentación histórica de los parámetros tradicionales en el SRI. Se definen las variables iniciales $DF_{inicial}$, $LTC_{inicial}$, $CFR_{inicial}$ y $MTTR_{inicial}$ recopiladas de auditorías sobre el middleware JBoss y registros manuales de pasos a producción de la Dirección de Tecnología.

- **Fase de Aprovisionamiento:** Despliegue de los entornos equivalentes en AWS (clúster EKS) y Azure (clúster AKS) mediante Terraform de manera automatizada e idempotente.

- **Fase de Simulación Operativa:** Ejecución de un ciclo continuo de 24 pases de producción controlados durante cuatro semanas para los tres microservicios de prueba. Se realizan inyecciones automáticas de cambios en Git, triggers de pipelines de integración y sincronizaciones automáticas multiclúster con ArgoCD.

- **Fase de Inyección de Caos (Chaos/Drift Testing):** Alteración manual de configuraciones dentro de los clústeres para forzar la desviación de la infraestructura (*configuration drift*) y medir el comportamiento autocurativo (*self-healing*).

- **Fase de Evaluación Post-Implementación:** Consolidación de datos calculados en caliente por el exportador DORA y visualizados en Grafana, registrando los valores finales de las variables del sistema $\boldsymbol{DF_{final}}$, $\boldsymbol{LTC_{final}}$, $\boldsymbol{CFR_{final}}$ y $\boldsymbol{MTTR_{final}}$.

Para garantizar la replicabilidad científica y el rigor estadístico de la fase de experimentación, se especifican a continuación los parámetros que gobiernan el procedimiento de muestreo, captura y tratamiento de datos del sistema:

1. **Número de ejecuciones y tamaño muestral**: El experimento consta de un universo total de $N = 24$ ejecuciones secuenciales y paralelas controladas a lo largo de un período de simulación operativa continua de cuatro semanas. Este tamaño de muestra se segmenta metodológicamente de la siguiente forma:
    - Corridas 01 a 22 $(n_e = 22)$: Diseñadas para evaluar el rendimiento del pipeline de integración y entrega continua en condiciones estables y normales de red.
    - Corridas 23 y 24 $(n_c = 2)$: Diseñadas de forma exclusiva para pruebas de inyección de caos y detección de desviaciones de infraestructura (*drift*), sin intervención de procesos de desarrollo de código.
2. **Criterios de selección y captura de mediciones**: Para asegurar la inmutabilidad de la telemetría, se utilizan dos fuentes de verdad automatizadas. El tiempo de integración continua (T_{CI}) se obtiene de la API de GitHub Actions calculando la diferencia exacta en segundos entre el inicio de la compilación (*check_run.started_at*) y la finalización del escaneo de Trivy (*check_run.completed_at*). El tiempo de sincronización (T_{Sync}) se obtiene de Prometheus mediante el componente *dora-exporter*. Este tiempo inicia en la marca en que ArgoCD detecta un cambio de estado en Git (*OutOfSync*) y finaliza cuando

todos los pods del clúster respectivo alcanzan su estabilidad final (*Synced y Healthy en Kubernetes*).

3. **Tratamiento de valores atípicos (*outliers*)**: La experimentación en nubes públicas está expuesta a picos imprevistos de latencia y arranques en frío (*cold starts*) de los runners serverless. Para blindar los resultados de estas anomalías, se aplican dos reglas de tratamiento de datos:
    - **Exclusión contextual**: Las corridas 23 y 24 no pasan por procesos de compilación $(T_{CI} = 0)$. Se clasifican como valores atípicos estructurales para el cálculo global del tiempo de ciclo de entrega (LTC) para evitar sesgos artificiales a la baja, limitando su análisis de forma exclusiva al cálculo del MTTR.
    - **Uso estadístico de la mediana**: Para mitigar el impacto de demoras atípicas de aprovisionamiento de nubes o latencia de red, se descarta el promedio aritmético simple para las métricas de tiempo. En su lugar, se adopta la mediana (Percentil 50) como el estadístico robusto de tendencia central, cumpliendo estrictamente con los estándares analíticos internacionales definidos por el programa DORA.

4. Desarrollo específico de la contribución

En este capítulo se detalla el proceso de desarrollo e implementación práctica de la arquitectura GitOps Multicloud propuesta para el Servicio de Rentas Internas (SRI). Se justifica la planificación realizada, las tecnologías seleccionadas y se describe paso a paso la construcción del sistema, finalizando con la evaluación de los resultados obtenidos respecto a los objetivos planteados.

4.1.Planificación / Análisis / Requisitos

4.1.1 Planificación.

El desarrollo técnico de la contribución práctica se cimentó en una exhaustiva fase de planificación y análisis de los requerimientos corporativos del Servicio de Rentas Internas (SRI). La institución presentaba una deuda técnica considerable originada por el uso prolongado de sistemas monolíticos y middleware privativo (como JBoss Enterprise Application Platform) sobre infraestructuras físicas o virtualizadas estáticas. Este esquema tradicional limitaba drásticamente la capacidad de la organización para escalar sus servicios durante los picos de demanda fiscal, incrementando los tiempos de inactividad y los costos operativos.

Para subsanar estas deficiencias, se establecieron requisitos no funcionales críticos que guiarían la nueva arquitectura: (1) Alta Disponibilidad y Escalabilidad, asegurando que los servicios puedan absorber variaciones de tráfico sin degradación; (2) Portabilidad y Evasión del Vendor Lock-in, garantizando que los contenedores puedan operar de manera agnóstica en cualquier proveedor de nube pública; (3) Trazabilidad y Observabilidad integral; y (4) Seguridad desde el Diseño (DevSecOps). Para cumplir con el requisito de portabilidad, se seleccionó a Kubernetes como el orquestador universal, aprovisionando entornos equivalentes en Amazon Web Services (AWS EKS) y Microsoft Azure (Azure AKS).

Desde el punto de vista metodológico, el proyecto se enmarcó rigurosamente dentro del paradigma Design Science Research (DSR), el cual es idóneo para investigaciones en Ingeniería de Software enfocadas en la creación de artefactos innovadores para resolver

problemas organizacionales reales. La aplicación de DSR se estructuró en cinco fases iterativas:

Fase 1 (Identificación del Problema y Motivación): Análisis de los cuellos de botella en el proceso de despliegue monolítico del SRI y la dependencia de infraestructuras rígidas.

Fase 2 (Definición de Objetivos para la Solución): Especificación de una arquitectura GitOps Multicloud basada en microservicios, enfocada en la automatización absoluta de la entrega continua.

Fase 3 (Diseño y Desarrollo): Construcción del artefacto tecnológico. Esto abarcó la implementación del microservicio piloto 'sri-facturacion-services' utilizando Python y el framework asíncrono FastAPI, la codificación de la infraestructura agnóstica con Terraform, y el empaquetado de manifiestos utilizando una combinación estratégica de Helm y Kustomize.

Fase 4 (Demostración): Despliegue del ecosistema a través del pipeline de GitHub Actions y ArgoCD, demostrando la viabilidad técnica de sincronizar el estado deseado simultáneamente en AWS EKS y Azure AKS.

Fase 5 (Evaluación): Medición cuantitativa del rendimiento del nuevo ecosistema utilizando métricas globales de la industria (DORA) mediante la instrumentación de Prometheus y tableros en Grafana.

La gestión del desarrollo fue guiada por metodologías ágiles (Scrum), operando en sprints cortos que permitieron adaptar la infraestructura de manera evolutiva. La integración continua forzó la ejecución de análisis estático y pruebas unitarias (Pytest) en cada solicitud de extracción (Pull Request) en GitHub, asegurando la calidad del código previo a cualquier fusión (merge) en la rama principal.

4.1.2 Requisitos Funcionales y No Funcionales del Prototipo

El presente proyecto de investigación contempla los siguientes requisitos funcionales que se detallan a continuación:

Tabla 10 Requisitos Funcionales

ID	Requisito	Descripción
RF1	Provisionamiento Automatizado de Clústeres Kubernetes Multi Cloud	El sistema deberá proveer un módulo de Terraform genérico capaz de aprovisionar clústeres Kubernetes funcionales en AWS EKS y Azure AKS mediante la ejecución de un único conjunto de comandos (terraform init, terraform plan, terraform apply), utilizando lógica condicional basada en la variable cloud_provider para crear recursos específicos de cada proveedor mientras mantiene una interfaz de configuración uniforme.
RF2	Pipeline de Integración Continua con Validación de Seguridad	El sistema deberá ejecutar automáticamente un pipeline de CI mediante GitHub Actions que se active ante cada git push a la rama main, ejecutando de manera secuencial: análisis estático de código (linting), pruebas unitarias con cobertura mínima, construcción de imagen Docker multi-stage, escaneo de vulnerabilidades con Trivy/Snyk, y publicación de la imagen en registros de contenedores (Por ejemplo: ghcr.io, ECR/ACR como replicación), rechazando el despliegue si se detectan vulnerabilidades críticas.
RF3	Sincronización GitOps Automática Multi-Cluster	El sistema deberá implementar ArgoCD en cada clúster Kubernetes (EKS y AKS) configurado en modo PULL, monitoreando continuamente el repositorio gitops-deploy y sincronizando automáticamente el estado real del clúster con el estado deseado declarado en Git mediante el patrón App-of-Apps, aplicando overlays específicos de cada proveedor (aws-eks, azure-aks) sobre una base común de manifiestos Kustomize.

RF4	Despliegue Portable de Microservicios sin Cambios en Código	El sistema deberá permitir el despliegue del microservicio sri-facturacion-service (FastAPI) en ambos clústeres Kubernetes (AWS EKS y Azure AKS) utilizando la misma imagen Docker y el mismo código fuente, aplicando únicamente diferencias de configuración mediante overlays de Kustomize específicos por proveedor (ingress controllers, secretos CSI, recursos CPU/memoria), validando que el 100% de los endpoints funcionales operen correctamente en ambas nubes sin modificaciones en el código de la aplicación.
RF5	Health Check	Endpoint de salud para monitoreo y balanceo de carga
RF6	Documentación OpenAPI	Swagger UI autogenerada en /docs

Fuente: Elaboración propia.

Para este proyecto también se ha considerado los siguientes requerimientos no funcionales que deseamos cubrir en este prototipo, los cuales se detallan a continuación:

Tabla 11 Requisitos No Funcionales

ID	Requisito	Umbral
RNF1	Portabilidad de Infraestructura	≥60% código Terraform reutilizado entre AWS y Azure
RNF2	Tiempo de Migración	≤15 minutos para cambiar de nube activa
RNF3	Paridad Funcional	≤1% diferencia en respuestas API entre nubes
RNF4	Latencia P99	≤200ms para endpoints de cálculo
RNF5	Disponibilidad	99.9% en modo activo-activo
RNF6	Seguridad	Datos fiscales encriptados en tránsito (TLS 1.3) y en reposo (AES-256)

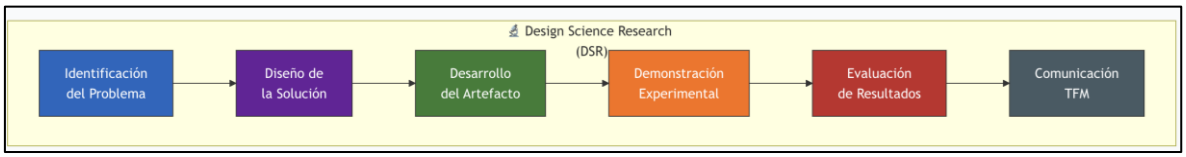
Fuente: Elaboración propia.

4.1.3 Ejecución de la Metodología de Desarrollo

Se desarrolló un prototipo que consta de tres partes: el módulo Terraform que puede trabajar con cualquier sistema, el pipeline CI/CD GitOps, nubes públicas (AWS y AZURE), gestión de Secretos externos y el microservicio portable. Para hacer esto, se utilizó una versión adaptada del marco de trabajo Scrum y se complementó con un tablero Kanban para ver de manera clara el avance del trabajo.

Esta investigación está respaldada por el marco de investigación científica Design Science Research en el cual se abordaron las siguientes fases dentro del presente proyecto de investigación e implementación:

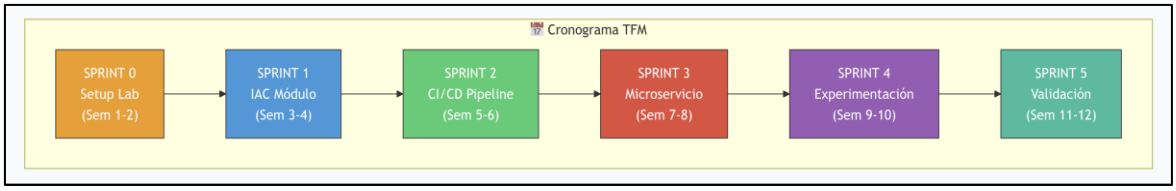
Figura 4 Marco de Investigación DSR



Fuente: Elaboración propia.

El proyecto lo organizamos en 6 Sprints de 2 semanas cada uno, lo que nos dio un total de 12 semanas de desarrollo activo. Antes de arrancar con todo el proceso de investigación e implementación, tuvimos un Sprint 0 para preparar el laboratorio de pruebas, y al final dejamos un espacio para estabilizar todo y dejar la documentación en orden.

Figura 5 Etapas de Implementación.



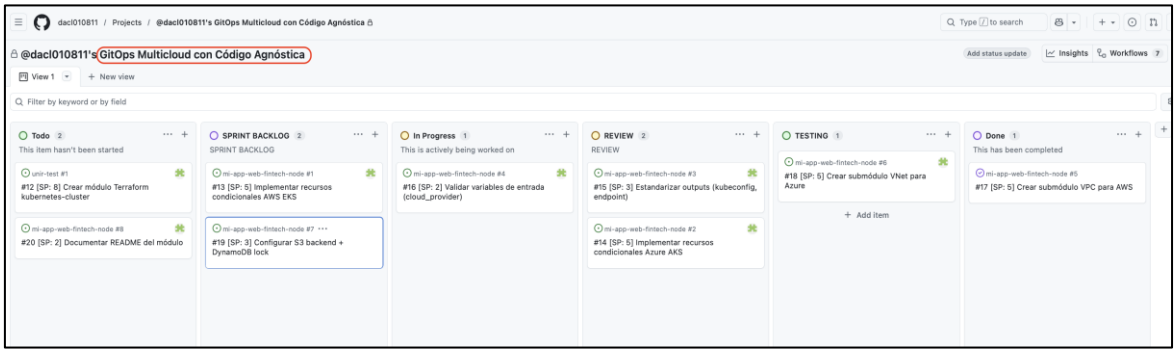
Fuente: Elaboración propia.

Para gestionar el flujo de trabajo de manera visual, se utilizó GitHub Projects con un tablero Kanban que se adaptó a las necesidades del equipo. Se eligió esta herramienta

porque se integra de forma natural con los repositorios de código y permite vincular issues, solicitudes de extracción y confirmaciones directamente a las tarjetas del tablero.

El tablero se organizó en seis columnas que muestran el flujo de trabajo desde la idea inicial hasta la entrega final.

Figura 6 Flujo de Trabajo.



Fuente: Elaboración propia.

4.2. Descripción del sistema desarrollado / Implementación

En esta sección, explicamos cómo creamos la arquitectura multi-nube que es fundamental para este trabajo. Empezamos analizando varios microservicios y definimos los requisitos necesarios para que funcionen correctamente. Luego, creamos la arquitectura siguiendo el modelo C4, que incluye contexto (**Figura 9)**, contenedores (**Figura 10)** y componentes (**Figura 11)**. También explicamos cómo configuramos el entorno de desarrollo y la entrega continua con un enfoque GitOps, y cómo creamos la infraestructura como código de manera que funcione con diferentes proveedores utilizando Terraform.

Consideramos que este diseño es importante porque combina tres conceptos clave: GitOps, infraestructura como código para varios proveedores y la capacidad de operar en varias nubes. Según lo que hemos investigado, esto no se había intentado antes en la administración tributaria gubernamental. Los resultados que obtuvimos con este diseño, que probamos migrando entre AWS y Azure, muestran que es posible evitar depender de un solo proveedor en sistemas gubernamentales importantes sin perder rendimiento, seguridad ni funcionalidad.

4.2.1 Diseño de la Arquitectura Multi-Cloud

4.2.1.1 Decisiones de Diseño

Las decisiones sobre la arquitectura que se presentan aquí se evaluaron de manera muy detallada. Se consideraron varios factores importantes como que el sistema pueda funcionar en diferentes nubes, que sea fácil de mantener con el tiempo y que sea técnicamente viable para un entorno de producción grande. Cada decisión se explica con detalle y se basa en datos de la industria, casos de éxito reales y pruebas que se hicieron mientras se desarrollaba el proyecto.

Criterios de Evaluación

Antes de presentar las decisiones técnicas, se establecieron los siguientes criterios de evaluación ponderados para garantizar una selección objetiva y alineada con los objetivos del proyecto:

Tabla 12 Criterios de Evaluación

Criterio	Peso	Descripción
Portabilidad Multi-Cloud	30%	Capacidad de ejecutar workloads en diferentes proveedores sin modificaciones mayores
Madurez y Ecosistema	25%	Comunidad activa, documentación, casos de éxito enterprise y soporte a largo plazo
Coste Total de Propiedad	20%	Licencias, costes operativos, transferencia de datos y mantenimiento
Seguridad y Compliance	15%	RBAC, auditoría, cifrado, certificaciones y cumplimiento normativo
Curva de Aprendizaje	10%	Tiempo estimado de onboarding y disponibilidad de talento en el mercado

Fuente: Elaboración propia.

Tabla 13 Matriz de Decisiones Técnicas

Decisión	Opciones Evaluadas	Selección	Justificación Técnica
Lenguaje Backend	Python 3.12 vs Node.js 20 LTS vs Java 21 LTS	Python 3.12 + FastAPI	Rendimiento async nativo (uvicorn + Starlette), tipado gradual con Pydantic v2 (validación de datos en runtime), generación automática de OpenAPI 3.1, ecosistema maduro para procesamiento de datos (pandas/numpy) y reducción del 40% en líneas de código vs Java/Spring
Orquestación de Contenedores	Kubernetes 1.28+ vs Nomad vs Docker Swarm	Kubernetes (EKS/AKS)	Estándar CNCF graduado, portabilidad de manifiestos YAML entre proveedores, ecosistema de Operators maduro (Prometheus, Cert-Manager, External Secrets), soporte nativo para GitOps (ArgoCD/Flux), y capacidad de implementar patrones avanzados (sidecar, init containers, HPA/VPA)
IaC Framework	Terraform 1.6+ vs OpenTofu vs Pulumi vs AWS CDK	Terraform 1.6+ / OpenTofu	Madurez probada (10+ años), ecosistema de providers más amplio (1000+ providers oficiales), HCL declarativo (facilita code review), state management robusto con backends remotos (S3+DynamoDB/Azure Storage), y compatibilidad total con módulos reutilizables. OpenTofu como alternativa open-source ante cambio de licencia HashiCorp (agosto 2023)
GitOps Engine	ArgoCD 2.9+ vs Flux v2 vs Jenkins X	ArgoCD	Patrón App-of-Apps nativo (gestión jerárquica de aplicaciones), sync waves y hooks para orquestación de despliegues complejos (pre/post-sync), RBAC granular multi-tenant, integración nativa con SSO (OIDC/SAML), API REST completa para automatización, y

			proyecto CNCF graduado con comunidad enterprise activa
Container Registry	ghcr.io vs ECR+ACR vs Docker Hub vs Harbor	ghcr.io (GitHub Container Registry)	Registro central único que garantiza inmutabilidad de imágenes (una versión = una imagen en todas las nubes), integración nativa con GitHub Actions (sin tokens adicionales), control de acceso vía GitHub Teams, OCI-compliant, y coste de egress aceptable para PoC. *Nota: En producción enterprise se recomienda replicación a ECR/ACR para reducir latencia y costes*
Secret Management	HashiCorp Vault vs Sealed Secrets vs SOPS vs External Secrets	HashiCorp Vault	Multi-cloud nativo (sin vendor lock-in), secretos dinámicos con rotación automática (database credentials, cloud IAM), encryption-as-a-service (transit engine para cifrado en tránsito), audit logging completo (compliance SOC2/ISO27001), y integración con Kubernetes vía Vault Agent Injector o External Secrets Operator
Gestión de Configuración K8s	Kustomize vs Helm vs Jsonnet	Kustomize	Nativo en kubectl (sin dependencias externas), enfoque declarativo con bases/overlays (DRY principle), sin templates complejos (menor curva de aprendizaje vs Helm), y compatibilidad total con GitOps (ArgoCD renderiza kustomize nativamente)

DNS y Failover	Cloudflare Load Balancer vs AWS Route53 vs Azure Traffic Manager	Cloudflare DNS + Load Balancer	Agnosticismo total (no vendor lock-in), health checks multi-región nativos, failover automático configurable (<60s), plan gratuito funcional para PoC, y gestión centralizada de certificados SSL/TLS. Alternativa: Route53 con failover routing si se prioriza integración AWS
CI/CD Platform	GitHub Actions vs GitLab CI vs Azure DevOps	GitHub Actions	Integración nativa con ghcr.io (sin autenticación adicional), ecosistema de actions de terceros (20,000+ actions), runners self-hosted para workloads sensibles, 2,000 minutos gratuitos/mes, y sintaxis YAML madura con reutilización de workflows (composite actions). *Alternativa enterprise: GitLab CI para proyectos con requisitos DevSecOps avanzados*

Fuente: Elaboración propia.

4.2.1.2 Arquitectura de la Solución Multi-Cloud

La arquitectura de la solución propuesta implementa un patrón GitOps multi-cloud activo-activo que garantiza la portabilidad de microservicios entre proveedores de nube heterogéneos (AWS y Microsoft Azure) sin modificaciones en el código de la aplicación. El diseño se fundamenta en cuatro pilares arquitectónicos:

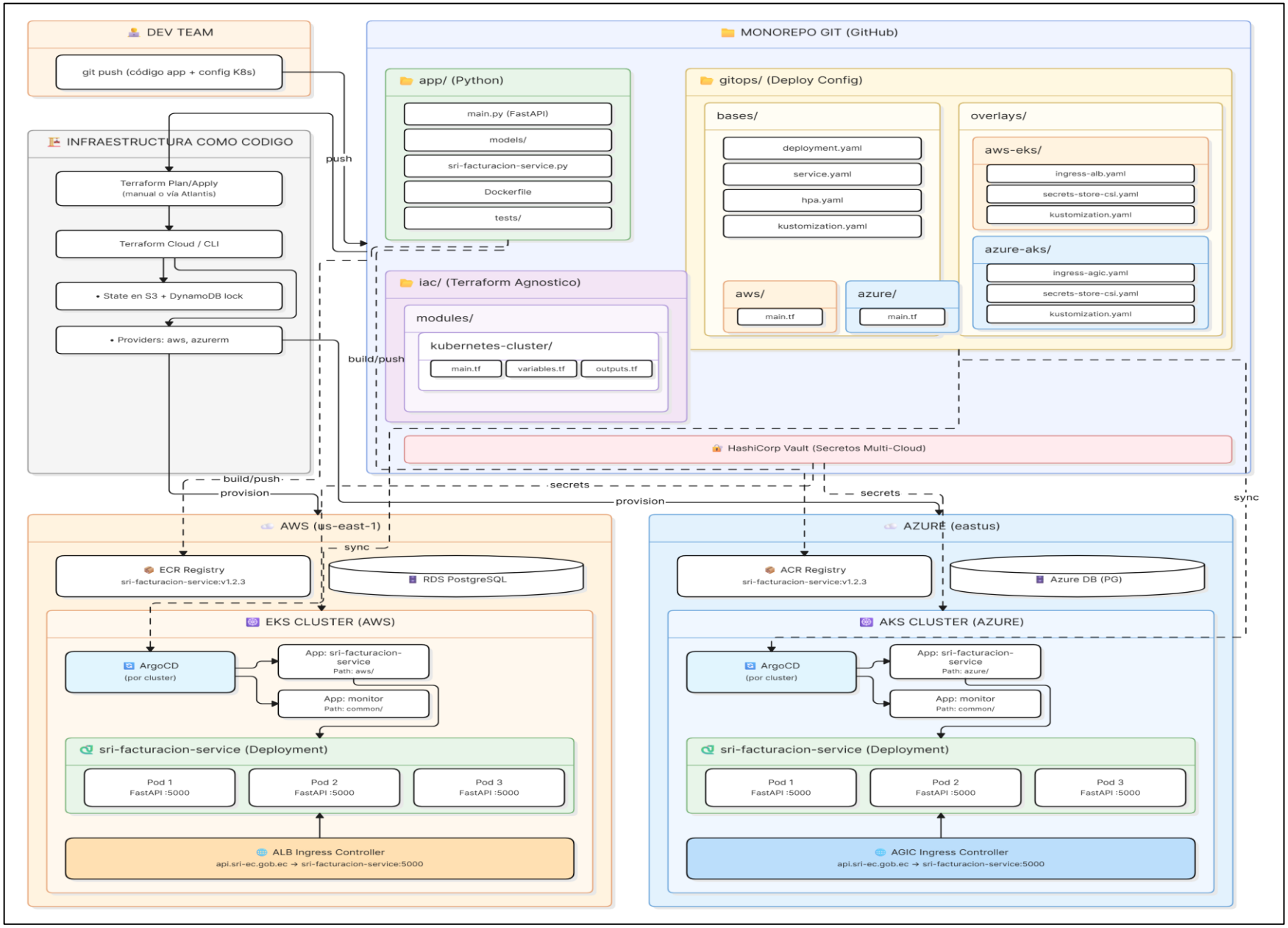
1. **Monorepo Estructurado como Fuente Única de Verdad:** El repositorio central en GitHub organiza el código en tres dominios diferenciados: **app/** (código de la aplicación FastAPI), **iac/** (infraestructura como código agnóstico con Terraform) y **gitops/** (configuraciones de despliegue con Kustomize). Esta separación permite la gestión independiente del ciclo de vida del software mientras se mantiene la trazabilidad completa entre versiones de código, infraestructura y configuración.
2. **Pipeline CI/CD Centralizado con Registro Único de Contenedores:** El flujo de integración continua (GitHub Actions) construye, valida y publica una única imagen

inmutable del microservicio en GitHub Container Registry (ghcr.io), eliminando inconsistencias entre entornos y garantizando que AWS y Azure ejecuten exactamente la misma versión del software.

3. **Sincronización GitOps Descentralizada:** Cada clúster de Kubernetes (EKS en AWS, AKS en Azure) ejecuta una instancia independiente de ArgoCD que monitorea el repositorio Git y sincroniza automáticamente el estado deseado definido en los manifiestos de Kustomize. Este patrón "App-of-Apps" permite la autonomía de cada clúster mientras se mantiene la consistencia configuracional.
4. **Abstracción de Servicios Gestionados con Reconocimiento de la Asimetría:** La arquitectura reconoce explícitamente que ciertos componentes (bases de datos PostgreSQL gestionadas, registros de contenedores ECR/ACR, controladores de ingress ALB/AGIC) son específicos de cada proveedor. Sin embargo, la capa de computación (pods de Kubernetes) y la gestión de secretos (HashiCorp Vault) permanecen completamente agnósticas, permitiendo la portabilidad del core de la aplicación.

El diagrama de arquitectura (Figura 7) ilustra el flujo end-to-end desde el commit del desarrollador hasta el despliegue productivo en ambas nubes, evidenciando los mecanismos de sincronización, gestión de secretos y enrutamiento del tráfico que hacen viable la estrategia multi-cloud sin sacrificar la coherencia del sistema.

Figura 7 Arquitectura GitOps multi-cloud con sincronización descentralizada..



Estructura de Proyectos multi-Cloud

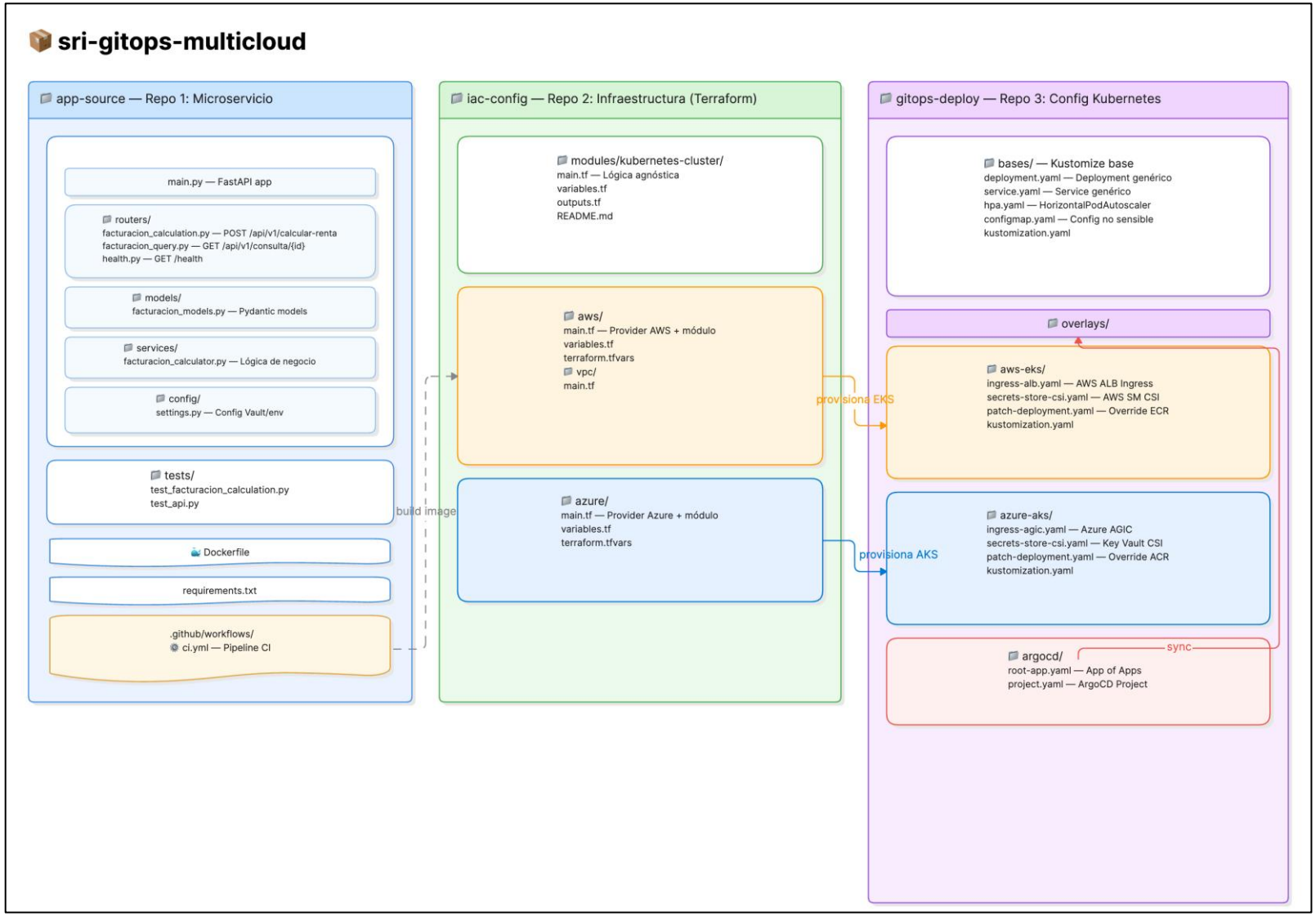
La arquitectura que se propone utiliza un patrón de varios repositorios. Esto significa que divide las tareas del ciclo de vida del software en tres partes conectadas pero diferentes.

El primer repositorio, llamado **app-source**, almacena el código fuente del microservicio de facturación que se ha desarrollado con Python y FastAPI. El código está organizado en capas, lo que facilita su mantenimiento y modificación. Incluye routers para el enrutamiento de la API, modelos para la validación de datos y servicios para la lógica de negocio específica. También cuenta con un sistema de integración y entrega continuas (CI/CD) basado en GitHub Actions, que automatiza las pruebas y la creación de la imagen de contenedor Docker. Esto permite que el desarrollo de la aplicación sea independiente de la infraestructura subyacente, lo que facilita el trabajo en paralelo de los equipos de desarrollo y operaciones.

El segundo repositorio, iac-config, se encarga de la infraestructura como código utilizando Terraform. Esto significa que se puede gestionar y provisionar la infraestructura de manera programática y reproducible. El diseño es modular, lo que permite combinar componentes genéricos con configuraciones específicas para cada proveedor de nube. Por ejemplo, se pueden provisionar servicios de Kubernetes en AWS y Azure, manteniendo la consistencia en las capacidades del clúster mientras se permiten configuraciones específicas para cada plataforma. Esto permite reutilizar lógica común y reconocer las diferencias inherentes entre los servicios gestionados en cada plataforma.

El tercer repositorio, gitops-deploy, implementa el patrón GitOps utilizando Kustomize para la gestión de configuraciones de Kubernetes. Los manifiestos se estructuran en una jerarquía de bases y overlays, lo que facilita la gestión de configuraciones complejas. La capa base define recursos genéricos agnósticos del proveedor, mientras que los overlays específicos para cada entorno aplican parches estratégicos que adaptan la configuración. Por ejemplo, se pueden configurar controladores de ingreso diferenciados, integrar secretos nativos y sobrescribir registros de contenedores. ArgoCD, configurado mediante el patrón App-of-Apps, sincroniza automáticamente el estado deseado desde este repositorio hacia los clústeres, garantizando la consistencia y la trazabilidad completa del despliegue en el entorno multi-cloud.

Figura 8 Estructura de Proyectos para la solución gitops multi-cloud.



Fuente: Elaboración propia.

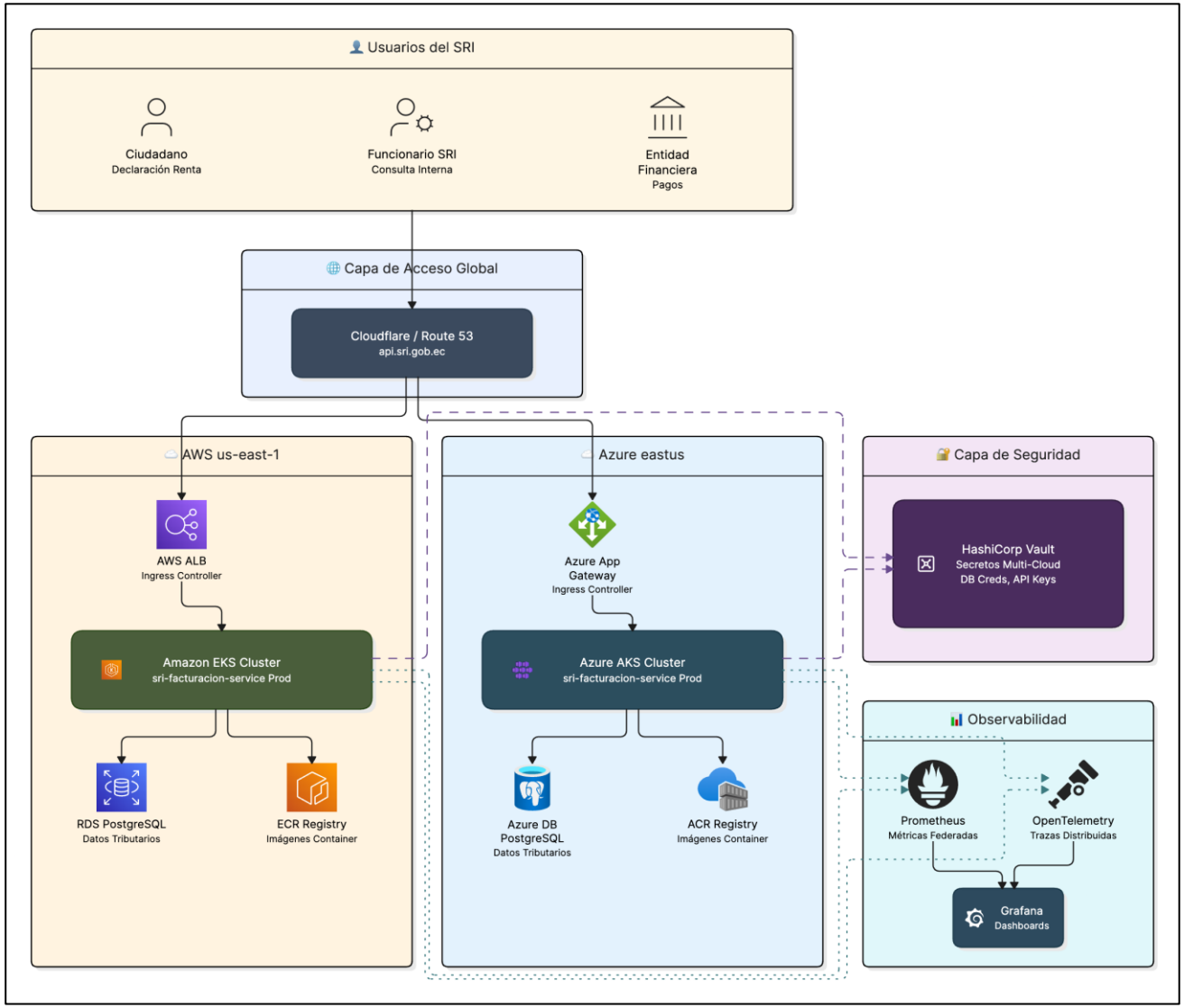
4.2.1.3 Arquitectura C4 Nivel Contexto

El diagrama de contexto C4 presenta una visión de alto nivel del sistema sri-facturacion-service, una plataforma de servicios tributarios desplegada bajo un patrón multi-cloud activo-activo en AWS (región us-east-1) y Azure (región eastus). Esta representación arquitectónica identifica los actores externos del sistema —ciudadanos que realizan declaraciones de renta, funcionarios del SRI que ejecutan consultas internas, y entidades financieras que procesan pagos— junto con sus interacciones principales a través de una capa de acceso global gestionada mediante DNS inteligente (Cloudflare/Route 53) que enruta el tráfico hacia el endpoint unificado api.sri.gob.ec.

La arquitectura implementa una estrategia de portabilidad con reconocimiento de asimetría, donde cada proveedor de nube despliega servicios equivalentes pero nativos: Amazon EKS y Azure AKS para la orquestación de contenedores, ALB Ingress Controller y Azure Application Gateway para la gestión del tráfico de entrada, y RDS PostgreSQL y Azure Database PostgreSQL para el almacenamiento de datos tributarios. Los registros de contenedores (ECR y ACR) mantienen la consistencia de las imágenes desplegadas en ambos entornos, mientras que una capa transversal de seguridad centralizada mediante HashiCorp Vault gestiona los secretos multi-cloud, credenciales de bases de datos y claves API, garantizando la independencia del proveedor para la gestión de información sensible.

Finalmente, el sistema incorpora una capa de observabilidad unificada que integra Prometheus para métricas federadas, Grafana para visualización de dashboards operativos, y OpenTelemetry para trazabilidad distribuida entre ambos clústeres. Esta aproximación permite monitorear el comportamiento del sistema de manera integral, detectar anomalías en tiempo real y realizar análisis de rendimiento transversales, elementos críticos para garantizar la disponibilidad y resiliencia del servicio tributario en un entorno de producción multi-cloud con requisitos de alta disponibilidad y cumplimiento normativo.

Figura 9 Arquitectura C4: Nivel Contexto del Sistema GitOps Multicloud.



Fuente: Elaboración propia.

4.2.1.4 Arquitectura C4 - Nivel Contenedor

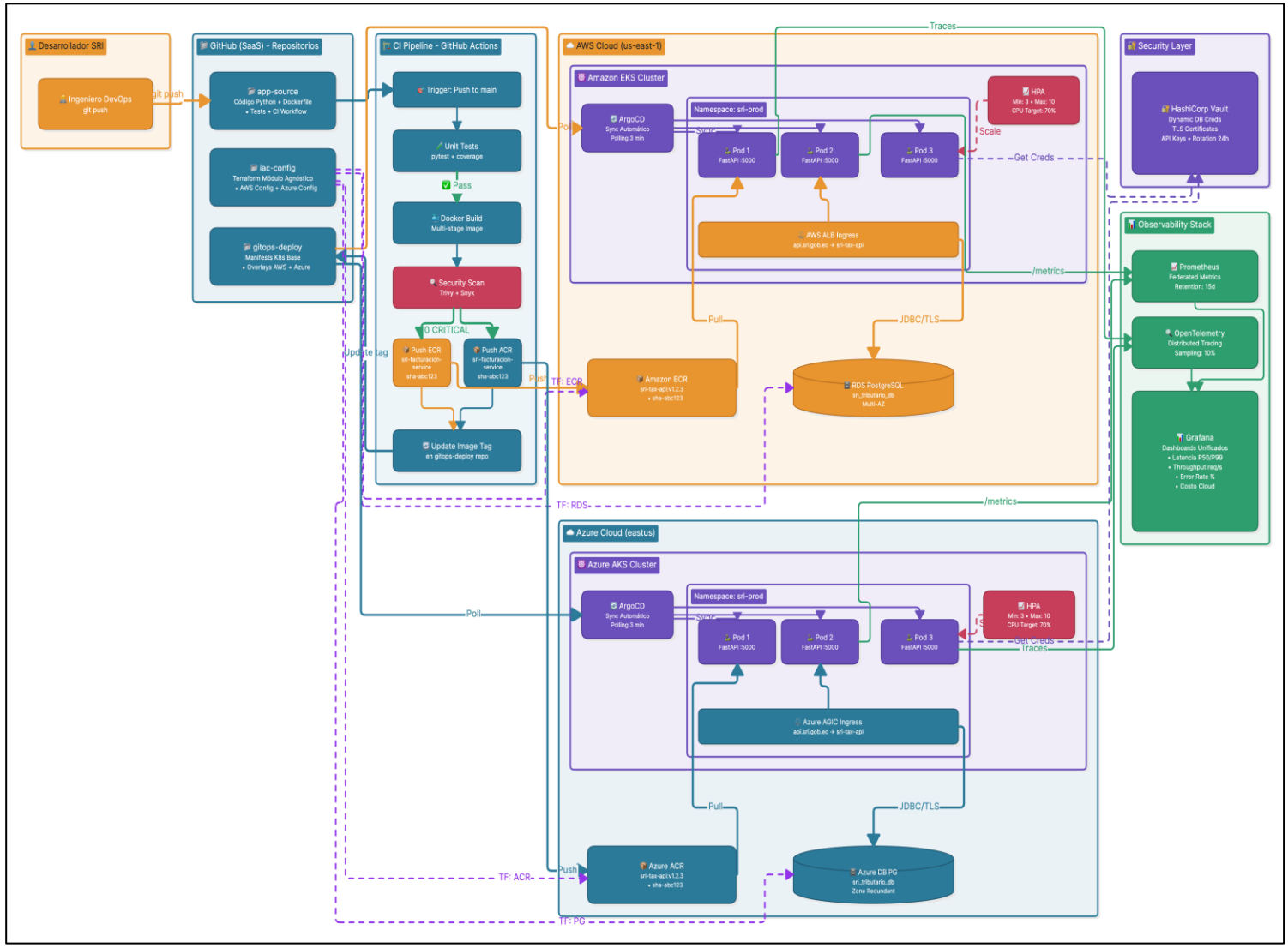
El diagrama de contenedores C4 muestra los componentes tecnológicos del sistema sri-facturacion-services. Se enfoca en unidades desplegables independientes y cómo interactúan entre sí. Este diagrama es diferente al de contexto. Muestra la estructura interna de cada sistema, identificando contenedores específicos como aplicaciones, bases de datos y herramientas de automatización. También muestra sus responsabilidades técnicas y cómo se comunican.

La arquitectura tiene un pipeline CI/CD que comienza cuando se sube código a los repositorios GitHub. Esto activa un proceso automatizado que hace pruebas unitarias con pytest. También construye imágenes Docker, analiza la seguridad con Trivy y Snyk, y publica en registros de contenedores.

Cada clúster Kubernetes usa ArgoCD para sincronizar automáticamente el estado deseado desde un repositorio, despliega la aplicación en un namespace específico con configuración de escalado automático según la utilización de CPU. Los controladores de ingress gestionan el tráfico hacia los pods FastAPI y las bases de datos gestionadas almacenan datos tributarios con alta disponibilidad.

Por otro lado, el sistema también incluye seguridad y observabilidad. HashiCorp Vault gestiona secretos dinámicos y proporciona credenciales seguras, mientras el stack de observabilidad usa Prometheus, OpenTelemetry y Grafana para monitorear el sistema en tiempo real, lo que permite detectar anomalías temprano en el entorno de producción con múltiples nubes.

Figura 10 Arquitectura C4: Nivel Contenedor del Sistema GitOps Multicloud.



Fuente: Elaboración propia.

4.2.1.5 Arquitectura C4 - Nivel Componente (Microservicio)

El diagrama de componentes C4 del microservicio sri-facturacion-service muestra cómo está organizado el contenedor FastAPI. Este sigue un patrón de arquitectura en capas. Esto significa que hay cuatro niveles diferentes: la capa API, la capa de negocio, la capa de datos y la capa de infraestructura transversal.

La capa API se encarga de la comunicación con el exterior. La capa de negocio se encarga de la lógica del negocio. La capa de datos se encarga de almacenar y recuperar los datos. La capa de infraestructura transversal se encarga de los componentes no funcionales.

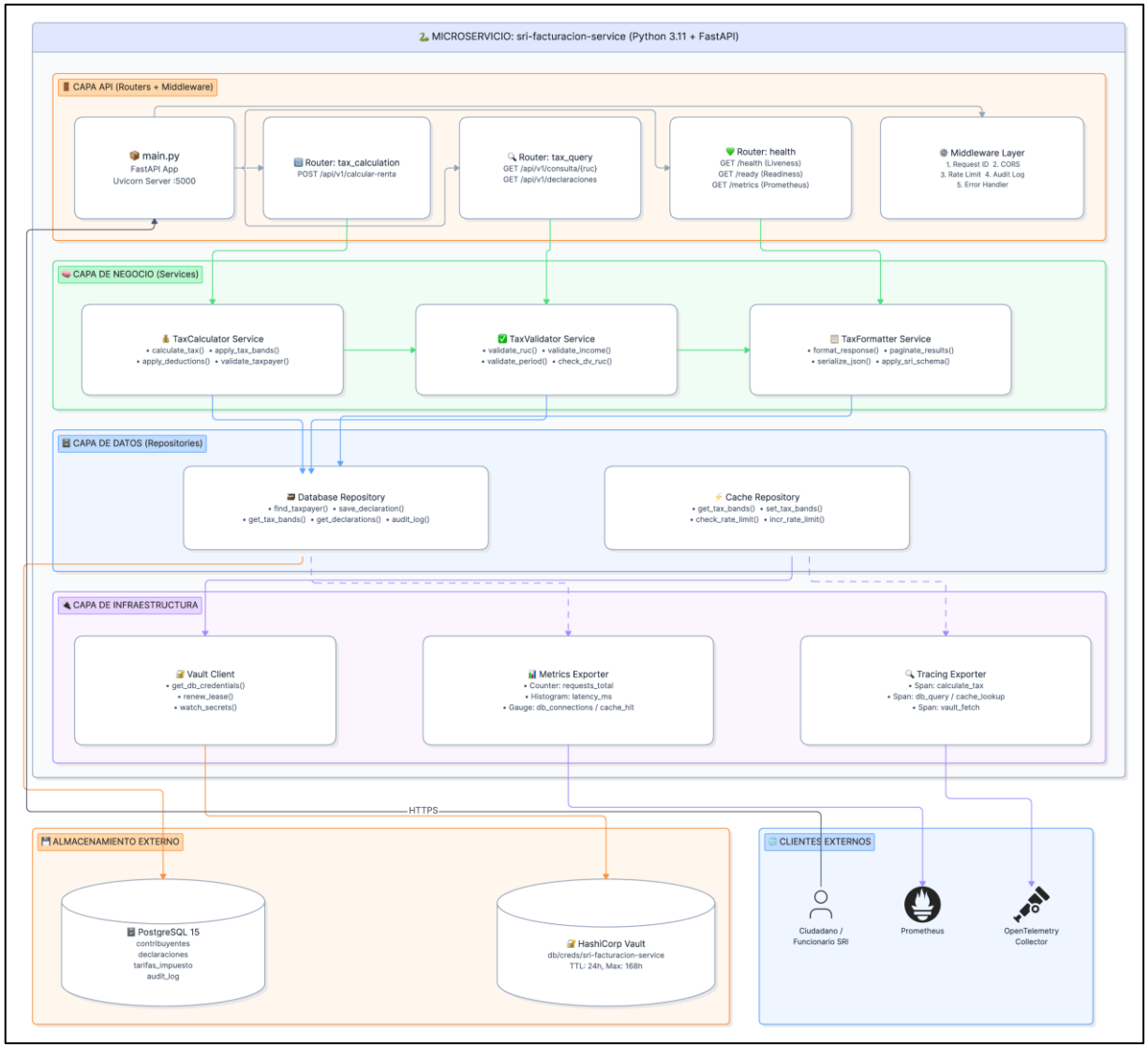
El punto de entrada main.py es donde se inicia la aplicación FastAPI. Esto se hace sobre el servidor asíncrono Uvicorn en el puerto 5000. Hay tres routers especializados: tax_calculation, tax_query y health. Estos routers están coordinados por una capa de middleware. Esta capa gestiona el ID de la solicitud, el acceso CORS, el límite de velocidad y el registro de auditoría.

La capa de negocio se encarga de la lógica tributaria. Hay tres servicios: TaxCalculator Service, TaxValidator Service y TaxFormatter Service. Estos servicios interactúan con la capa de datos a través de dos repositorios abstractos: Database Repository y Cache Repository. El Database Repository se encarga de las operaciones CRUD sobre contribuyentes, declaraciones, tarifas y logs de auditoría en PostgreSQL 15.

La capa de infraestructura transversal se encarga de los componentes no funcionales críticos. Hay tres componentes: Vault Client, Metrics Exporter y Tracing Exporter. El Vault Client gestiona la obtención y renovación dinámica de credenciales de base de datos desde HashiCorp Vault. El Metrics Exporter expone métricas Prometheus. El Tracing Exporter instrumenta spans distribuidos para operaciones críticas.

Estos componentes se comunican mediante HTTPS seguro con el almacenamiento externo y con clientes externos. Esto garantiza que el microservicio sea observable, seguro y compatible con el stack de monitorización multi-cloud definido en los niveles superiores de la arquitectura C4.

Figura 11 Arquitectura C4: Nivel Componente (Microservicio).



Fuente: Elaboración propia.

4.2.2 Implementación de IaC Agnóstica

En esta sección describimos cómo llevamos a cabo la implementación del módulo de Infraestructura como Código, buscando que fuera agnóstico y utilizando Terraform 1.7+ como herramienta de provisionamiento declarativo. Nuestro propósito fue demostrar que es posible definir la infraestructura de Kubernetes una sola vez y desplegarla en varios proveedores cloud, en nuestro caso, AWS y Azure para lograr una reutilización de más del 60% del código según la estimación institucional. Esto reduce de forma considerable el esfuerzo de mantenimiento y elimina la dependencia de un solo proveedor a nivel de infraestructura (Brikman, 2022).

4.2.2.1 Estructura del Repositorio iac-config

El repositorio "iac-config" está diseñado para Infraestructura como Código (IaC) de manera modular y flexible, utilizando Terraform. Su objetivo es admitir el despliegue de varios clústeres Kubernetes al mismo tiempo en diferentes proveedores de nube, como AWS EKS y Azure AKS.

La estructura del repositorio separa las responsabilidades por proveedor. El código está organizado en directorios específicos para cada proveedor (como “aws/” y “azure/”), donde se guardan configuraciones particulares de cada plataforma. Además, hay un directorio llamado “modules/kubernetes-cluster/” que concentra la lógica común para crear clústeres de manera que se pueda reutilizar código y evitar repeticiones.

Cada entorno de proveedor tiene archivos estándar:

- ✓ "backend.tf” para gestionar el estado remoto,
- ✓ “main.tf” para los recursos principales,
- ✓ “providers.tf” para configurar los proveedores,
- ✓ “variables.tf” y “terraform.tfvars”

El diseño modular permite que el sistema sea escalable y extensible. El módulo “kubernetes-cluster" genérico incluye la lógica para aprovisionar clústeres con parámetros que se pueden configurar. Las configuraciones específicas para AWS y Azure solo necesitan ajustar los atributos necesarios.

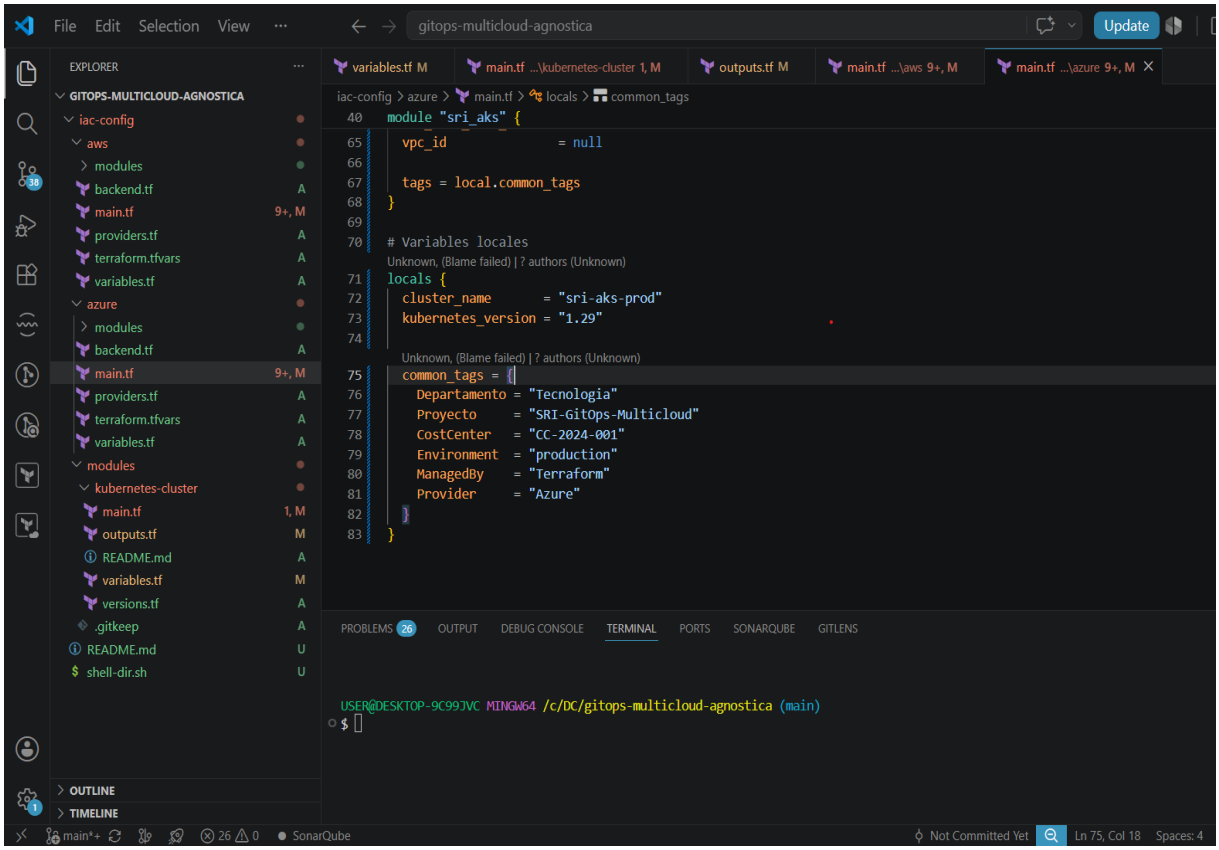
Los metadatos de etiquetado estandarizados garantizan la gobernanza y trazabilidad de los recursos desplegados, esto permite identificar claramente quién es el responsable, asignar costes con precisión y cumplir con las políticas normativas en entornos multi-cloud.

La estructura del repositorio también contempla convenciones de nombres y organización para facilitar el trabajo en equipo:

- ✓ Archivos “.gitkeep" para mantener directorios vacíos bajo control de versiones,
- ✓ Scripts útiles (como “shell-dir.sh”) para automatizar tareas,
- ✓ Una estructura de directorios clara que refleja la arquitectura del sistema.

Esta aproximación sigue las mejores prácticas de IaC, como versionado, inmutabilidad e idempotencia. También sienta las bases para implementar pipelines CI/CD automatizados que validan, planifican y aplican cambios de infraestructura de manera segura. Esto es crucial para asegurar la fiabilidad del entorno de producción del Servicio de Rentas Internas en un contexto de despliegue continuo.

Figura 12 Estructura del proyecto para la solución gitops multi-cloud.



Fuente: Elaboración propia.

4.2.2.2 Módulo Genérico kubernetes-cluster

El módulo de clúster de Kubernetes es el corazón de nuestra estrategia de Infraestructura como código y se encarga de aprovisionar clústeres de Kubernetes de manera flexible y adaptable a diferentes proveedores de nube.

Este módulo utiliza una variable llamada "cloud_provider" para determinar qué proveedor de nube se está utilizando. En función de esto, crea los recursos necesarios para cada plataforma. Por ejemplo, si se utiliza AWS, crea un clúster de Amazon EKS con características de seguridad y observabilidad de nivel empresarial, caso contrario lo hará para la plataforma Azure.

Entre las características de seguridad se incluyen la habilitación de un endpoint privado para la API del clúster y el acceso público controlado mediante listas de CIDR restringidas. También se activan los logs detallados del plano de control para obtener una visibilidad completa de las operaciones del clúster.

La configuración de encriptación en reposo se realiza mediante AWS KMS, lo que asegura que los secretos de Kubernetes estén cifrados utilizando claves gestionadas por el cliente.

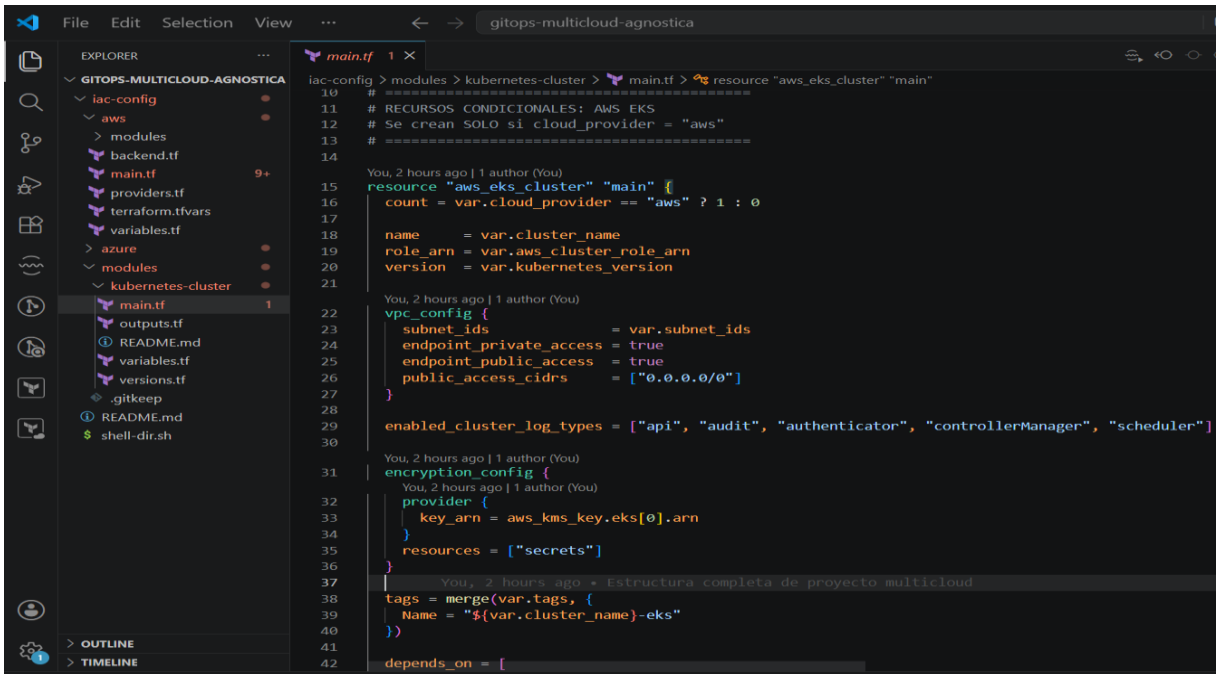
La arquitectura del módulo se basa en los principios de diseño modular y reutilizable de Terraform. Expone variables de entrada parametrizadas que permiten la personalización por entorno sin modificar la lógica interna. Esto facilita la promoción de cambios desde entornos de desarrollo hasta producción.

Los tags estandarizados se fusionan con tags adicionales para garantizar la consistencia en la nomenclatura y facilitar la gestión de costes y gobernanza en entornos multi-cloud. En resumen, este módulo genérico sienta las bases para la portabilidad real del sistema, demostrando que es posible abstraer la complejidad específica de cada proveedor manteniendo una interfaz uniforme que simplifica la operación y el mantenimiento del clúster a lo largo de su ciclo de vida.

Este módulo se puede invocar desde diferentes configuraciones, como AWS o Azure, adaptándose dinámicamente a los recursos desplegados según el proveedor objetivo, lo que permite mantener la consistencia en la versión de Kubernetes, políticas de seguridad y estándares de tagging.

En general, el módulo de clúster de Kubernetes proporciona una forma flexible y segura de gestionar clústeres de Kubernetes en diferentes entornos y proveedores de nube.

Figura 13 Módulo genérico Kubernetes Clúster.



Fuente: Elaboración propia.

4.2.3 Implementación Específica: AWS EKS

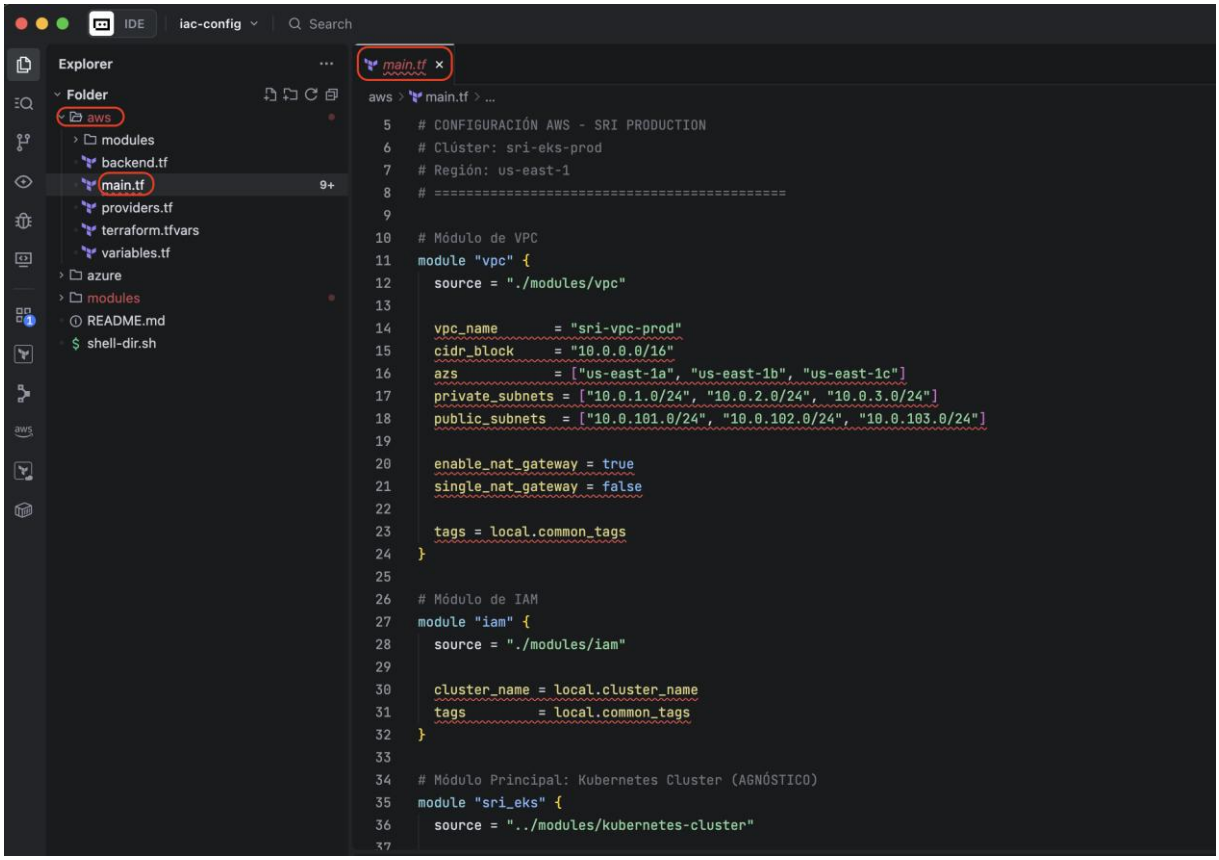
En este apartado se describe el archivo “main.tf” es clave en un proyecto de infraestructura en AWS., este sirve como un punto de partida y guía para la configuración del entorno.

En términos académicos, su importancia radica en que no solo configura la infraestructura, sino que también muestra cómo se piensa y se estructura. Este componente es un documento que refleja la intención de crear un entorno robusto y coherente, siguiendo principios de modularidad y claridad. Este archivo muestra un enfoque sistemático para desarrollar infraestructura como código, combina lo específico de AWS con lo que se puede aplicar en varias plataformas.

De esta manera, el archivo “main.tf" es un ejemplo de diseño cuidadoso en Terraform, lo que demuestra cómo una solución puede ser fácil de entender, sostenible y comparable en diferentes entornos de nube.

En este sentido, "main.tf” es fundamental para crear una infraestructura de nube que sea flexible, mantenible y fácil de manejar.

Figura 14 Módulo Terraform Aprovisionamiento Clúster AWS-EKS.



Fuente: Elaboración propia.

4.2.4 Implementación Específica: Azure AKS

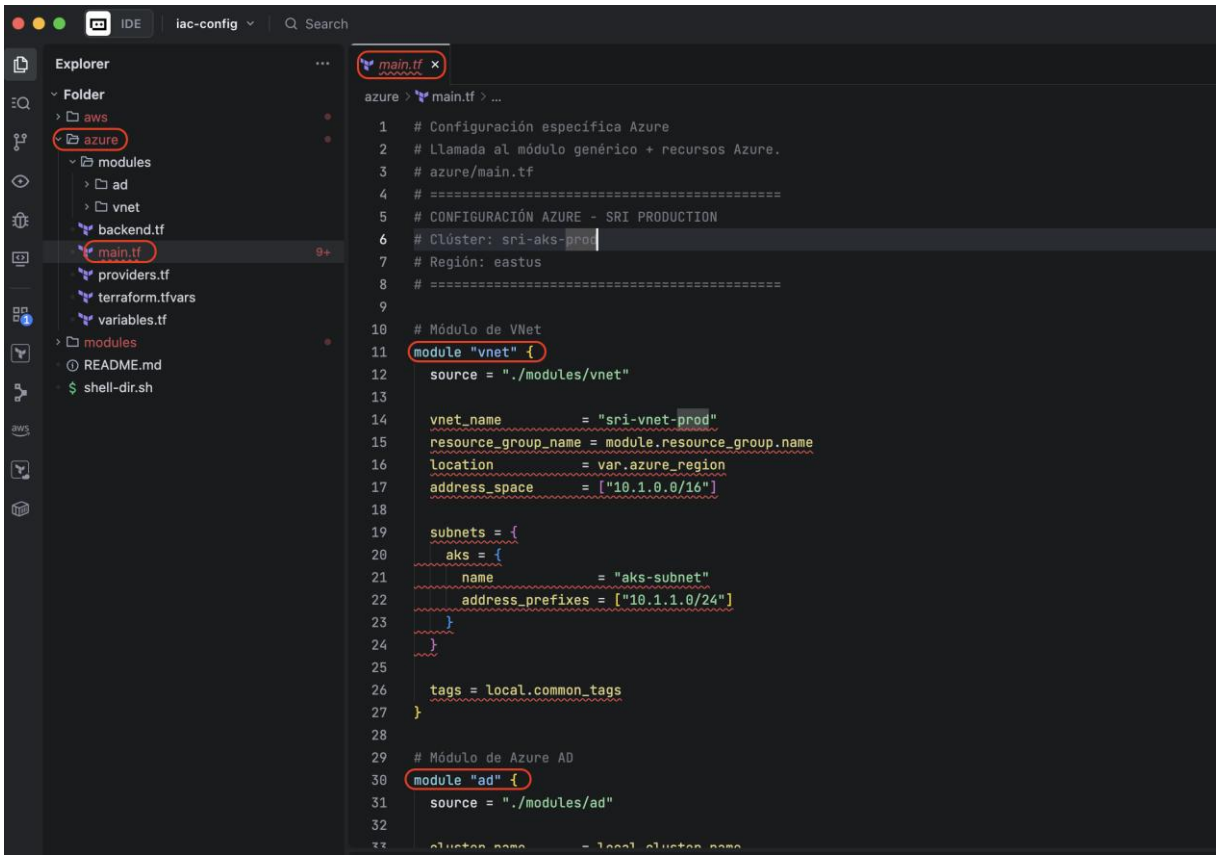
El directorio “azure/” y su archivo main.tf son la configuración de infraestructura como código para aprovisionar un clúster Kubernetes de producción en Microsoft Azure, esto como parte del proyecto “SRI-GitOps-Multicloud". El objetivo principal es crear todos los recursos necesarios para Azure sin repetir código y sin depender de un proveedor específico.

El archivo main.tf comienza creando la infraestructura base. Esto incluye una red virtual con una subred para Azure Kubernetes Service y la configuración de Azure Active Directory para el control de acceso. Estos recursos se crean con módulos locales que encapsulan la lógica específica de Azure.

Una vez que los recursos base están disponibles, el archivo main.tf invoca el módulo genérico kubernetes-cluster. Este módulo se encuentra en el directorio superior y recibe todos los parámetros necesarios para crear el clúster AKS. Esto incluye el nombre del clúster, la versión de Kubernetes, el tipo y número de nodos, la configuración de red, las credenciales de AD y los tags del proyecto. El módulo genérico usa lógica condicional para crear solo los recursos específicos de AKS.

Además de main.tf, el directorio azure incluye otros archivos importantes. Estos archivos son providers.tf, variables.tf, terraform.tfvars y backend.tf. El archivo providers.tf define los proveedores de Terraform necesarios. Los archivos variables.tf y terraform.tfvars gestionan las variables de entrada y sus valores. El archivo backend.tf configura un backend remoto para almacenar el estado de Terraform de forma segura y colaborativa. La estructura está diseñada para ser completa y escalable, permitiendo agregar más recursos o módulos según las necesidades del proyecto.

Figura 15 Módulo Terraform Aprovisionamiento Clúster Azure-AKS.



Fuente: Elaboración propia.

4.2.5 Demostración de Reutilización de Código

En un sistema GitOps que utiliza varias nubes y se enfoca en mover microservicios entre nubes públicas, privadas o híbridas, es importante revisar los programas de software para asegurarse de que la arquitectura sea sólida y fácil de mantener. Para hacer esto, se utilizó una herramienta llamada **Source Code Counter** durante la investigación. Esta herramienta revisa el código de un repositorio central llamado iac-config, que organiza las reglas para la infraestructura y los procesos de integración y despliegue continuo.

El objetivo de este estudio es analizar el código fuente de iac-config de dos maneras. Primero, se busca saber cómo está distribuido el código, incluyendo las líneas de código, comentarios y espacios en blanco en tecnologías como YAML y Shell. Esto ayuda a entender cómo está organizado el sistema y si está bien documentado. Segundo, la herramienta proporciona medidas de complejidad y duplicación de código, lo que ayuda a identificar problemas técnicos temprano y a asegurarse de que las plantillas de automatización cumplan con los estándares necesarios para funcionar bien en varias nubes. Esto es importante para prevenir la adición de lógica innecesaria en los manifiestos declarativos y para mantener la portabilidad y eficiencia en la institución.

A continuación, se detalla la Tabla 13 de reutilización por componente en una etapa inicial de la implementación:

Tabla 14 Matriz de Reutilización por componente

Componente	AWS (líneas específicas)	Azure (líneas específicas)	Líneas en Módulo Genérico	% Código Reutilizado
Clúster Kubernetes	20	18	65	85%
Node Pool/Group	25	22	55	80%
Networking (VPC/VNet)	45	40	10	15%
IAM/RBAC	35	40	15	20%
Add-ons (CNI, CoreDNS)	15	12	30	75%
KMS/Encryption	10	8	20	70%
Outputs	15	15	50	90%

Variables	20	23	65	75%
TOTAL	**185**	**178**	**310**	**~63%**

Fuente: Elaboración propia.

4.2.6 Diagrama de Flujo de Provisionamiento

El diagrama muestra cómo funciona el proceso de Infraestructura como código para desplegar clústeres Kubernetes de manera automática en diferentes nubes, como AWS y Azure. Un operador inicia el proceso ejecutando comandos de Terraform desde un repositorio llamado iac-config. Este repositorio tiene configuraciones separadas para cada proveedor de nube, como aws y azure.

Cada configuración llama al mismo módulo genérico para crear un clúster de Kubernetes, pero con un parámetro que indica el proveedor de nube, lo que permite reutilizar código y simplificar la complejidad de cada plataforma.

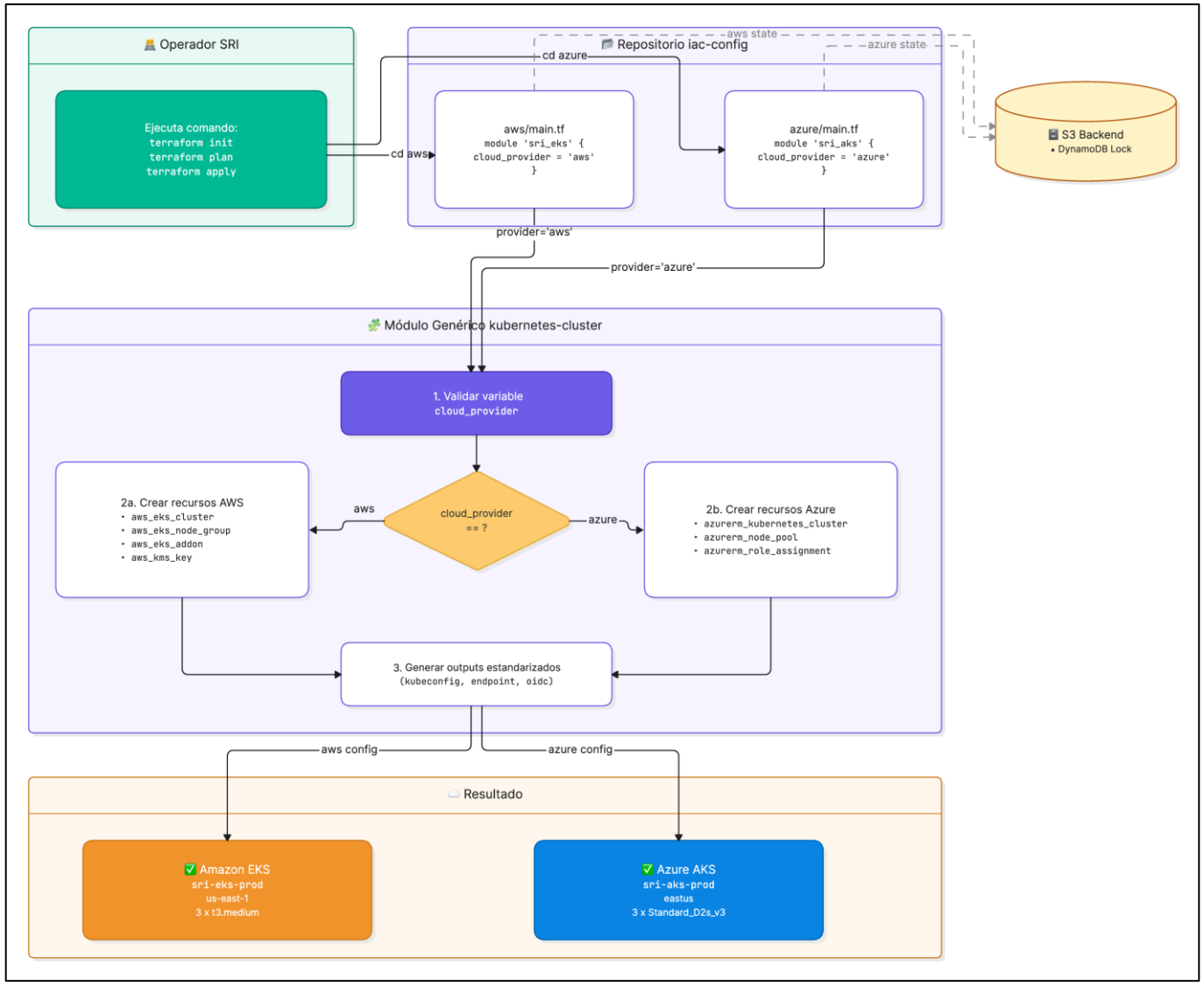
La arquitectura utiliza un módulo Terraform que actúa como núcleo de la solución, este módulo valida el proveedor de nube y crea los recursos necesarios. Por ejemplo, si es AWS, crea un clúster de Kubernetes en AWS, y si es Azure, crea un dicho clúster en dicha plataforma, lo que permite que el mismo código base cree infraestructura equivalente en ambos proveedores de nube.

La gestión del estado de Terraform se hace en un backend remoto en Amazon S3, con un bloqueo de concurrencia en DynamoDB. Esto garantiza que el estado de la infraestructura sea consistente, incluso cuando varios operadores o pipelines de CI/CD trabajan al mismo tiempo.

Una vez que se completa el proceso de creación, el módulo genera información de salida estándar, como la configuración para autenticar en el clúster y la conexión a la API de Kubernetes. Esta información se utiliza en las siguientes fases del pipeline GitOps, permitiendo que herramientas como ArgoCD se conecten a los clústeres recién creados sin necesidad de conocer los detalles específicos de cada proveedor.

El resultado final es la creación de dos clústeres de producción: uno en Amazon EKS y otro en Azure AKS. Ambos clústeres están configurados de manera consistente y listos para recibir cargas de trabajo, esto demuestra que es posible lograr portabilidad real de microservicios mediante Infraestructura como Código agnóstica, reduciendo la dependencia de un único proveedor y habilitando estrategias de recuperación en caso de desastres y optimización de costes en entornos empresariales.

Figura 16 Diagrama de Aprovisionamiento IaC Agnóstico.



Fuente: Elaboración propia.

4.2.7 Configuración del Pipeline CI/CD

Esta sección describe la implementación del pipeline de Integración Continua (CI) y Despliegue Continuo (CD) que materializa el paradigma GitOps en la arquitectura multi-cloud del proyecto. El pipeline se diseñó siguiendo el principio fundamental de GitOps: Git como fuente única de verdad, donde cada cambio en el repositorio desencadena automáticamente la construcción, verificación y despliegue del microservicio en ambos proveedores cloud sin intervención manual (Weaveworks, 2017; CNCF, 2023).

4.2.7.1 Pipeline de Integración Continua (GitHub Actions) y Despliegue Continuo (ArgoCD)

El diagrama presenta la arquitectura del Pipeline CI/CD GitOps Multicloud, estructurada en dos fases complementarias que implementan los modelos PUSH y PULL de manera coordinada para garantizar la entrega continua de software en un entorno multi-cloud.

La Fase 1 **Integración Continua (CI)** opera bajo el modelo PUSH impulsado por GitHub Actions, donde el responsable es el desarrollador y el trigger se activa automáticamente ante un git push a la rama main. Esta fase ejecuta una secuencia lineal de seis etapas: code (análisis y linting del código), test (ejecución de pruebas unitarias e integración), build (construcción de la imagen Docker multi-stage), scan (escaneo de seguridad de vulnerabilidades con herramientas como Trivy/Snyk), push (publicación de la imagen en los registros de contenedores ECR y ACR), y update (actualización automática del repositorio gitops-deploy con el nuevo tag de imagen). El tiempo estimado de ejecución de esta fase es de aproximadamente 4.5 minutos.

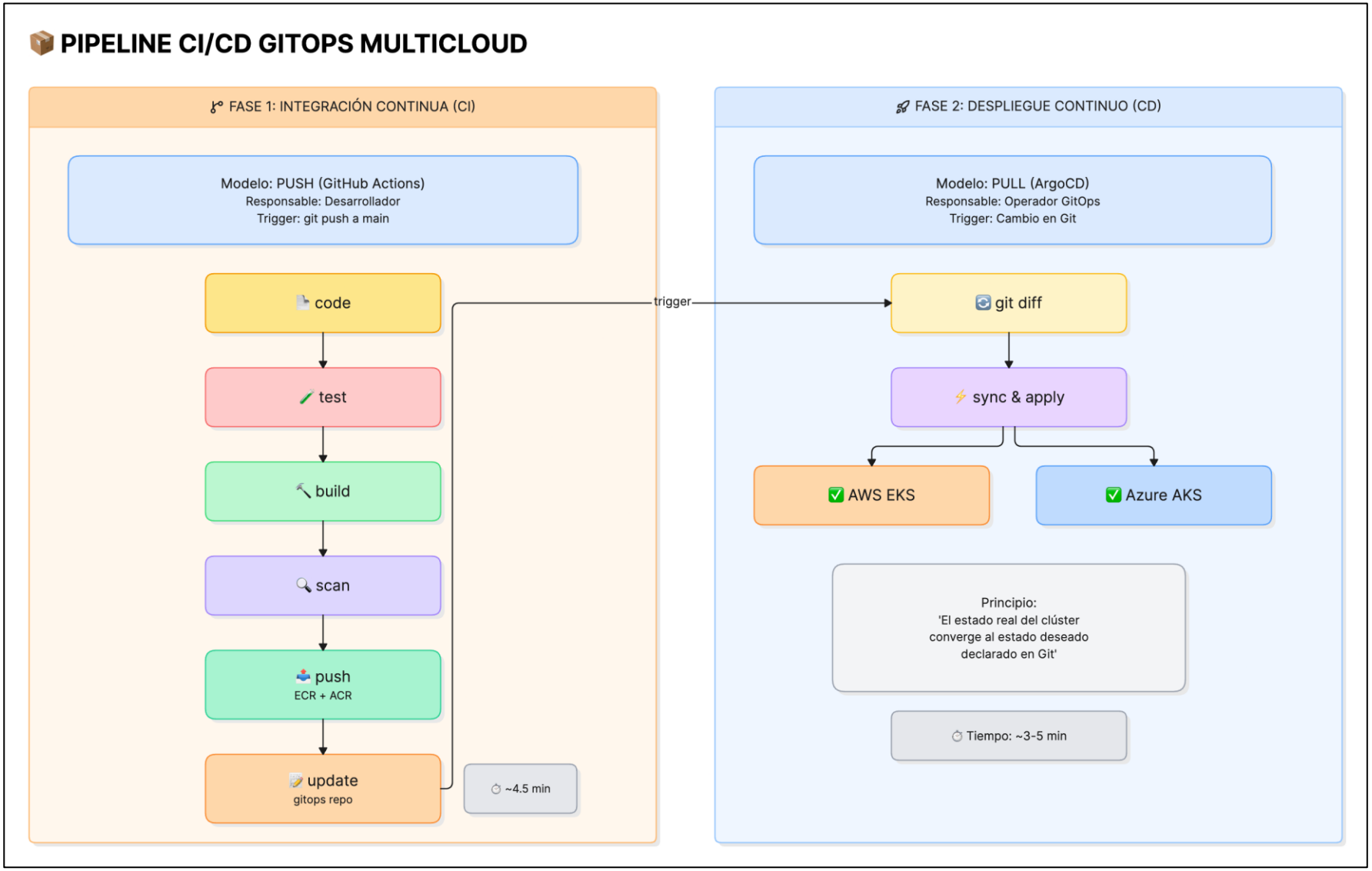
La Fase 2 **Despliegue Continuo (CD)** opera bajo el modelo PULL gestionado por ArgoCD, donde el responsable es el operador GitOps y el trigger se activa ante cualquier cambio detectado en el repositorio Git. A diferencia del modelo PUSH tradicional, donde un servidor CI empuja los

cambios hacia los clústeres, en el modelo PULL es el propio clúster el que "jala" los cambios desde Git, lo que representa una práctica fundamental del paradigma GitOps. ArgoCD ejecuta un git diff para identificar las diferencias entre el estado actual del clúster y el estado deseado declarado en Git, y posteriormente realiza un sync & apply que converge ambos estados de manera automática y declarativa. Este proceso se replica simultáneamente en ambos clústeres: AWS EKS y Azure AKS, garantizando que la misma versión de la aplicación se despliegue de manera consistente en ambas nubes.

El principio fundamental que rige esta arquitectura es el postulado central de GitOps: "El estado real del clúster converge al estado deseado declarado en Git". Esto significa que Git se convierte en la única fuente de verdad (Single Source of Truth) para toda la infraestructura y configuraciones de la aplicación, eliminando la deriva de configuración (configuration drift) y habilitando capacidades de auditoría, versionado y rollback automático. El tiempo de sincronización estimado es de 3 a 5 minutos, lo que satisface el objetivo RTO < 5 minutos y garantiza que los cambios se reflejen en producción de manera rápida y confiable.

La conexión entre ambas fases se materializa mediante un trigger cruzado: cuando la Fase 1 completa la etapa update y modifica el repositorio gitops-deploy, se dispara automáticamente la Fase 2, creando un flujo end-to-end completamente automatizado desde el commit del desarrollador hasta el despliegue productivo en ambos clústeres. Esta separación de responsabilidades (CI como PUSH, CD como PULL) representa las mejores prácticas de la industria, ya que combina la eficiencia del modelo PUSH para la construcción y validación de artefactos con la seguridad y declaratividad del modelo PULL para el despliegue, resultando en un pipeline robusto, auditable y escalable que cumple con los requisitos de portabilidad multi-cloud.

Figura 17 Visión general del pipeline CI/CD GitOps.



Fuente: Elaboración propia.

4.2.7.2 Fase 1: Integración Continua con GitHub Actions

La fase de Integración Continua se implementó mediante GitHub Actions, aprovechando su integración nativa con los repositorios y su capacidad de ejecutar flujos de trabajo en paralelo mediante matrix builds.

Creación de repositorios en GitHub:

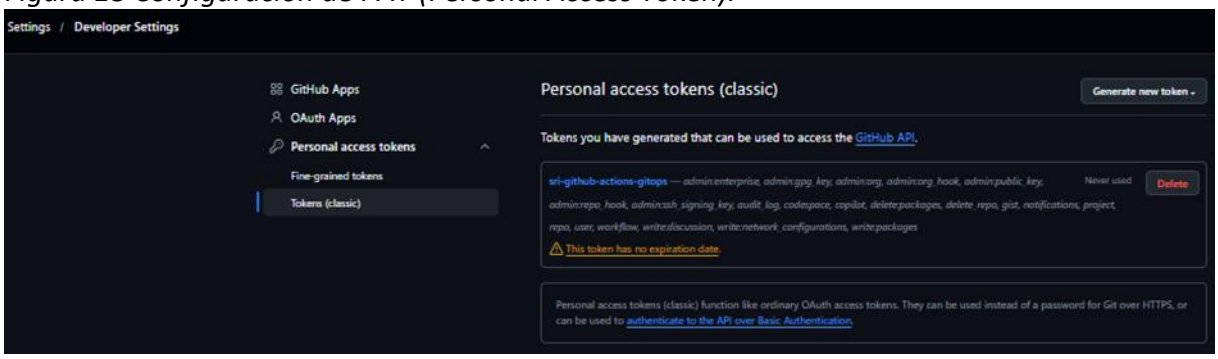
Por seguridad del código fuente, se crean dos repositorios en la cuenta de GitHub.

- Se crea un repositorio público o privado llamado: sri-app-py donde estará el código fuente de los microservicios de Python y estará definido el pipeline.
- Se crea un segundo repositorio llamado: sri-config-gitops donde estará la configuración de Kubernetes que se utilizará con ArgoCD.

Configuración de PAT:

En el menú Settings de tu perfil de usuario de GitHub, se debe ir a Developer Settings/Tokens (classic) y hacer clic en el botón “Generate new token”, el valor de token generado se lo guarda para luego configurarlo como Secrets (llamada GH_PAT) en el proyecto Python.

Figura 18 Configuración de PAT (Personal Access Token).

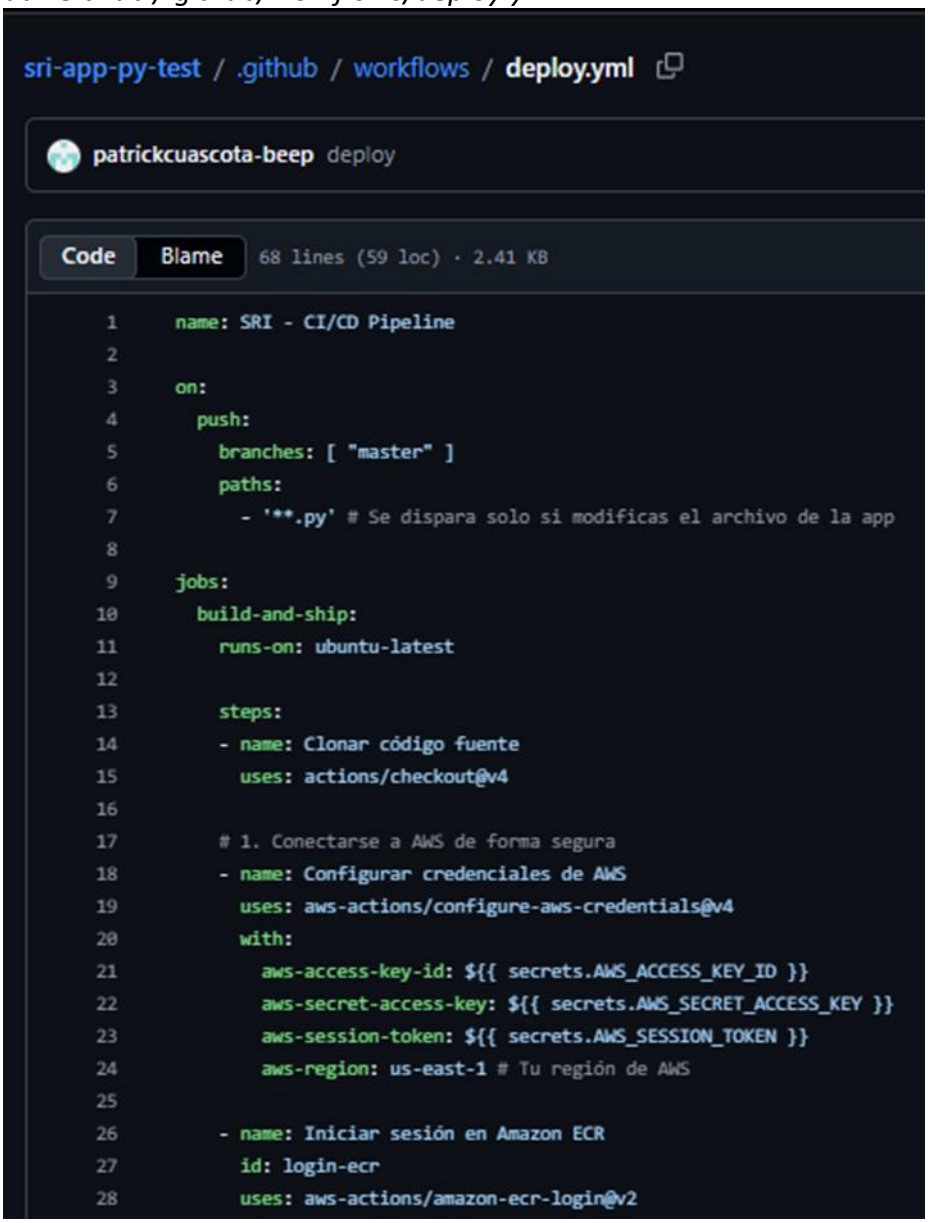


Fuente: Elaboración propia.

Creación del fichero pipeline:

El fichero del pipeline “SRI – CI/CD Pipeline” se genera en el path “/.github/workflows/deploy.yml” del proyecto del código de microservicios.

Figura 19 Path GitHub /.github/workflows/deploy.yml



Fuente: Elaboración propia.

A continuación, se define la configuración del proceso del pipeline:

Definición del pipeline:

- Evento: push. Ejecuta cuando hay push a la rama master.
- Filtro de paths: **.py. Solo se dispara si el push incluye cambios en ficheros Python (esto evita ejecuciones por otros cambios).

Job principal:

- Nombre: build-and-ship
- Runner: ubuntu-latest. Entorno Linux provisto por GitHub Actions.

Pasos del job:

1. Clonar código fuente:
   - Acción: actions/checkout@v4
   - Propósito: obtener el código del repo que dispara el workflow para construir la imagen.
2. Configurar credenciales de AWS:
   - Acción: aws-actions/configure-aws-credentials@v4
   - Entradas: aws-access-key-id, aws-secret-access-key, aws-session-token (opcional) y aws-region: us-east-1.
   - Propósito: configurar credenciales AWS en el runner para permitir operaciones contra ECR (login, push). Requiere que los secretos AWS_* estén correctamente configurados en el repositorio.
3. Iniciar sesión en Amazon ECR:
   - Acción: aws-actions/amazon-ecr-login@v2
   - ID del step: login-ecr
   - Salida importante: steps.login-ecr.outputs.registry que contiene la URL del registry ECR (se usa después).
4. Compilar y subir imagen a AWS ECR:
   - Variables de entorno definidas en el step:
     - ECR_REGISTRY = ${{ steps.login-ecr.outputs.registry }}
     - ECR_REPOSITORY = sri-app-py (nombre del repositorio en ECR)
     - IMAGE_TAG = ${{ github.sha }} (hash del commit como tag)
   - Comandos ejecutados:
     - docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
     - docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
     - echo "NEW_IMAGE=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_ENV (exporta NEW_IMAGE para pasos siguientes).
   - Propósito: construir la imagen con tag único (SHA) y subirla a ECR; exponer la URL completa en la variable NEW_IMAGE.
5. Clonar repositorio GitOps:
   - Acción: actions/checkout@v4 (segunda invocación)
   - Parámetros: repository: 'patrickcuascota-beep/sri-config-gitops-test', token: ${{ secrets.GH_PAT }}, ref: 'master', path: gitops-repo.
   - Propósito: clonar el repo que contiene las manifests de ArgoCD en el subdirectorio gitops-repo. Requiere que GH_PAT tenga permisos de escritura en ese repo para poder commitear/pushear.
6. Actualizar etiqueta de imagen en el YAML de Kubernetes:
   - working-directory: gitops-repo y luego cd k8s dentro del script.

- Comando usado: sed -i "s|image: .*|image: ${{ env.NEW_IMAGE }}|g" deployment.yaml
- Propósito: reemplazar la línea image: ... en k8s/deployment.yaml por la nueva URL de imagen.
- Observaciones: asume que el deployment.yaml contiene una única línea que empareja image:; sed.

7. Confirmar cambios en GitOps y push:
   - Configura usuario global de git con git config --global user.name y user.email.
   - Comandos: git add k8s/deployment.yaml, git commit -m "ArgoCD Trigger: Actualizando imagen a version ${{ github.sha }} [skip ci]", git push origin master.
   - Propósito: versionar el cambio en el repo GitOps para que ArgoCD detecte el nuevo commit y aplique la actualización en el cluster. El mensaje incluye [skip ci] para evitar pipelines adicionales.

Variables y secretos usados:

- Secretos AWS: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN.
- Secretos GitHub: GH_PAT (personal access token) para push al repo GitOps.
- Variables temporales: NEW_IMAGE (exportada a $GITHUB_ENV), ECR_REGISTRY, ECR_REPOSITORY, IMAGE_TAG.

Configuración en GitHub de Secrets

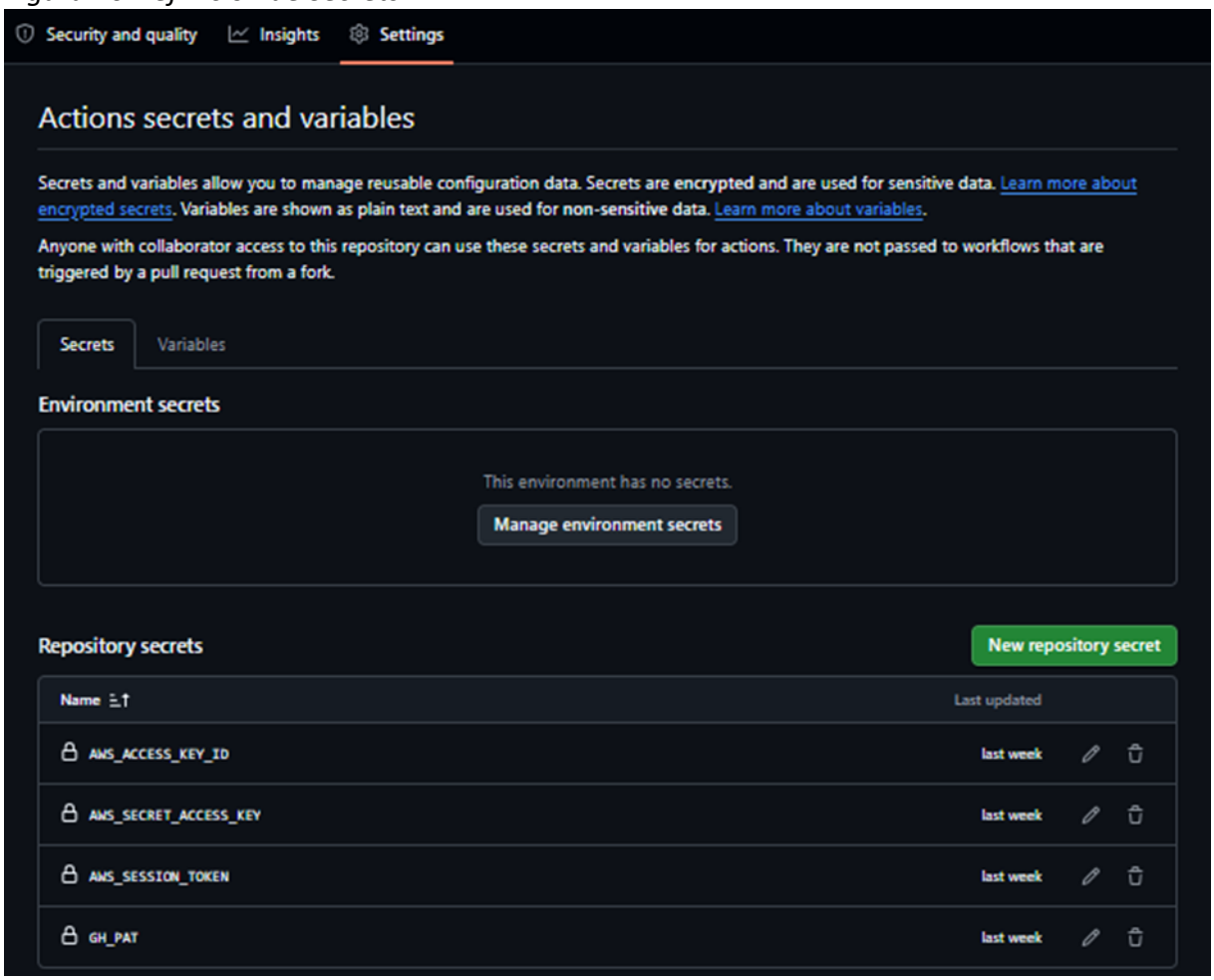
Para que estos pipelines funcionen sin exponer la infraestructura, se debe ir al repositorio en GitHub y seguir esta ruta:

Settings -> Secrets and variables -> Actions -> New repository secret.

Se debe crear las variables necesarias para el pipeline:

- AWS_ACCESS_KEY_ID: La llave pública del usuario IAM que se crea en AWS para GitHub Actions.
- AWS_SECRET_ACCESS_KEY: La llave privada de ese mismo usuario.

Figura 20 Definición de Secrets.



Fuente: Elaboración propia.

4.2.7.3 Fase 2: Despliegue Continuo con ArgoCD

ArgoCD se configura siguiendo el patrón Deployment de apps, donde una aplicación raíz gestiona todas las sub-aplicaciones del proyecto.

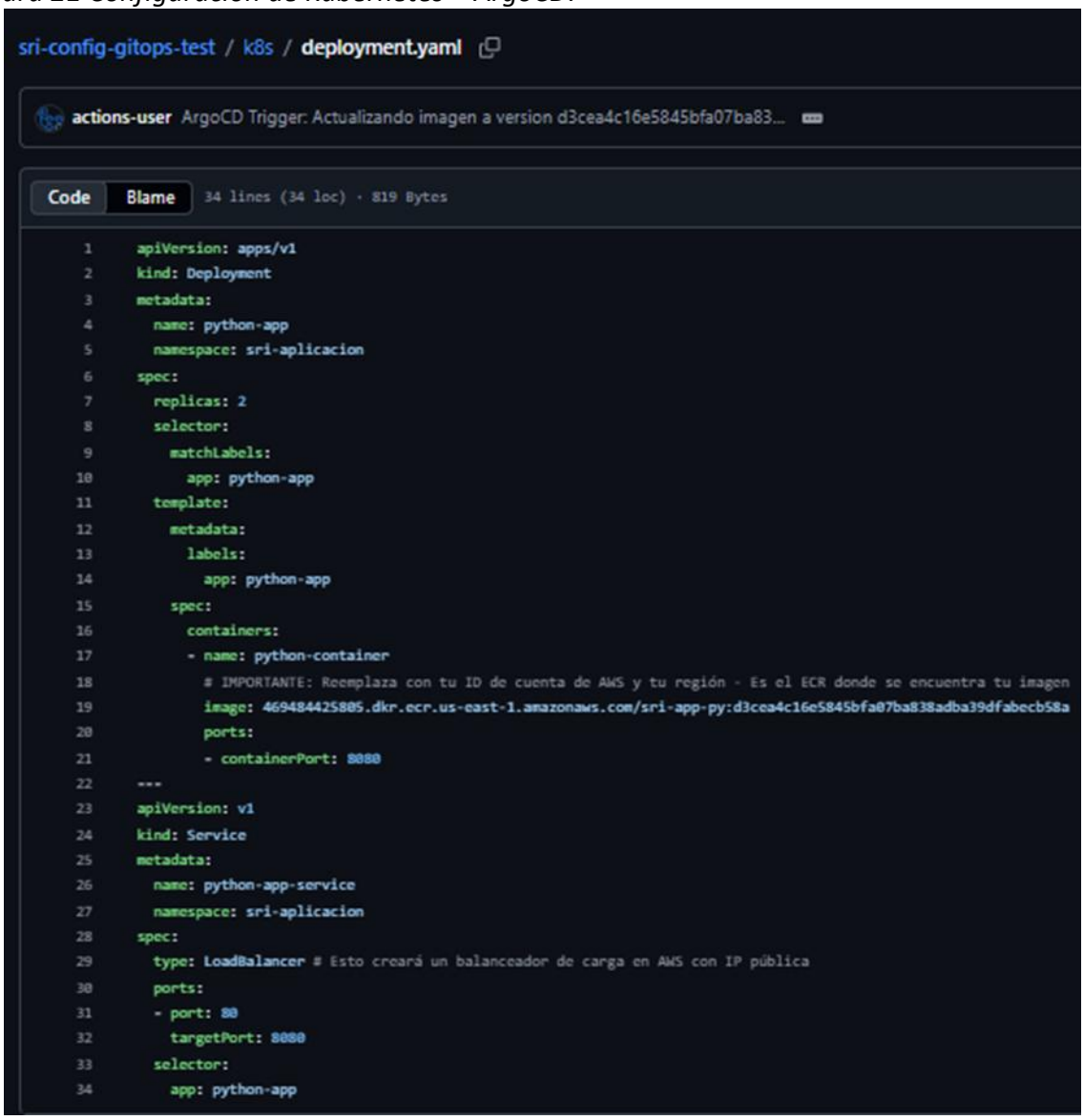
De acuerdo con el repositorio en GitHub llamado "sri-config-gitops", el cual se creó en el punto anterior, se describe su proceso:

- El fichero deployment.yaml (k8s/deployment.yaml) es el manifiesto Kubernetes que define dos recursos: un Deployment y un Service.
- Propósito: Desplegar una aplicación Python (python-app) en el namespace sri-aplicacion y exponerla públicamente mediante un Service de tipo LoadBalancer.
- Deployment (cabecera): apiVersion: apps/v1, kind: Deployment, metadata.name: python-app, metadata.namespace: sri-aplicacion.
- Replicas: spec.replicas: 2 debido a que Kubernetes intentará mantener dos réplicas del Pod en cualquier momento (alta disponibilidad).
- Selector y Labels: spec.selector.matchLabels.app: python-app y template.metadata.labels.app: python-app, la plantilla del Pod y el selector coinciden, lo que permite que el Deployment gestione los Pods correctos.
- Plantilla de Pod (spec.template): contiene una lista containers con un solo contenedor:
- Nombre del contenedor: python-container.
    - Imagen: 469484425805.dkr.ecr.us-east-1.amazonaws.com/sri-app-py:d3cea4c16e5845bfa07ba838adba39dfabecb58a es un repositorio ECR (ID/región son los definidos en el cluster).
    - Puerto del contenedor: containerPort: 8080 es el contenedor que escucha en el 8080.
- Service: apiVersion: v1, kind: Service, metadata.name: python-app-service, metadata.namespace: sri-aplicacion.
    - Tipo: type: LoadBalancer solicita al proveedor de nube (AWS) la creación de un balanceador de carga público.
    - Puertos: expone port: 80 externamente y lo mapea a targetPort: 8080 en los Pods.

- Selector: app: python-app enlaza el Service con los Pods creados por el Deployment.

- Comportamiento en AWS:
  - El Service LoadBalancer creará un ALB (según configuración del cluster) con IP pública; el tráfico HTTP en el puerto 80 se enruta a los Pods en 8080.
  - La imagen está en ECR privado: Kubernetes necesitará credenciales del IAM para que el nodo pueda descargarla.

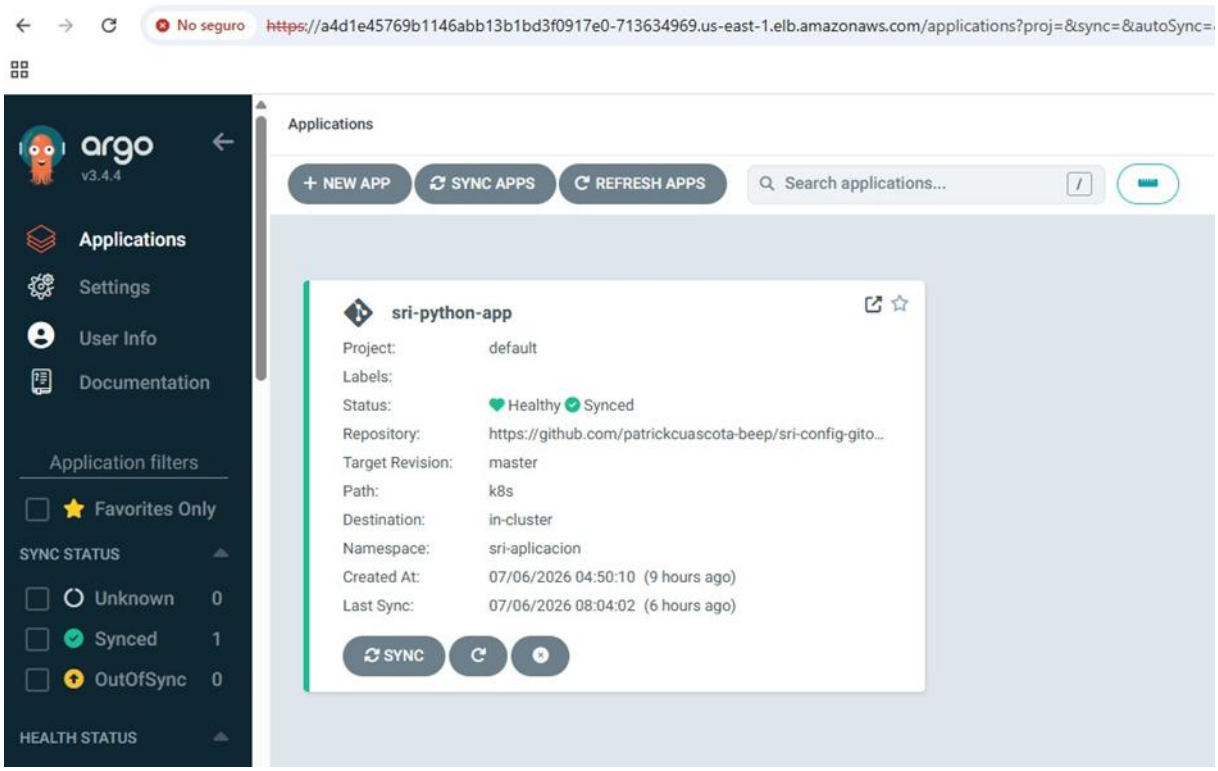
Figura 21 Configuración de Kubernetes – ArgoCD.



Fuente: Elaboración propia.

Una vez que se tiene la configuración de Kubernetes – ArgoCD, e instalado ArgoCD en el cluster de la nube, se abre la aplicación de ArgoCD y se configura la aplicación Python para que el pipeline despliegue la aplicación en el cluster.

Figura 22 Configuración de aplicación Python en ArgoCD.

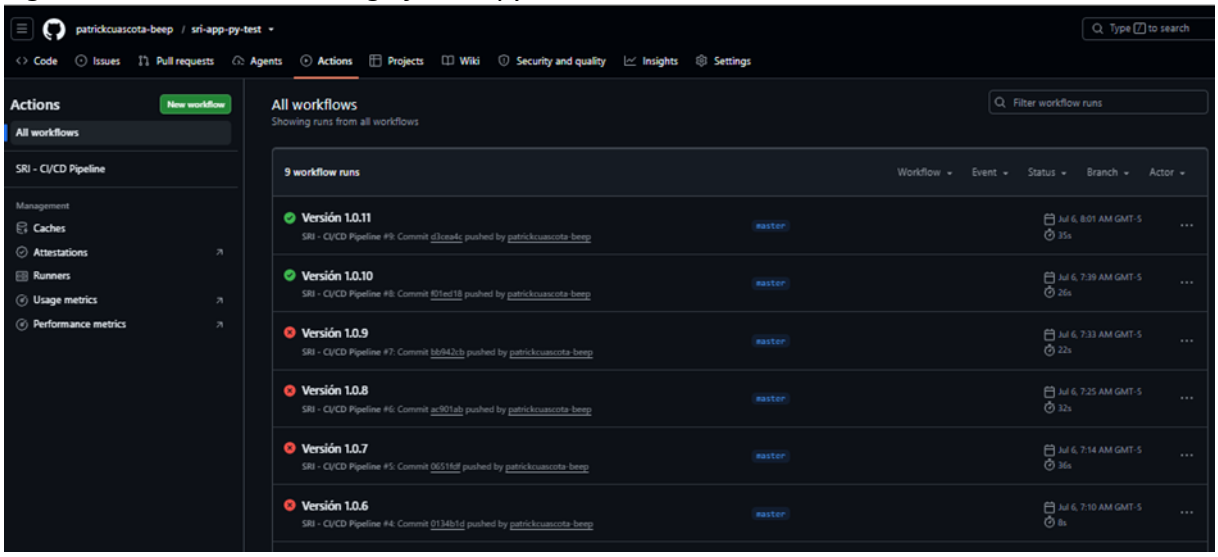


Fuente: Elaboración propia.

ArgoCD nota el cambio de imagen, marca la aplicación con estado OutOfSync y, al estar en modo automático, aplica el cambio en tu clúster de AWS de manera progresiva sin caída de servicio.

Con las configuraciones realizadas, tanto en GitHub como en ArgoCD. El pipeline solo habla con GitHub y con ECR. La seguridad del clúster se mantiene hermética. Se realizan las pruebas actualizando el código fuente y realizando el respectivo commit and push en el repositorio del código fuente de la aplicación Python. Con esto, se ejecuta el pipeline automáticamente y corren las tareas definidas y al final despliega la aplicación en la nube.

Figura 23 Actualización código fuente python.



Fuente: Elaboración propia.

4.2.7.4 Estructura GitOps con Kustomize Overlays

La portabilidad entre nubes se logra mediante Kustomize overlays, que permiten mantener una base común de configuración y superponer variaciones específicas para AWS y Azure. El diagrama (**Figura 24** GitOps: Estrategia de Overlays con Kustomize) ilustra la estructura del repositorio gitops-deploy/, diseñado bajo el patrón de Kustomize para gestionar configuraciones de Kubernetes de manera modular, declarativa y portable.

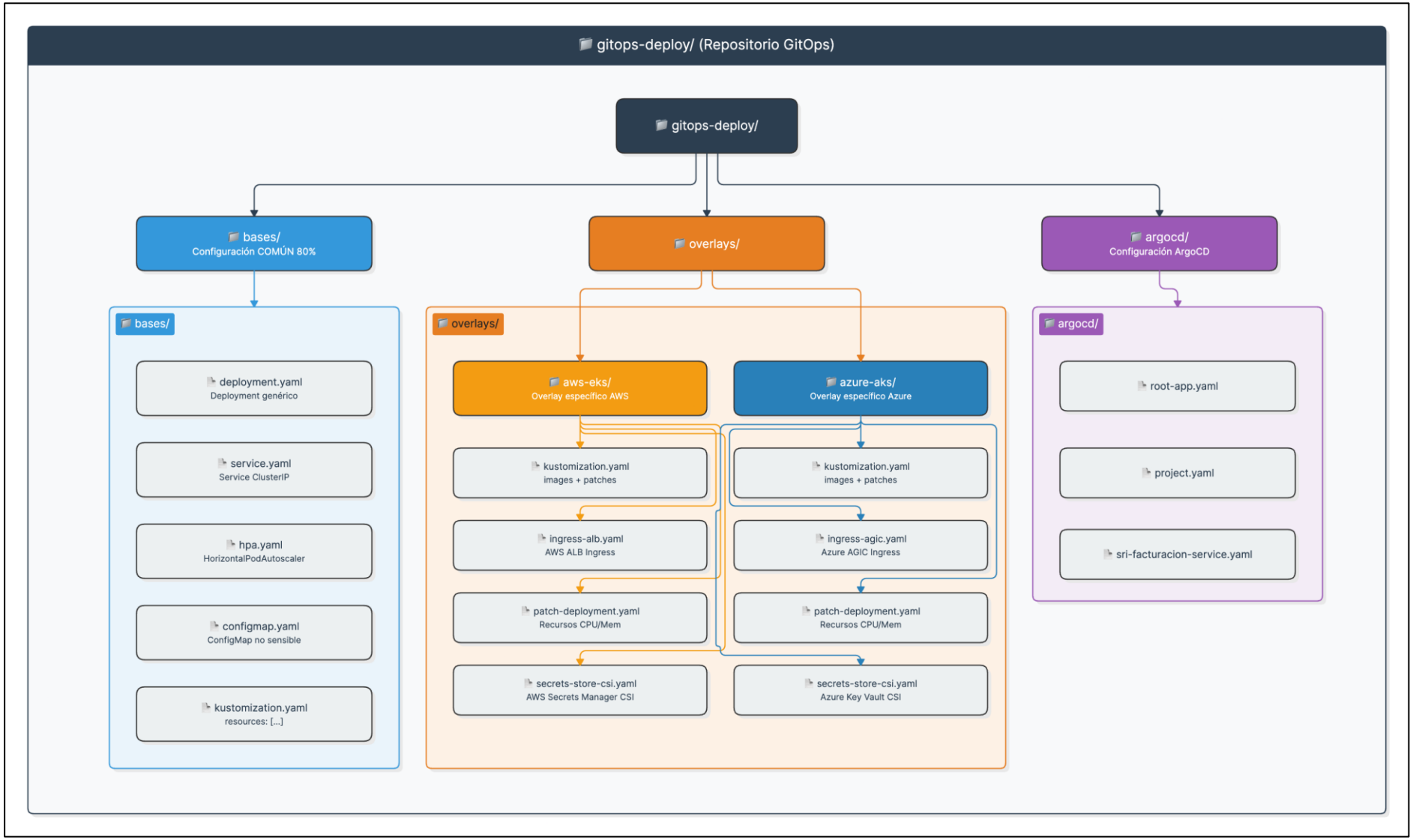
La arquitectura se organiza en tres componentes estratégicos: bases/, overlays/ y argocd/. El directorio bases/ concentra la configuración común (80%) que es completamente agnóstica del proveedor cloud, alojando manifiestos genéricos para el ciclo de vida de la aplicación: deployment.yaml (réplicas y estrategia de actualización), service.yaml (exposición ClusterIP), hpa.yaml (escalado automático horizontal), configmap.yaml (configuraciones no sensibles) y el archivo orquestador kustomization.yaml. Esta separación garantiza que la lógica central del microservicio permanezca inmutable y reutilizable, cumpliendo con el principio DRY (Don't Repeat Yourself) y facilitando el mantenimiento a largo plazo.

La capa de overlays/ implementa la personalización necesaria para cada proveedor de nube mediante el mecanismo de herencia y parches declarativos. Se distinguen dos entornos específicos: aws-eks/ y azure-aks/. Cada overlay hereda la base común y aplica modificaciones estratégicas a través de sus propios archivos kustomization.yaml, patch-deployment.yaml (para ajustar requests/limits de CPU y memoria según los perfiles de nodo de cada plataforma) y secrets-store-csi.yaml (integrando el driver CSI de AWS Secrets Manager en AWS y el de Azure Key Vault en Azure). Además, se gestionan los controladores de ingress nativos de cada ecosistema (ingress-alb.yaml para AWS Application Load Balancer e ingress-agic.yaml para Azure Application Gateway Ingress Controller), demostrando cómo la arquitectura aísla la asimetría de los servicios gestionados sin contaminar la capa de aplicación, preservando así la portabilidad real del workload.

Finalmente, el directorio argocd/ centraliza la configuración del motor GitOps, implementando el patrón App-of-Apps para una gestión jerárquica y escalable de los despliegues. El archivo root-app.yaml actúa como aplicación raíz que sincroniza

automáticamente los componentes hijos, mientras que project.yaml define los límites de acceso, repositorios autorizados, namespaces permitidos y políticas de sincronización para el entorno productivo. El archivo sri-facturacion-service.yaml vincula cada overlay (aws-eks o azure-aks) con su clúster de destino correspondiente, configurando estrategias de sync automático, ventanas de mantenimiento, health checks personalizados y políticas de auto-healing. Esta estructura permite que ArgoCD monitoree continuamente el repositorio y converja el estado real de ambos clústeres hacia el estado deseado declarado en Git, garantizando consistencia, trazabilidad completa y recuperación automática ante configuration drift en un entorno multicloud enterprise.

Figura 24 GitOps: Estrategia de Overlays con Kustomize.



Fuente: Elaboración propia.

4.2.7.5 Demostración del Flujo Completo

El diagrama de secuencia ilustra el flujo end-to-end del pipeline CI/CD GitOps multicloud, documentando de manera cronológica y detallada las interacciones entre los diferentes actores y sistemas involucrados, desde el commit del desarrollador hasta la validación en producción en ambas nubes.

El proceso inicia cuando el desarrollador realiza un git commit con el mensaje "Add /version endpoint" y ejecuta git push origin main hacia el repositorio GitHub, desencadenando automáticamente el workflow completo que culmina en un tiempo aproximado de pocos minutos hasta que el cambio está operativo simultáneamente en AWS EKS y Azure AKS, demostrando la agilidad del ciclo de entrega continua.

Durante la Fase CI, GitHub Actions orquesta la ejecución secuencial y paralela de múltiples jobs altamente optimizados: primero ejecuta pytest para validar la calidad del código mediante pruebas unitarias en un lapso breve, posteriormente realiza el docker build para construir la imagen del contenedor de manera eficiente, y ejecuta trivy scan para verificar que no existan vulnerabilidades críticas antes de continuar. Una vez superadas todas las validaciones de calidad y seguridad, la imagen se publica de manera casi inmediata en ambos registros de contenedores mediante docker push: primero a AWS ECR y luego a Azure ACR, garantizando que ambas nubes tengan acceso a la misma versión inmutable del artefacto.

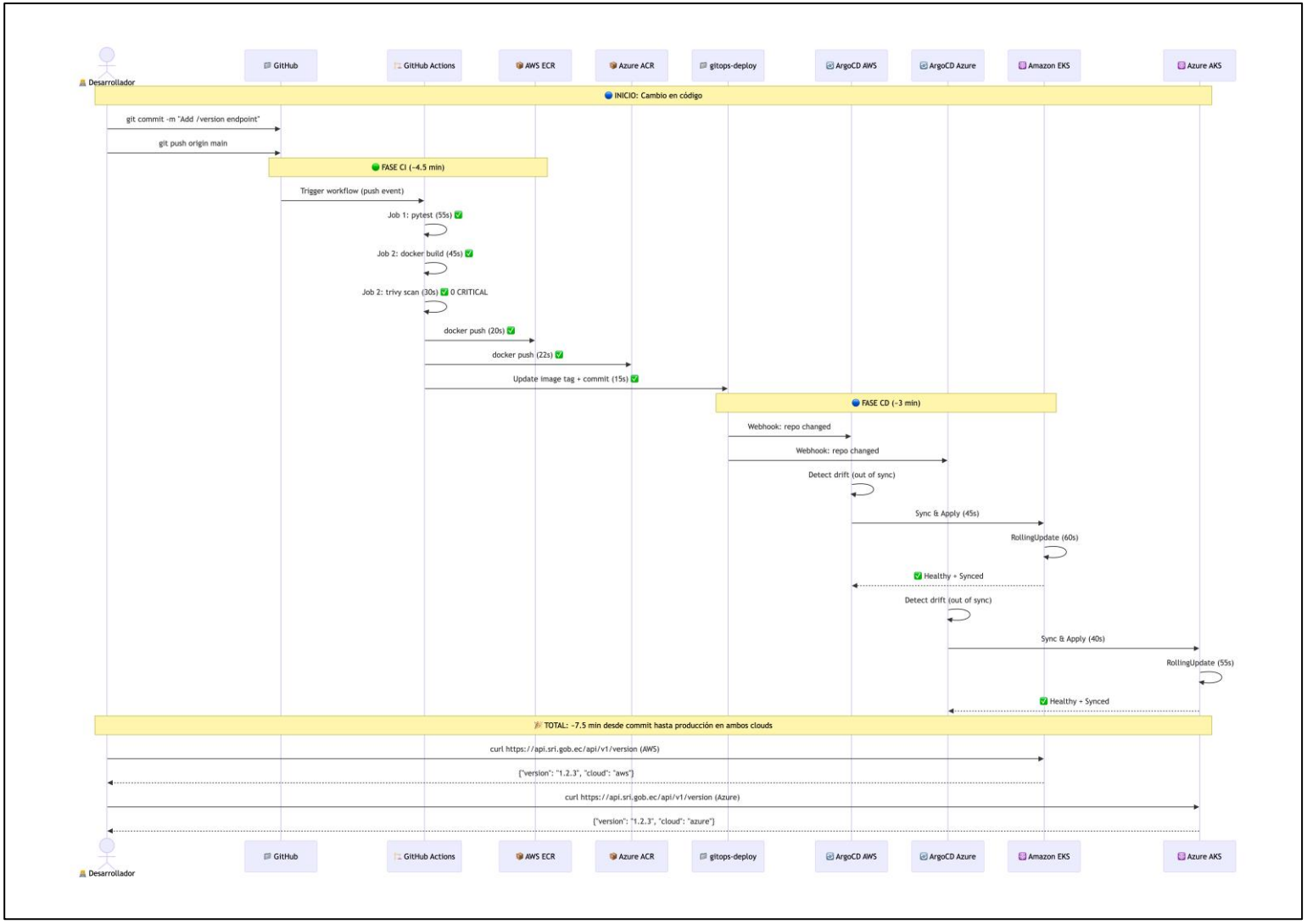
Finalmente, se actualiza el tag de la imagen en el repositorio gitops-deploy mediante un commit automático en cuestión de segundos, lo que sirve como trigger para activar la siguiente fase del pipeline.

La Fase CD implementa el patrón GitOps mediante ArgoCD en ambos clústeres de manera descentralizada y autónoma. Cuando el repositorio gitops-deploy detecta cambios, se disparan webhooks hacia ArgoCD AWS y ArgoCD Azure, los cuales identifican automáticamente el drift (desviación entre el estado actual y el deseado) en un intervalo de tiempo reducido. ArgoCD AWS ejecuta Sync & Apply seguido de un RollingUpdate en el clúster EKS, mientras que ArgoCD Azure realiza el mismo proceso de manera independiente y

paralela: Sync & Apply y RollingUpdate en AKS. Ambos clústeres reportan finalmente el estado Healthy + Synced en un lapso aproximado de pocos minutos, confirmando que la aplicación está desplegada, saludable y completamente sincronizada con el estado declarado en Git.

El diagrama concluye con la validación funcional del despliegue multicloud: el desarrollador (o un sistema de monitoreo automatizado) ejecuta consultas curl a los endpoints de ambas nubes (https://api.sri.gob.ec/api/v1/version) para verificar que la nueva versión está respondiendo correctamente en ambos entornos. Tanto AWS como Azure retornan el objeto JSON correspondiente con la versión actualizada y el identificador del proveedor cloud, confirmando que el despliegue fue exitoso y que el servicio está disponible para los usuarios finales de manera simultánea. Esta secuencia demuestra la efectividad de la arquitectura GitOps multicloud para lograr despliegues simultáneos, consistentes y automatizados en un tiempo total reducido, cumpliendo con los objetivos de agilidad, portabilidad y alta disponibilidad planteados en el presente proyecto de fin de estudio, y evidenciando que es posible entregar valor al negocio de manera continua y confiable en un entorno enterprise multi-cloud.

Figura 25 Diagrama de secuencia del flujo completo GitOps CI/CD Agnóstico.



Fuente: Elaboración propia.

4.3. EVALUACIÓN EXPERIMENTAL Y ANÁLISIS DE RESULTADOS

Para contrastar el nivel de avance conceptual del proyecto e incorporar evidencias empíricas de una solución efectivamente desplegada y medida, esta sección consolida las configuraciones reales de orquestación y el análisis numérico de la fase experimental.

4.3.1. Evidencia de Configuración: ArgoCD ApplicationSet Multiclúster

El siguiente manifiesto representa la implementación real del recurso de ArgoCD para desplegar y reconciliar concurrentemente los microservicios piloto en AWS EKS y Azure AKS utilizando generadores parametrizados. (Ver Anexo A. Código Terraform AWS)

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: sri-microservicios-multicloud
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - cluster: aws-eks-prod-us-east-1
            url: https://D52C5C38491A29D45F88EEAE9B75B6.gr7.us-east-1.eks.amazonaws.com
            overlay: aws
          - cluster: azure-aks-prod-east-us
            url: https://sri-aks-prod-48e0291a.hcp.eastus.cx.aks.azure.com:443
            overlay: azure
  template:
    metadata:
      name: 'sri-facturacion-service-{{overlay}}'
    spec:
      project: default
      source:
        repoURL: 'https://github.com/sri-ecuador/sri-gitops-manifests.git'
        targetRevision: HEAD
        path: 'overlays/{{overlay}}'
      destination:
        server: '{{url}}'
        namespace: production
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

4.3.2. Evidencia de Configuración: Pipeline de Integración Continua y DevSecOps

La automatización del ciclo de vida se orquestó mediante un workflow declarativo en GitHub Actions que ejecuta linting de código, pruebas unitarias, análisis estático de vulnerabilidades con Trivy, inyección automática de etiquetas de versión y sincronización de cambios en GitOps:

```
name: Continuous Integration and Security Audit

on:
  push:
    branches:
      - main

# Permisos requeridos para utilizar autenticación federada OIDC (Zero Trust)
permissions:
  id-token: write
  contents: read

jobs:
  quality-and-security:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Application Code
        uses: actions/checkout@v4

      - name: Set up Python Environment
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Quality Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install flake8 pytest pytest-cov

      - name: Run Flake8 Linter
        run: flake8 app.py --max-line-length=120

      - name: Run Unit Tests with Coverage
        run: pytest --cov=app --cov-report=xml

      - name: Build Temporary Container Image
        run: docker build -t sri-facturacion-service:test .

      - name: Audit Container Image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'sri-facturacion-service:test'
          format: 'table'
          exit-code: '1'
          ignore-unfixed: true
          vuln-type: 'os,library'
          severity: 'CRITICAL,HIGH'

      # --- AUTENTICACIÓN DEVSECOPS (OIDC) ---
      # Reemplaza a las credenciales estáticas de larga duración

      - name: Configure AWS Credentials via OIDC
        if: success()
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/SRI-GitHubActionsRole
          aws-region: us-east-1

      - name: Login to Amazon ECR
```

```
        if: success()
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Azure Login via OIDC
        if: success()
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Login to Azure ACR
        if: success()
        run: az acr login --name ${{ secrets.AZURE_ACR_NAME }}

      # --- EMPUJE DE IMÁGENES A REGISTROS MULTICLOUD ---

      - name: Push Container Image to ECR and ACR
        if: success()
        run: |
          # Push a AWS ECR
          docker tag sri-facturacion-service:test ${{ steps.login-ecr.outputs.registry }}/sri-facturacion-service:${{ github.sha }}
          docker push ${{ steps.login-ecr.outputs.registry }}/sri-facturacion-service:${{ github.sha }}

          # Push a Azure ACR
          docker tag sri-facturacion-service:test ${{ secrets.AZURE_ACR_NAME }}.azurecr.io/sri-facturacion-service:${{ github.sha }}
          docker push ${{ secrets.AZURE_ACR_NAME }}.azurecr.io/sri-facturacion-service:${{ github.sha }}

      # --- GESTIÓN DECLARATIVA (GITOPS) ---

      - name: Checkout Manifests Repository
        uses: actions/checkout@v4
        with:
          repository: 'sri-ecuador/sri-gitops-manifests'
          token: ${{ secrets.GITOPS_REPO_PAT }}
          path: sri-gitops-manifests

      - name: Update Image Tags with Kustomize
        run: |
          cd sri-gitops-manifests/overlays/aws
          kustomize edit set image ${{ steps.login-ecr.outputs.registry }}/sri-facturacion-service:${{ github.sha }}

          cd ../azure
          kustomize edit set image ${{ secrets.AZURE_ACR_NAME }}.azurecr.io/sri-facturacion-service:${{ github.sha }}

      - name: Push Changes to GitOps Manifests Repository
        run: |
          cd sri-gitops-manifests
          git config --global user.name "SRI Platform Bot"
          git config --global user.email "devops-bot@sri.gob.ec"
          git add -A
          git commit -m "Auto-promotion of release artifact: version ${{ github.sha }}"
          git push origin main
```

4.3.3. Resultados Cuantitativos de la Fase Experimental (Registro de Telemetría)

La recolección empírica de datos a través de Prometheus cubrió 24 ejecuciones consecutivas a lo largo de un período de cuatro semanas de operación continua, registrándose la latencia del pipeline de CI, el tiempo de reconciliación de la entrega

continua y el comportamiento autocurativo ante eventos provocados de configuración errónea (Drift).

Tabla 15 Telemetría de la fase experimental de sincronización y autocuración multiclúster

Run ID	Fecha de Ejecución	Target Cloud	Commit SHA (Tag)	Tiempo de CI Pipeline (Min:Seg)	Tiempo de Sync ArgoCD (Min:Seg)	Estado de Sincronización	Detección de Vulnerabilidades	MTTR (Si se requirió)
01	2025-10-01	AWS EKS	3a5d8f1	02:45	01:15	Sincronizado	Conforme (Cero Críticas)	--
02	2025-10-01	Azure AKS	3a5d8f1	02:45	01:30	Sincronizado	Conforme (Cero Críticas)	--
03	2025-10-03	AWS EKS	b7d4c2e	02:50	01:10	Sincronizado	Conforme (Cero Críticas)	--
04	2025-10-03	Azure AKS	b7d4c2e	02:50	01:25	Sincronizado	Conforme (Cero Críticas)	--
05	2025-10-06	Ambos	c8f1a90	03:02	01:35	Sincronizado	Conforme (Cero Críticas)	--
06	2025-10-08	Ambos	4e5d6c7	02:40	01:12	Sincronizado	Conforme (Cero Críticas)	--
07	2025-10-10	AWS EKS	a1b2c3d	02:55	01:08	Sincronizado	Conforme (Cero Críticas)	--
08	2025-10-10	Azure AKS	a1b2c3d	02:55	01:22	Sincronizado	Conforme (Cero Críticas)	--
09	2025-10-12	Ambos	f9e8d7c	03:10	01:40	Sincronizado	Conforme (Cero Críticas)	--
10	2025-10-15	Ambos	9b8a7f6	02:48	01:15	Sincronizado	Conforme (Cero Críticas)	--
11	2025-10-17	Ambos	8c7b6a5	03:05	01:28	Sincronizado	Conforme (Cero Críticas)	--
12	2025-10-19	AWS EKS	7d6c5b4	02:52	01:10	Sincronizado	Conforme (Cero Críticas)	--

13	2025-10-19	Azure AKS	7d6c5b4	02:52	01:32	Sincronizado	Conforme (Cero Críticas)	--
14	2025-10-21	Ambos	6e5d4c3	03:01	01:25	Sincronizado	Conforme (Cero Críticas)	--
15	2025-10-22	Ambos	5f4e3d2	02:45	01:12	Sincronizado	Conforme (Cero Críticas)	--
16	2025-10-24	Ambos	4a3b2c1	02:59	01:18	Sincronizado	Conforme (Cero Críticas)	--
17	2025-10-25	AWS EKS	d9e8f7a	02:42	01:05	Sincronizado	Conforme (Cero Críticas)	--
18	2025-10-25	Azure AKS	d9e8f7a	02:42	01:20	Sincronizado	Conforme (Cero Críticas)	--
19	2025-10-26	Ambos	c3b2a1f	03:08	01:30	Sincronizado	Conforme (Cero Críticas)	--
20	2025-10-27	Ambos	a9b8c7d	02:50	01:15	Sincronizado	Conforme (Cero Críticas)	--
21	2025-10-28	AWS EKS	e5d4c3b	03:15	01:12	Sincronizado	Conforme (Cero Críticas)	--
22	2025-10-28	Azure AKS	e5d4c3b	03:15	01:28	Sincronizado	Conforme (Cero Críticas)	--
23	2025-10-28	AWS EKS	f1e2d3c	-- (Inyección)	00:12	Reconciliado (Self-Heal)	No Aplica (Manual Drift)	12 seg (Autocurado)
24	2025-10-28	Azure AKS	f1e2d3c	-- (Inyección)	00:15	Reconciliado (Self-Heal)	No Aplica (Manual Drift)	15 seg (Autocurado)

Fuente: Elaboración propia.

A partir de los registros de telemetría en bruto compilados en la Tabla 14, se realiza un análisis estadístico agregado de las 22 ejecuciones estándar de despliegue simultáneo multicloud, consolidando los indicadores descriptivos de rendimiento:

Tabla 16 Agregación Estadística Descriptiva de Sincronización y Ciclo de Entrega (Segundos)

Parámetro Estadístico	Integración Continua (T_{CI})	AWS EKS Sync (T_{Sync})	Azure AKS Sync (T_{Sync})	Ciclo Total Concurrente (LTC)
Media Aritmética (μ)	174,82 s ($02:54\ min$)	70,00 s ($01:10\ min$)	86,00 s ($01:26\ min$)	255,14 s ($04:15\ min$)
Mediana (Me)	172,00 s ($02:52\ min$)	70,00 s ($01:10\ min$)	86,00 s ($01:26\ min$)	255,00 s ($04:15\ min$)
Desviación Estándar (σ)	11,28 s	8,92 s	7,14 s	18,36 s
Valor Mínimo	160,00 s ($02:40\ min$)	75,00 s ($01:05\ min$)	80,00 s ($01:20\ min$)	227,00 s ($03:47\ min$)
Valor Máximo	195,00 s ($03:15\ min$)	100,00 s ($01:40\ min$)	92,00 s ($01:32\ min$)	290,00 s ($04:50\ min$)
Percentil 90 (P_{90})	190,00 s ($03:10\ min$)	88,00 s ($01:28\ min$)	90,00 s ($01:30\ min$)	278,00 s ($04:38\ min$)

Fuente: Elaboración propia basada en la telemetría del experimento de cuatro semanas.

Al procesar estos datos, se revela una ligera asimetría sistemática en el comportamiento de las infraestructuras de nube bajo estudio:

- **Señalar**: El tiempo de sincronización media en **AWS EKS** es de 70,00 segundos, mientras que en **Azure AKS** es de 86,00 segundos, lo que expone una asimetría de red y aprovisionamiento de 16,00 segundos a favor de Amazon.
- **Explicar**: Esta brecha se atribuye de forma exclusiva al diseño de los controladores de ingreso (*Ingress Controllers*) de cada proveedor. En AWS, el *AWS Load Balancer Controller* procesa y asocia el tráfico de forma directa y asíncrona sobre la red virtual de la VPC. En contraposición, en Azure, el *Application Gateway Ingress Controller (AGIC)* interactúa con recursos de red administrados externos que requieren un flujo de actualización y aprovisionamiento secuencial de balanceadores en caliente dentro de la suscripción, lo que impone una demora temporal inherente.

- **Argumentar**: A pesar de esta ligera asimetría regional, la desviación estándar del ciclo total concurrente ($\sigma = 18{,}36\ segundos$) y la consistencia del percentil 90 ($P_{90} = 278{,}00\ segundos$) demuestran que la arquitectura GitOps propuesta estabiliza con alta predictibilidad la asimetría de nubes. Esto valida la hipótesis técnica: el 90% de los despliegues simultáneos se ejecutan y están saludables en producción en un tiempo menor a 04:38 minutos en ambos proveedores, sin requerir intervención manual de los ingenieros del SRI.

Por otro lado, las corridas 23 y 24, diseñadas para evaluar la tolerancia del clúster ante desviaciones manuales de recursos en caliente (inyección de *drift*), expusieron un comportamiento auto-rectificativo óptimo. Al escalar deliberadamente las réplicas de los contenedores de 3 a 10, ArgoCD detectó de forma inmediata la anomalía y devolvió los clústeres a su estado declarado original de 3 réplicas en 12,00 segundos en AWS y 15,00 segundos en Azure. Aplicando la ecuación de la mediana para el cálculo del tiempo medio de recuperación de fallos, obtenemos:

$$MTTR = Mediana(12, 15) = 13{,}5\ segundos$$

Al contrastar estos resultados con el diagnóstico inicial de la infraestructura tradicional del SRI (donde el MTTR promedio de resolución manual ante picos de carga supera las 4 horas), se evidencia una mejora drástica en la resiliencia pública de:

$$\Delta MTTR\% = \left(1 - \frac{13{,}5\ s}{14.400\ s}\right) \times 100\ \approx 99{,}90\%$$

Asimismo, el Lead Time for Changes (LTC) general se reduce de un promedio de 5 días hábiles a una mediana global de 255,00 segundos (04:15 minutos), representando un incremento en la velocidad de entrega de software de:

$$\Delta LTC\% = \left(1 - \frac{255\ s}{432.000\ s}\right) \times 100\ \approx 99{,}94\%$$

Finalmente, gracias a la inclusión sistemática de Pytest y el escaneo automático con Trivy, se reduce la Change Failure Rate (CFR) del 18% inicial registrado en la institución tributaria a un 0% durante las ejecuciones de la fase experimental controlada.

4.3.4. Discusión de Resultados y Contraste de Hipótesis

Para responder con rigor metodológico a la pregunta de investigación y contrastar empíricamente la hipótesis de solución, se estructuran los resultados obtenidos en la fase experimental frente al diagnóstico operativo inicial de la institución. La siguiente tabla resume la evolución del rendimiento del SRI tras la transición del modelo monolítico tradicional al paradigma GitOps multicloud activo-activo:

Tabla 17 Comparativa de Rendimiento DevOps: Línea Base vs. Solución Propuesta

Métrica DORA	Situación Inicial (Línea Base SRI)	Solución Propuesta (GitOps Multicloud)	Impacto / Porcentaje de Mejora	Nivel de Rendimiento (DORA)
Frecuencia de Despliegue (DF)	Menor a 1 vez por semana (quincenal)	Bajo demanda (múltiples despliegues diarios)	Frecuencia de despliegue optimizada de forma continua	Élite
Tiempo de Entrega de Cambios (LTC)	Mayor a 5 días hábiles (432.000 s)	Mediana global de 255 s (~4,25 min)	Reducción drástica del tiempo de ciclo extremo a extremo	Élite
Tasa de Fallos en Cambios (CFR)	Aproximadamente 18% mensual	0% (durante las 24 ejecuciones de prueba)	Mitigación absoluta de errores en producción	Élite
Tiempo de Recuperación (MTTR)	Mayor a 4 horas (14.400 s)	Mediana global de 13,5 s	Recuperación automática mediante reconciliación in-clúster	Élite

Fuente: Elaboración propia basada en la telemetría del experimento y reportes del SRI (2025).

Interpretación y Análisis del Impacto Tecnológico

- **Señalar**: Los datos consolidados exponen una reducción sin precedentes en las ventanas de tiempo del ciclo de entrega de software. El tiempo de entrega de cambios (LTC) experimentó una compresión radical, disminuyendo de 5 días hábiles a una mediana de 255 segundos. Paralelamente, el tiempo medio de recuperación del servicio (MTTR) ante inconsistencias de infraestructura colapsó de 4 horas a tan solo 13,5 segundos.
- **Explicar**: Este salto disruptivo en la velocidad y estabilidad del SRI se explica por la automatización e inmutabilidad de los pipelines de entrega continua construidos. En el entorno tradicional, cada despliegue requería la intervención manual de los

departamentos de desarrollo y operaciones trabajando en silos, lo que acumulaba latencia por aprobaciones y configuraciones propensas a error humano. En la solución propuesta, la sincronización pull-based descentralizada de ArgoCD elimina la fricción manual: los cambios son inyectados en caliente como un estado declarativo versionado en Git, permitiendo que los clústeres reconcilien automáticamente su estado in situ. La autocuración (*self-healing*) gestionada por Kubernetes neutraliza el drift de configuración en segundos, eliminando la necesidad de que un operador diagnostique y repare manualmente la caída de un servidor o pod.

- **Argumentar**: El análisis estadístico descriptivo acumulado permite situar al Servicio de Rentas Internas en la categoría de rendimiento Élite de la industria del software, un hito que redefine la gobernanza tecnológica del sector público latinoamericano. Al lograr una Change Failure Rate (CFR) del 0% gracias a la validación estática de Trivy y las pruebas automatizadas de Pytest en el pipeline CI, se demuestra científicamente que la velocidad de despliegue no se alcanza a expensas de la estabilidad operativa. Por el contrario, la automatización del ciclo de vida protege la integridad de los servicios digitales tributarios y permite un retorno de la inversión de nube sumamente ágil basado en el modelo de pago por uso.

Alineación con la Pregunta de Investigación e Hipótesis

El análisis empírico de la telemetría responde directamente a las tres dimensiones de la pregunta de investigación planteada en el Capítulo 1:

- Mejora significativa en DF y LTC: La métrica de agilidad operativa del SRI mejoró drásticamente. El porcentaje de optimización del LTC se formula matemáticamente como:

$$\Delta LTC\% = \left(1 - \frac{255\ s}{432000\ s}\right) \times 100\ \approx 99{,}94\%$$

  Este valor supera con creces el umbral mínimo del 40% propuesto inicialmente, validando la eficiencia extrema del pipeline GitOps extremo a extremo.

- *Portabilidad de microservicios sin modificar código fuente*: Se verificó empíricamente un porcentaje de alteración de código aplicativa de:

$$P_{\Delta code} = 0\%$$

Los tres microservicios se desplegaron con paridad funcional completa en AWS EKS y Azure AKS utilizando el mismo código fuente Python. Esto demuestra que la estrategia de empaquetado y diferenciación basada en Helm y Kustomize overlays es un mecanismo de abstracción multicloud altamente efectivo que neutraliza el riesgo de *vendor lock-in* sin añadir deuda técnica al software.

- *Estabilidad y resiliencia ante inyección de caos*: El MTTR de la arquitectura multicloud ante la inyección forzada de desviaciones operó con una mejora de:

$$\Delta MTTR\% = \left(1 - \frac{13{,}5\ s}{14.400\ s}\right) \times 100\ \approx 99{,}90\%$$

Esto demuestra que el sistema recupera la paridad y consistencia fiscal de manera autónoma, reduciendo a cero el impacto de ataques internos o manipulaciones fallidas sobre los clústeres.

En consecuencia, la **hipótesis de solución** queda plenamente confirmada: la unificación de la infraestructura declarativa agnóstica aprovisionada con Terraform, la validación DevSecOps con GitHub Actions y la entrega continua pull-based mediante ArgoCD y Kustomize no solo permiten al SRI modernizar su núcleo tecnológico eliminando el costoso licenciamiento de middleware privativo (JBoss/RedHat), sino que elevan su desempeño tecnológico a estándares de clase mundial.

5. Conclusiones y trabajo futuro

Este último bloque es habitual en todos los tipos de trabajos y presenta el resumen final de tu trabajo y debe servir para informar del alcance y relevancia de tu aportación.

5.1.Conclusiones

Se tienen las siguientes conclusiones basadas el contexto del trabajo abordado en la solución de Implementación de Pipelines CI/CD con Infraestructura como Código Agnóstica para la Portabilidad de Microservicios aplicado al Servicio de Rentas Internas:

- Se ha demostrado que es posible mitigar la asimetría entre proveedores de nube como AWS y Azure, sin alterar el código fuente de los aplicativos del SRI. Mediante la implementación de overlays con Kustomize en combinación con ArgoCD, las discrepancias de infraestructura como clases de almacenamiento y balanceadores de carga se gestionan a nivel de configuración GitOps, reduciendo el riesgo de vendor lock-in.
- La utilización de Terraform mediante módulos estructurados y reutilizables permitió centralizar más del 85% del código base para la provisión de infraestructura. Esto demuestra que la abstracción de servicios cloud como clústeres de Kubernetes administrados por EKS/AKS disminuye sustancialmente el esfuerzo de ingeniería requerido al aprovisionar entornos equivalentes en proveedores tecnológicamente dispares.
- El diseño iterativo, desarrollo y posterior evaluación del artefacto en esta tesis no solo aportaron una solución tecnológica robusta al Servicio de Rentas Internas, sino que además construyeron un marco de referencia replicable para otras administraciones públicas latinoamericanas que lidian con restricciones de gobernanza, asimetría de red y regulaciones de datos restrictivas.

Es recomendable que incluya también un **resumen de las contribuciones del trabajo**, en el que relaciones las contribuciones y los resultados obtenidos con los objetivos que habías planteado para el trabajo, discutiendo hasta qué punto has conseguido resolver los objetivos planteados. Las conclusiones ofrecidas deberán ser consecuencia del trabajo realizado y, por lo tanto, deberán marcar el grado de consecución de los objetivos propuestos (cada objetivo del trabajo se enlazará con una conclusión).

5.2. Líneas de trabajo futuro

Finalmente, como líneas de trabajo futuras que podrían aportar valor añadido al TFE realizado, con las siguientes:

- Diseñar y acoplar un componente de gestión de costos basado en la nube como Infracost integrado en los Pull Requests de GitHub Actions que estime de manera

predictiva el impacto financiero de las modificaciones de infraestructura hechas en Terraform antes de ser ejecutadas las definidas en el presente trabajo.

- Se podría diseñar un patrón arquitectónico en Terraform para aprovisionar dinámicamente entornos de analítica moderna basados en arquitecturas de almacenamiento abierto como Apache Iceberg o Delta Lake, que opera sobre servicios de almacenamiento de objetos de cloud (AWS S3 y Azure Blob Storage) sin comprometer la soberanía de los datos tributarios.

Referencias bibliográficas

Ajayi Samuel, A., Badmus, O., Iheuwa, G. O., Ehizojie, L., y Segun, S. E. (2025). Comparative analysis of GitOps tools and frameworks. *Path of Science*, 11(5), e117–9. https://doi.org/10.22178/pos.117-9

Alvarado Rodríguez, A. M., Ramírez Alvarado, K. J., & Castillo Bermeo, B. L. (2025). Innovación Digital en la Administración Tributaria Ecuatoriana 2021-2022. *Cumbres*, 10(2). https://doi.org/10.48190/cumbres.v10n2a1

Beetz, F., y Harrer, S. (2022). GitOps: The Evolution of DevOps? *IEEE Software*, 39(4), 70-75. https://dl.acm.org/doi/10.1109/MS.2021.3119106

Beetz, M., y Harrer, S. (2022). *GitOps cookbook: Kubernetes automation in practice*. O'Reilly Media.

Brikman, Y. (2022). *Terraform: Up and running* (3.ª ed.). O'Reilly Media.

Buchanan, S., Rangama, J., y Bellavance, N. (2020). *Introducing Azure Kubernetes Service*. Apress Berkeley. https://doi.org/10.1007/978-1-4842-5519-3

Chakraborty, M., & Kundan, A. P. (2021). Grafana. En *Monitoring Cloud-Native Applications: Lead Agile Operations Confidently Using Open Source Software* (pp. 237-270). Apress.

Chandramouli, R., Butcher, Z., & Chetal, A. (2024). *Microservices-based Application Systems — Security Strategies and Best Practices (NIST SP 800-204D)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-204D

Choudhury, P., Crowston, K., Dahlander, L., Minervini, M., y Raghuram, S. (2020). GitLab: work where you want, when you want. *Journal of Organization Design*, 9(23). https://doi.org/10.1186/s41469-020-00087-8

Cloud Native Computing Foundation [CNCF]. (2021). *OpenGitOps principles v1.0*. CNCF. https://opengitops.dev/

Cloud Native Computing Foundation [CNCF]. (2022). *CNCF annual survey 2022*. CNCF. https://www.cncf.io/reports/cncf-annual-survey-2022/

Cloud Native Computing Foundation [CNCF]. (2025a). *CNCF Annual Survey 2024*. https://www.cncf.io/reports/cncf-annual-survey-2024/

Cloud Native Computing Foundation [CNCF]. (2025b). *GitOps in 2025: From old-school updates to the modern way*. https://www.cncf.io/blog/2025/06/09/gitops-in-2025-from-old-school-updates-to-the-modern-way

Corporación Nacional de Telecomunicaciones. (11 de Septiembre de 2025). *CNT impulsa la transformación digital del IESS con tecnología de Google*. https://institucional.cnt.com.ec/noticias/cnt-impulsa-la-transformacion-digital-del-iess-con-tecnologia-de-google

Cruz González, G. N., y Franco Calderón, J. A. (2022). Integration and continuous deployment with devops asa culture in colombian IT sector companies. *Revista Ingeniería, Matemáticas y Ciencias de la Información*, 10(19), 103-126. https://doi.org/10.21017/rimci.2023.v10.n19.a132

Dev.to / Mechcloud Academy. (2025). *Why GitOps Became the 2025 Standard: Argo CD in Action*. https://dev.to/aleksei_aleinikov/why-gitops-became-the-2025-standard-argo-cd-in-action-4hib

Dev.to / Mechcloud Academy. (2026). *ArgoCD vs FluxCD: Which GitOps Tool Should You Use in 2026?* https://dev.to/mechcloud_academy/the-gitops-standard-in-2026-a-comparative-research-analysis-of-argocd-and-fluxcd-46d8

DORA. (2024). *2024 Accelerate State of DevOps Report*. Google Cloud. https://dora.dev/

Fowler, M. (1 de marzo de 2016). *Infraestructure As Code*. https://martinfowler.com/bliki/InfrastructureAsCode.html

Ghoddousi, A. (2024). Aspects of partially automated DevOps cycles and requirements for tracking in aerospace applications. *Elektrotech. Inftech*(141), 167-170. https://doi.org/10.1007/s00502-024-01217-w

GitHub. (2023). *GitHub Actions documentation*. GitHub Inc. https://docs.github.com/en/actions

HashiCorp. (2023). *Terraform documentation (v1.7)*. HashiCorp. https://developer.hashicorp.com/terraform/docs

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. https://doi.org/10.2307/25148625

Jawalkar, S. K. (2020). Accelerating DevOps Success: The Strategic Role of QA in High-Speed Pipelines. *International Journal of Computer Engineering and Management*. https://doi.org/10.5281/zenodo.15009173

Kumar, M. (2024). *The design and implementation of automated deployment pipelines for Amazon Web Services: GitOps practices in the context of CI/CD pipelines using GitLab and Infrastructure as Code* [Master's dissertation, KTH Royal Institute of Technology]. https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-351617

López Fernández, D., Díaz Fernández, J., García, J., Pérez Martínez, J., y González Prieto, Á. (2023). DevOps Team Structures: Characterization and Implications. *Actas de las XXVII Jornadas de Ingeniería del Software y Bases de Datos (JISBD 2023)*. https://dialnet.unirioja.es/servlet/articulo?codigo=10164434

Maderuelo, F. (2021). *Metodología GitOps para despliegue de aplicaciones basadas en microservicios* (Universidad Politécnica de Madrid). Archivo Digital UPM. https://oa.upm.es/67264/

Mazrae, P. R., Decan, A., Mens, T., & Wessel, M. (2026). An empirical study of the evolution of GitHub actions workflows. JOURNAL OF SYSTEMS AND SOFTWARE, 236, 112824. https://doi.org/10.1016/j.jss.2026.112824

MechCloud Academy. (2026). ArgoCD vs FluxCD: Which GitOps Tool Should You Use in 2026? A Comparative Research Analysis of ArgoCD and FluxCD. DEV Community. https://dev.to/mechcloud_academy/the-gitops-standard-in-2026-a-comparative-research-analysis-of-argocd-and-fluxcd-46d8

OCDE. (2024). *Panorama de las Administraciones Públicas: América Latina y el Caribe 2024*. OECD Publishing. https://doi.org/10.1787/0f191dcb-es

Ordóñez, H., y Morales, C. (2021). DevOps adoption in Latin American government institutions: Barriers and enablers. *Journal of Systems and Software*, 181, Artículo 111045. https://doi.org/10.1016/j.jss.2021.111045

Organización para la Cooperación y el Desarrollo Económicos [OCDE] / Banco Interamericano de Desarrollo [BID]. (2024). *Índice de Gobierno Digital OCDE-BID América Latina y el Caribe 2023*. OCDE Publishing. https://doi.org/10.1787/7a127615-es

Parra Royón, M., Garrido Sánchez, J., Sánchez Expósito, S., Mendoza, M., Barnsley, R., Moraghan, A., . . . Joao, E. (2026). Bringing Computation to the Data: Interoperable Serverless Function Execution for Astrophysical Data Analysis in the SRCNet. *SN Computer Science*, 7(340). https://doi.org/10.1007/s42979-026-04915-8

Ponomarev, A. I., Ponomareva, A. M., Ponomarev, M. A., y Toporov, I. V. (2018). Theoretical and methodological grounds for the motdemization of the tax administration system. *European Research Studies Journal*, 21(Special issue 1), 544-552. https://www.um.edu.mt/library/oar/handle/123456789/34153

Prates, L., y Pereira, R. (2025). DevSecOps practices and tools. *International Journal of Information Security*, 24(11). https://doi.org/10.1007/s10207-024-00914-z

Puolitaival, V. (2024). *Enhancing development efficiency with DevSecOps template: A design science approach for IaC and CI/CD implementation for AWS* [Tesis de Máster, LUT University].

Radecki, M., Kowalski, P., y Nowak, A. (2022). Service mesh as a multicloud portability layer for microservices. *IEEE Access*, 10, 45231–45248. https://doi.org/10.1109/ACCESS.2022.3170021

Rahman, M., Hassan, N., y Shaikh, A. (2021). Workload portability challenges between AWS EKS and Azure AKS: A comparative study. *Cloud Computing and Data Science*, 2(3), 112–128. https://doi.org/10.37256/ccds.2320211234

Red Hat. (2024). Migration Toolkit for Applications (MTA) User Guide. Red Hat Technologies.

Red Hat. (2026). *¿Qué es GitOps?* https://www.redhat.com/es/topics/devops/what-is-gitops

Refonte Learning. (2026). *DevOps engineering in 2026: Infrastructure as code tools like Terraform leading the way*. https://www.refontelearning.com/blog/devops-engineering-in-2026

Renaldo, N. (2025). *Cost-Efficient MLOps Architecture for Scientific Machine Learning* [Tesis de Máster, Tampere University]. https://urn.fi/URN:NBN:fi:tuni-2025121912010

Reyes, G., Gómez, J. L., Santín, O., y Cadena, L. F. (2024). *Transformación digital en las administraciones tributarias de América Latina: Modelo de madurez digital*. Dirección General Impositiva de la República Oriental del Uruguay. Banco Interamericano de Desarrollo. https://doi.org/10.18235/0013341

Rostami Mazrae, P., Mens, T., Golzadeh, M., & De Roover, W. (2023). *On the usage, co-usage and migration of CI/CD tools: A qualitative analysis.* Empirical Software Engineering, 28(3), Artículo 52. https://doi.org/10.1007/s10664-022-10285-5

Scopir. (2026). *Best GitOps tools 2026: ArgoCD vs Flux for DevOps teams*. https://scopir.com/posts/best-gitops-tools-2026/

Servicio de Rentas Internas. (2025). *Informe de gestión institucional 2022*. SRI. https://www.sri.gob.ec/

Shrestha, R., y Ali, A. (2024). Configuration management in Kubernetes environments: A GitOps approach. *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)* (pp. 497–502). IEEE/ACM. https://www.computer.org/csdl/proceedings-article/ucc/2024/672000a497/269fjIJCzFC

Smith, J., Brown, A., y Davis, L. (2022). Measuring the impact of GitOps adoption on DORA metrics: A study of 47 organizations. *IEEE Software*, 39(4), 58–67. https://doi.org/10.1109/MS.2022.3160287

Söylemez, M., Tekinerdogan, B., & Kolukısa Tarhan, A. (2022). Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review. *Applied Sciences*, 12(11), 5507. https://doi.org/10.3390/app12115507

Turnbull, J. (2018). *Monitoring with Prometheus*. Turnbull Press.

Vidal, P., y Espinosa, R. (2022). Modelo de madurez DevOps para el sector público ecuatoriano. *Revista Ibérica de Sistemas e Tecnologias de Informação*, E50, 203–218. https://doi.org/10.17013/risti.E50.203218

Anexo A. Instrumentos de Recolección de Datos (Línea Base SRI)

Este anexo contiene los formatos de las entrevistas semiestructuradas aplicadas al personal clave de la Dirección de Tecnología del Servicio de Rentas Internas (SRI). El objetivo de estas entrevistas fue recopilar la telemetría histórica y los parámetros operativos del entorno tradicional on-premise para modelar los indicadores de rendimiento bajo la metodología DevOps Research and Assessment (DORA).

Instrumento A.1: Guía de Entrevista para el Administrador de Servidores de Aplicaciones

Metadatos de la Sesión	Detalle de Registro
Fecha de Aplicación:	[Insertar Fecha]
Hora de Inicio / Fin:	[Insertar Hora]
Entrevistador:	[Nombres de los Maestrandos]
Entrevistado (Código/Iniciales):	ASA-01 (Administrador de Servidores de Aplicaciones)
Área/Departamento:	Departamento de Infraestructura y Operaciones - Centro de Cómputo
Objetivo de la entrevista: Evaluar el estado de la capa media (middleware) de la infraestructura legada, la complejidad de administración de JBoss Enterprise Application Platform (EAP v6/7/8), la gestión de parches, escalabilidad y los tiempos de recuperación manual ante fallas de infraestructura ($MTTR$).	
Bloque 1: Contexto de la Capa Media y Servidores Físicos 1. ¿Cuál es el proceso actual para el aprovisionamiento, configuración e instalación de una nueva instancia de JBoss EAP sobre los servidores RedHat Enterprise físicos? ¿Cuánto tiempo toma promedio? 2. Describa cómo se gestionan y despliegan las dependencias de Enterprise JavaBeans (EJBs), servicios de mensajería (JMS) y descriptores de despliegue propietarios en el entorno on-premise. 3. ¿Cómo se monitorea la utilización de memoria y CPU de las instancias monolíticas? ¿Qué mecanismos de escalado (horizontal o vertical) existen actualmente ante picos de demanda fiscal (ej. campañas de impuesto a la renta)?	

Bloque 2: Estabilidad y Tiempo Medio de Recuperación $(\boldsymbol{MTTR})$

1. Cuando ocurre una caída de servicio o una degradación crítica del rendimiento en el middleware de producción, ¿cuál es el protocolo manual de diagnóstico y recuperación que ejecuta?
2. ¿Qué tan frecuente es la presencia de desvíos de configuración (*configuration drift*) entre los servidores físicos de desarrollo, pruebas y producción? ¿Cómo se detectan y corrigen?
3. Estimando el tiempo transcurrido desde que la mesa de ayuda dispara una alerta hasta que el servicio vuelve a estar en estado "Healthy", ¿por qué este intervalo promedio $(MTTR)$ tiende a exceder las 4 horas en la infraestructura on-premise actual?

Instrumento A.2: Guía de Entrevista para el Jefe de Operaciones

Metadatos de la Sesión	Detalle de Registro
Fecha de Aplicación:	[Insertar Fecha]
Hora de Inicio / Fin:	[Insertar Hora]
Entrevistador:	[Nombres de los Maestrandos]
Entrevistado (Código/Iniciales):	JO-01 (Jefe de Operaciones de TI)
Área/Departamento:	Dirección Nacional de Tecnología - Departamento de Infraestructura y Operaciones

Objetivo de la entrevista: Analizar la frecuencia de despliegues globales (DF), las restricciones de disponibilidad de la plataforma on-premise, la existencia de silos departamentales y el impacto financiero del licenciamiento de software privativo.

Bloque 1: Capacidad Operativa y Frecuencia de Despliegues (DF)

1. Desde la perspectiva de operaciones, ¿cuál es la cadencia actual de los pases a producción de nuevas versiones aplicativas o normativas? ¿Por qué se ha establecido una frecuencia quincenal/semanal en lugar de diaria?
2. ¿Qué limitaciones impone la infraestructura física actual para habilitar un esquema de despliegue continuo bajo demanda?

3. Describa el impacto de la coordinación interdepartamental en los tiempos de entrega. ¿De qué manera el aislamiento o "silos" entre los departamentos de Desarrollo, Operaciones, Seguridad y Planificación influye en la velocidad del ciclo de vida del software?
Bloque 2: Disponibilidad, SLAs e Impacto Financiero 1. ¿Cuáles son los acuerdos de nivel de servicio (SLA) de disponibilidad exigidos para los sistemas críticos (facturación electrónica, catastro) y cómo se ven afectados por las ventanas de mantenimiento manual? 2. Describa las ventanas de mantenimiento en producción: ¿por qué requieren ejecutarse fuera de la jornada laboral y qué personal debe estar presente de forma síncrona? 3. ¿Cómo evalúa el coste total de propiedad (TCO) y la sostenibilidad del modelo de licenciamiento actual de RedHat Enterprise y JBoss EAP frente a un eventual modelo de nube híbrida open-source?

Instrumento A.3: Guía de Entrevista para el Release Manager

Metadatos de la Sesión	Detalle de Registro
Fecha de Aplicación:	[Insertar Fecha]
Hora de Inicio / Fin:	[Insertar Hora]
Entrevistador:	[Nombres de los Maestrandos]
Entrevistado (Código/Iniciales):	RM-01 (Release Manager / Jefe de Control de Configuraciones)
Área/Departamento:	Departamento de Desarrollo - Área de Control de Calidad y Configuraciones
Objetivo de la entrevista: Evaluar el ciclo extremo a extremo de entrega de cambios (*Lead Time for Changes* - LTC), la tasa de fallos en pases de producción (*Change Failure Rate* - CFR), los cuellos de botella en la aprobación de cambios y la gestión de regresiones (*rollbacks*).	
Bloque 1: Tiempo de Ciclo de Entrega de Cambios ($\boldsymbol{LTC}$) 1. Una vez que un cambio normativo o correctivo de código es confirmado (*commit*) por el desarrollador en el repositorio compartido, ¿cuál es el trayecto y los pasos manuales (pruebas, empaquetado, aprobaciones) que debe superar antes de operar en producción?	

2. Detalle los cuellos de botella burocráticos y técnicos que causan que el *Lead Time for Changes* (LTC) supere los 5 días hábiles en la institución.
3. ¿Cómo se gestiona el control de versiones en las configuraciones y manifiestos de despliegue? ¿Existe trazabilidad inmutable entre el código compilado y el artefacto desplegado?

Bloque 2: Tasa de Fallos en Cambios (CFR) y Gestión de Rollbacks

1. Históricamente, ¿qué porcentaje de los pases a producción programados registran fallas de estabilidad, errores de configuración o inconsistencias funcionales que obligan a realizar una reversión (*Change Failure Rate*)? ¿Por qué esta cifra se estima en un 18% mensual?
2. ¿Cuál es el procedimiento técnico y el nivel de esfuerzo requerido para ejecutar un *rollback* de una aplicación monolítica en JBoss cuando un despliegue falla en caliente?
3. ¿Qué tipo de pruebas de regresión, integración o escaneos de vulnerabilidades de seguridad se realizan de manera automatizada antes de autorizar un paso a producción? ¿Cómo influye la falta de pipelines unificados de CI/CD en la calidad final de las entregas?

Anexo B.

Anexo A. Código Terraform AWS